

Werner Hartmann
Stefanie Jäckel
Myke Näf
Raimond Reichert

Informatikunterricht planen und durchführen

2. Auflage

OPEN ACCESS



Springer Vieweg

Informatikunterricht planen und durchführen

Werner Hartmann • Stefanie Jäckel •
Myke Näf • Raimond Reichert

Informatikunterricht planen und durchführen

2. Auflage



Springer Vieweg

Werner Hartmann
Wettingen, Schweiz

Stefanie Jäckel
Hermsdorf, Deutschland

Myke Näf
Zürich, Schweiz

Raimond Reichert
Uster, Schweiz



ISBN 978-3-662-72785-0 ISBN 978-3-662-72786-7 (eBook)
<https://doi.org/10.1007/978-3-662-72786-7>

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <https://portal.dnb.de> abrufbar.

© Der/die Herausgeber bzw. der/die Autor(en) 2006, 2026

Dieses Buch ist eine Open-Access-Publikation.

Open Access Dieses Buch wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor*in(nen) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Buch enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des/der betreffenden Rechteinhaber*in einzuholen.

Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede kommerzielle Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des/der Autor*in und ggf. des/der Herausgeber*in. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen. Der Verlag hat eine nicht-exklusive Lizenz zur kommerziellen Nutzung des Werkes erworben.

Die Wiedergabe von allgemein beschreibenden Bezeichnungen, Marken, Unternehmensnamen etc. in diesem Werk bedeutet nicht, dass diese frei durch jede Person benutzt werden dürfen. Die Berechtigung zur Benutzung unterliegt, auch ohne gesonderten Hinweis hierzu, den Regeln des Markenrechts. Die Rechte des/der jeweiligen Zeicheninhaber*in sind zu beachten.

Der Verlag, die Autor*innen und die Herausgeber*innen gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag noch die Autor*innen oder die Herausgeber*innen übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Planung/Lektorat: Leonardo Milla

Springer Vieweg ist ein Imprint der eingetragenen Gesellschaft Springer-Verlag GmbH, DE und ist ein Teil von Springer Nature.

Die Anschrift der Gesellschaft ist: Heidelberger Platz 3, 14197 Berlin, Germany

Wenn Sie dieses Produkt entsorgen, geben Sie das Papier bitte zum Recycling.

Vorwort

Was soll unterrichtet werden und wie soll unterrichtet werden? Diese Fragen stellen sich bei der Planung und Durchführung des Unterrichts in Informatik sowohl in der schulischen als auch in der betrieblichen Ausbildung. Zur Legitimation und zu den Inhalten eines Schulfaches Informatik gibt es viele Publikationen. Bisher liegen wenige praxisnahe Handreichungen zur Didaktik und Methodik des Informatikunterrichts vor. Hier setzt das vorliegende Buch an. Es bietet Unterstützung bei der Planung und Gestaltung des Unterrichts.

An wen richtet sich das Buch? Es wendet sich an Lehrkräfte ab der Sekundarstufe I und an weiterführenden Schulen, an Berufsschulen, an Kursleiterinnen und -leiter in der Aus- und Weiterbildung sowie an Lehramtsstudierende. Die unterschiedlichen Zielgruppen spiegeln sich in den gewählten Formulierungen und Beispielen wider. Die Leserinnen und Leser sind eingeladen, die Ausführungen auf die eigene Unterrichtspraxis zu übertragen.

Sind eine Didaktik und Methodik des Informatikunterrichts für Lehrerinnen und Lehrer ab der Sekundarstufe bis zu Schulungen im Rahmen der betrieblichen Weiterbildung überhaupt möglich? Aufgrund unserer langjährigen Erfahrung in der Informatikausbildung an verschiedenen Institutionen sind wir davon überzeugt. Die informatikspezifischen Probleme sind auf allen Stufen ähnlich: Informatik ist abstrakt, Informatik ist einem raschen Wandel unterworfen und Informatikunterricht beinhaltet Arbeit am Computer und mit Informatiksystemen.

Was bietet das Buch? Es liefert eine Reihe von Anleitungen zur Planung und Durchführung von Informatikunterricht.

Abb. 1 zeigt die übergeordnete Struktur des Buchs. Die einzelnen Kapitel sind gleich aufgebaut: Ein Beispiel aus dem Unterrichtsalltag und eine kompakte Übersicht bilden die Einleitung. Anschließend folgen eine Analyse und Empfehlungen sowie eine kurze Zusammenfassung. In den meisten Kapiteln werden die Ausführungen anhand konkreter Beispiele aus dem Unterricht illustriert, die von den Autoren erprobt wurden. Die Beispiele erstrecken sich über das ganze Themenspektrum des Informatikunterrichts. Sie richten sich an verschiedene Zielgruppen auf unterschiedlichen Altersstufen und sollen Ideen und Anregungen für den eigenen Unterricht vermitteln. Bei den Literaturangaben verzichten wir meist auf Weblinks und vermeiden so das Risiko, dass diese im Laufe der Zeit ungültig werden.

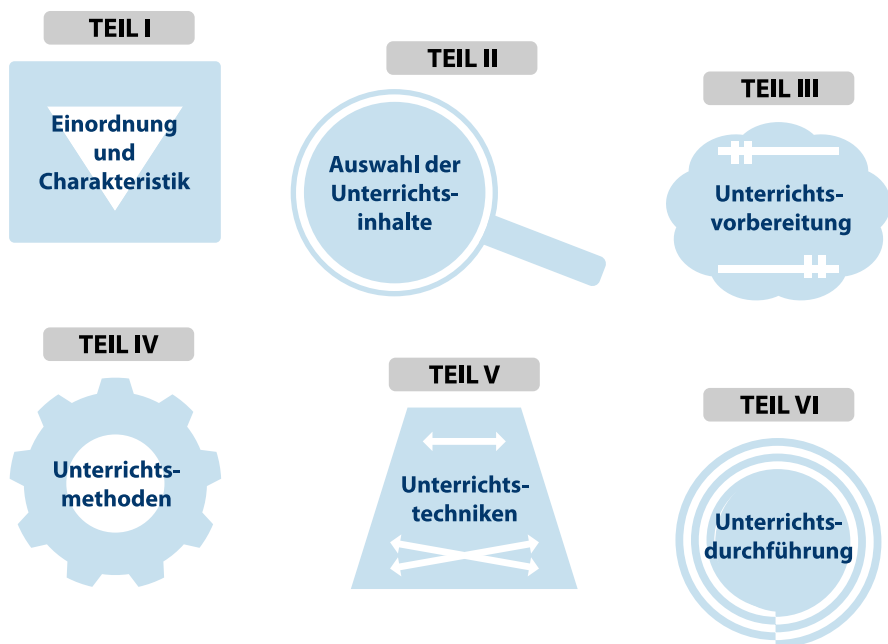


Abb. 1 Inhaltliche Struktur des Buchs

Bei Bedarf lassen sich diese weiterführenden Informationen schnell über Suchmaschinen finden.

Was bietet das Buch nicht? Es ersetzt nicht die Lektüre von Literatur zur allgemeinen Didaktik. Wir gehen davon aus, dass die Leserinnen und Leser bereits verschiedene Unterrichtsmethoden und -techniken kennen. Dazu gibt es gute Standardwerke. Die Auswahl der im Buch behandelten methodisch-didaktischen Aspekte ist zudem bewusst exemplarisch gehalten. Das Buch erhebt auch nicht den Anspruch, für die aufgeworfenen Probleme die einzig richtige, wissenschaftlich abgestützte Lösung anzubieten. Unterricht ist ein komplexes Gebilde, das sich nur schwer in allen Teilen erfassen und – um in der Sprache der Informatik zu bleiben – modellieren und implementieren lässt.

Wer steckt hinter dem Buch? Das Autorenteam verfügt über umfangreiche Unterrichtserfahrung in verschiedenen Bildungsstufen und Einrichtungen beginnend ab Klasse 5 in den Sekundarstufen I und II, unter anderem an Gymnasien in Baden und Aarau (Schweiz) und in Schleiz (Deutschland), und an weiterführenden Schulen, auf der Hochschulstufe und in Fortbildungen für Firmen und Behörden.

Das Buch wäre aber nicht entstanden ohne die Ideen und Beispiele von Studierenden im Rahmen von Kursen zur Informatikdidaktik und Lehrveranstaltungen an der ETH Zürich, der PH Bern, der FSU Jena, dem Studienseminar Ostthüringen und der PH Schwyz. Viele Überlegungen und Beispiele stammen auch aus der Entwicklung von Lehrmitteln und Onlinelernumgebungen des Autorenteam.

Wie ist dieses Buch entstanden? Die erste Fassung erschien im Jahr 2006. Aufgrund der anhaltend großen Nachfrage nach dieser praxisorientierten Handreichung wurde das Autorenteam erweitert und das Buch vollständig überarbeitet. Dank der großzügigen finanziellen Unterstützung der Hasler Stiftung (Schweiz) kann diese Ausgabe nun als Open-Access-Publikation einem breiten Publikum frei zugänglich gemacht werden.

Wettingen, Schweiz
Hermsdorf, Deutschland
Zürich, Schweiz
Uster, Schweiz

Werner Hartmann
Stefanie Jäckel
Myke Näf
Raimond Reichert

Inhaltsverzeichnis

Teil I Einordnung und Charakteristik

1 Informatikunterricht hat Informatik als Gegenstand	3
Literatur.	10
2 Charakteristik des Informatikunterrichts	13
Digitalität	14
Systemdenken	15
Realisierbarkeit	16
Dynamik	17

Teil II Auswahl der Unterrichtsinhalte

3 Auswahlkriterien für Inhalte	21
Literatur.	23
4 Fachsystematik	25
Great Principles of Computing	26
Bildungsstandards Informatik der Gesellschaft für Informatik	27
The Big Ideas in Computer Science for K-12 Curricula.	28
Literatur.	29
5 Fundamentale Ideen	31
Literatur.	42
6 Konzept- und Produktwissen	43

Teil III Unterrichtsvorbereitung

7 Zielgruppengerechte Zugänge	51
Literatur.	53
8 Unterrichtsplanung	55
Literatur.	60
9 Planung von Informatikschulungen	63

Teil IV Unterrichtsmethoden

10 Methodenvielfalt	69
Literatur.	71
11 Lernaufgaben	73
Literatur.	77
12 Puzzlemethode	79
Literatur.	84
13 Pair Learning	87
14 Rollenspiele	93
Freie Rollenspiele	94
Strukturierte Rollenspiele	96
15 Leitprogramme	101
Literatur.	104
16 Entdeckendes Lernen	105
Literatur.	109
17 Projektunterricht	111
Literatur.	117

Teil V Unterrichtstechniken

18 Motivation fördern	121
Inhalt – Bedeutsamkeit	122
Methode – Herausforderung	124
Produkt – Praxisbezug	126
Literatur.	127
19 Advance Organizer	129
Literatur.	132
20 Repräsentationstrias	135
Literatur.	141
21 Visualisieren, animieren und simulieren	143
Literatur.	147
22 Erst lesen, dann schreiben	149
Literatur.	153

Teil VI Unterrichtsdurchführung

23 Auswahl digitaler Werkzeuge	157
Didaktische und pädagogische Aspekte	158
Technische Aspekte	159
Aspekte des Supports	159

24	Entspannter unterrichten.	161
25	Mit Fehlern umgehen	167
	Literatur.	172
26	Fallstricke beim Prüfen	173
	Stichwortverzeichnis.	179

Teil I

Einordnung und Charakteristik

Informatikunterricht hat Informatik als Gegenstand

1

Einstieg: „Informatikunterricht flächendeckend eingeführt!“ Gymnasiallehrer N. ärgert sich über diese Zeitungsmeldung. Kaum steht an seiner Schule allen Schülerinnen und Schülern ein persönliches Tablet oder Notebook zur Verfügung, schon wird von Informatikunterricht gesprochen. Aber nicht nur das, die Schulleitung möchte die bisherige Lernplattform ersetzen, er soll die verschiedenen aktuellen Plattformen evaluieren und einen Vorschlag unterbreiten. Und: Die Musiklehrerin nutzt seit Neuestem eine webbasierte Übungssoftware für Chorstimmen. Ob N. nicht mal schauen könne, wie man die Abspielgeschwindigkeit reduziere. Lauter stressige Fragen ... Er ist doch Informatiklehrer und nicht Ratgeber für alles Digitale.

Digitale Werkzeuge kommen als Informatikanwendungen in allen Schulen und auf allen Schulstufen zum Einsatz, beispielsweise als Standardsoftware, fachspezifische Werkzeuge, Lernsoftware oder Lernplattformen. Oft werden all diese Nutzungen fälschlicherweise unter der Bezeichnung „Informatik“ zusammengefasst und Informatiklehrkräfte als zuständig für alle Fragen und Probleme rund um Digitales betrachtet. Gefragt ist eine klare begriffliche Abgrenzung, was unter Informatikunterricht zu verstehen ist und welche Rolle der Informatikunterricht für das Verständnis einer digital geprägten Welt hat.

Den Informations- und Kommunikationstechnologien (ICT) kommen im Bildungswesen unterschiedliche Rollen zu. Eine klare Unterscheidung dieser Rollen ist wichtig und bildet die Grundlage für eine vertiefte Auseinandersetzung mit Informatikunterricht. Abb. 1.1 zeigt die fünf Rollen von ICT in der Bildung.

ICT als Werkzeug im Alltag

Verbreitete Anwendungen wie Textverarbeitung, Tabellenkalkulation, Grafikprogramme, Übersetzungsprogramme und Kooperationsplattformen – um nur ein paar Beispiele zu nennen – werden sowohl im Berufsalltag als auch privat genutzt.

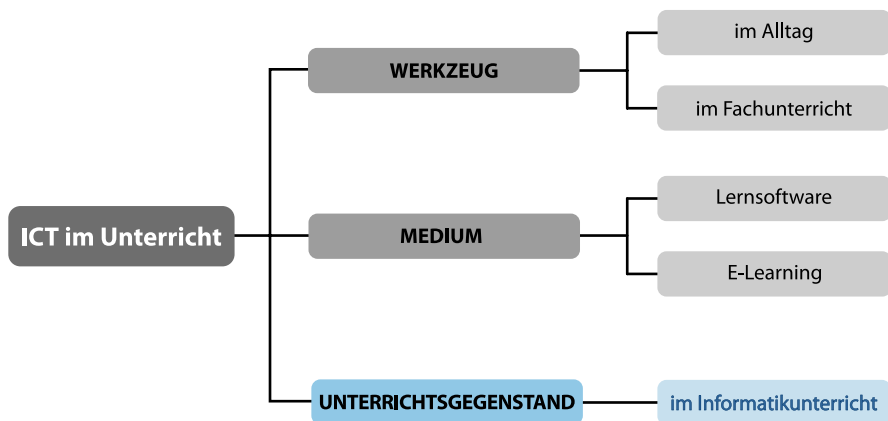


Abb. 1.1 Verschiedene Rollen von ICT im Unterricht

Für die Bedienung dieser Werkzeuge braucht es keine spezifischen Informatikkenntnisse. Für die fortgeschrittene, effiziente Nutzung ist aber ein Verständnis grundlegender informatischer Konzepte eine Voraussetzung. Dieses Verständnis fehlt vielen Anwenderinnen und Anwendern und verursacht insbesondere in der Berufswelt unnötige Probleme und Kosten. Ziel einer nachhaltigen Anwenderschulung ist es, die Effizienz und Kompetenz der Nutzer im Umgang mit digitalen Technologien zu erhöhen.

ICT als Werkzeug im Fachunterricht

Im Unterricht kommen neben Alltagswerkzeugen auch fachspezifische Werkzeuge zum Einsatz: im Mathematikunterricht etwa Computeralgebrasysteme, im Chemieunterricht Software zur Molekülmodellierung, im Geografieunterricht geografische Informationssysteme und im Kunstunterricht oder beim Bildnerischen Gestalten Bildbearbeitungsprogramme. Die Nutzung dieser Werkzeuge ist anspruchsvoller als das Benutzen der verbreiteten Anwendungsprogramme und setzt ein fundiertes Verständnis der Grundlagen des betreffenden Faches voraus. Mathematiksoftware nimmt zwar aufwendige Berechnungen ab, nicht aber die Wahl einer Lösungsstrategie oder das Interpretieren der berechneten Resultate. Ob sich eine bestimmte Software zur Molekülmodellierung im Chemieunterricht nutzbringend einsetzen lässt, muss von der Chemielehrerin entschieden werden und nicht vom Informatiklehrer. Die Lehrkräfte aller Fächer benötigen deshalb zwingend selbst digitale Kompetenzen. Im Informatikunterricht kommt der Nutzung von ICT-Werkzeugen naheliegenderweise eine besonders wichtige Rolle zu.

ICT als Medium in Form von Lernsoftware

Die multimedialen und interaktiven Möglichkeiten des Computers werden im Unterricht aller Fächer vielfältig zur Unterstützung von Lernprozessen genutzt, vom einfachen Vokabeltrainer über interaktive Lernumgebungen zu physikalischen Experimenten bis hin zur Simulation wirtschaftlicher Zusammenhänge. Der Computer übernimmt teilweise die Rolle der Lehrkraft als Vermittler, interagiert mit den

Lernenden und fördert individualisierte Lernprozesse im Unterricht. Auch im Informatikunterricht werden Lernumgebungen genutzt, zum Beispiel für den Einstieg ins Programmieren oder die Simulation von Netzwerken.

ICT als Medium beim E-Learning

E-Learning bezieht sich auf den gesamten Prozess des Lernens, der vollständig oder größtenteils online stattfindet. Dies umfasst verschiedene Formen und Methoden, wie zum Beispiel Onlinekurse, Webinare, virtuelle Klassenzimmer und Lernplattformen. E-Learning bietet für den Unterricht Möglichkeiten zur Kommunikation und Kooperation. Lernplattformen unterstützen nicht nur den Austausch von Unterrichtsmaterialien, sondern ermöglichen flexible und individualisierte, synchrone und asynchrone sowie orts- und zeitunabhängige Lernprozesse. Besonders für den Informatikunterricht gibt es vielfältige und variantenreiche E-Learning-Angebote, zum Beispiel Plattformen zum kollaborativen Programmieren oder Onlinekurse für ganze Themenbereiche.

ICT als Unterrichtsgegenstand im Informatikunterricht

Neben ihrer Rolle als Werkzeug und als Medium zur Über- und Vermittlung von Informationen sind Informations- und Kommunikationstechnologien selbst Unterrichtsgegenstand. Informatikunterricht beschäftigt sich mit den Grundlagen der Informatik, einschließlich Programmierung, Algorithmen, Datenstrukturen, Netzwerken und theoretischen Konzepten, aber auch Themen rund um Mensch und Gesellschaft. Ziel ist es, ein tiefes Verständnis für die Funktionsweise von Informationssystemen zu vermitteln sowie Problemlösungsfähigkeiten und logisches Denken zu fördern.

Die klare Abgrenzung des Informatikunterrichts gegenüber dem Einsatz von Standardsoftware, fachspezifischen Werkzeugen, Lernsoftware und E-Learning steckt auch die Aufgaben von Informatiklehrkräften ab. Sie sind nicht zuständig für allgemeine Fragen rund um die pädagogisch-didaktische Nutzung von ICT-Werkzeugen in anderen Unterrichtsfächern, die Auswahl von Lernsoftware oder die Entwicklung und Umsetzung einer E-Learning-Strategie. Informatiklehrkräfte sind auch nicht zuständig für Wartungs- und Supportaufgaben. Die Planung und Umsetzung einer professionellen IT-Infrastruktur sowie die Regelung der Wartungs- und Supportaufgaben fallen in den Zuständigkeitsbereich der Schulleitung und des Schulträgers.

Im Informatikunterricht geht es um die Auseinandersetzung mit Informations- und Kommunikationstechnologien als Unterrichtsgegenstand. In einer von Digitalität geprägten Welt darf sich der Informatikunterricht aber nicht nur isoliert auf technologische Aspekte beschränken. Um den Bildungsauftrag zu erfüllen, müssen verschiedene Perspektiven eingenommen werden. So ist es etwa schwierig, Informatikunterricht von Medienbildung abzugrenzen. Beispiel: Eine fundierte Auseinandersetzung mit Themen wie Fake News, Bias und Stereotypen in Chatbots oder Datenschutz und Datensicherheit bedingt sowohl ein fundiertes informatisches Verständnis als auch Wissen aus der Medienbildung.

Abb. 1.2 Dagstuhl-Dreieck (Eigene Darstellung)



Auch die Abgrenzung zwischen Informatikunterricht und Anwenderschulung ist nicht scharf. Beispiel: Die kompetente Nutzung einer Tabellenkalkulationssoftware, das Formulieren guter Prompts für Chatbots oder das Identifizieren von Fehlerquellen im häuslichen Netzwerk erfordert sowohl ein konzeptionelles Verständnis für die Funktionsweise dieser Informatiksysteme als auch produktbezogenes Anwenderwissen.

Als hilfreich für die Auseinandersetzung mit diesen und ähnlichen Fragen hat sich das Instrument des Dagstuhl-Dreiecks etabliert. Das Dagstuhl-Dreieck (siehe Abb. 1.2) wurde 2016 auf Schloss Dagstuhl anlässlich eines Treffens von Expertinnen und Experten der Informatik, der Informatikdidaktik, der Medienbildung und der Medienpädagogik sowie der Wirtschaft unter Mitwirkung eines der Autoren dieses Buches entwickelt (Gesellschaft für Informatik, 2016). Es betont, dass beim Thema Digitalität in der Bildung verschiedene Sichtweisen eingenommen werden müssen. Für ein umfassendes Verständnis von Digitalität reichen eine einseitige Sicht auf technische oder gesellschaftliche Aspekte oder nur praktische Anwendungskompetenzen nicht.

In der Dagstuhl-Erklärung werden die drei Perspektiven wie folgt beschrieben:

„Die *technologische Perspektive* hinterfragt und bewertet die Funktionsweise der Systeme, die die digitale vernetzte Welt ausmachen. Sie gibt Antworten auf die Frage nach den Wirkprinzipien von Systemen, auf Fragen nach deren Erweiterungs- und Gestaltungsmöglichkeiten. Sie erklärt verschiedene Phänomene mit immer wiederkehrenden Konzepten. Dabei werden grundlegende Problemlösestrategien und -methoden vermittelt. Sie schafft damit die technologischen Grundlagen und Hintergrundwissen für die Mitgestaltung der digitalen vernetzten Welt.

Die *gesellschaftlich-kulturelle Perspektive* untersucht die Wechselwirkungen der digitalen vernetzten Welt mit Individuen und der Gesellschaft. Sie geht z. B. den Fragen nach: Wie wirken digitale Medien auf Individuen und die Gesellschaft? Wie kann man Informationen beurteilen, eigene Standpunkte entwickeln und Einfluss auf gesellschaftliche und technologische Entwicklungen nehmen? Wie können Gesellschaft und Individuen digitale Kultur und Kultivierung mitgestalten?

Die *anwendungsbezogene Perspektive* fokussiert auf die zielgerichtete Auswahl von Systemen und deren effektive und effiziente Nutzung zur Umsetzung individueller und kooperativer Vorhaben. Sie geht Fragen nach, wie und warum Werkzeuge ausgewählt und genutzt werden. Dies erfordert eine Orientierung hinsichtlich der vorhandenen Möglichkeiten und Funktionsumfänge gängiger Werkzeuge in der jeweiligen Anwendungsdomäne und deren sichere Handhabung.“

Durch das Einnehmen der Perspektiven im Dagstuhl-Dreieck kann im Informatikunterricht ein umfassendes Verständnis der Technologien, ihrer Anwendung und ihrer Auswirkungen auf Individuum und Gesellschaft erfolgen.

Informations- und Kommunikationstechnologien haben im Bildungswesen verschiedene Rollen: Werkzeug im Alltag oder im Fachunterricht, Medium zur Unterstützung des Lernprozesses und eigentlicher Unterrichtsgegenstand. Um Missverständnissen vorzubeugen, muss der Begriff „Informatikunterricht“ präzisiert werden: Informatikunterricht bezeichnet den Unterricht mit ICT als Unterrichtsgegenstand. Dabei gilt es, sowohl die technologische als auch die gesellschaftlich-kulturelle und die anwendungsbezogene Perspektive angemessen zu berücksichtigen.

Beispiel 1: Suchmaschinen aus den Perspektiven des Dagstuhl-Dreiecks

Suchmaschinen sind heute ein täglicher Begleiter unseres Lebens. Sie ermöglichen schnellen und einfachen Zugang zu einer riesigen Menge an Informationen und Wissen. Durch die Personalisierung der Suchergebnisse liefern sie uns zielgerichtete Informationen, unterstützen Unternehmen aber auch beim Onlinemarketing. Suchmaschinen sind ein schönes Beispiel für die Betrachtung aus den drei Perspektiven des Dagstuhl-Dreiecks.

Technologische Perspektive:

- Wie kann eine Suchmaschine in einer Sekunde Millionen von Treffern liefern?
- Nach welchen Kriterien werden die Treffer in der Rangliste angeordnet?
- Welche Inhalte des Internets können von Suchmaschinen nicht erschlossen werden?
- Was versteht man unter einem Crawler und unter dem Index einer Suchmaschine?
- Was versteht man unter vertikalen und horizontalen Dokumentensammlungen?
- Wie sind die Begriffe „Ausbeute“ und „Präzision“ definiert?
- ...

Gesellschaftlich-kulturelle Perspektive:

- Wie finanzieren sich die Betreiber von Suchmaschinen?
- Beeinflussen Websites durch Bezahlung ihre Platzierung in den Suchergebnissen?
- Warum werden mir Werbeanzeigen zu kürzlich gesuchten Dingen angezeigt?
- Wie groß ist die Gefahr, dass ich mich durch die Nutzung einer Suchmaschine in einer Filterblase bewege?
- Welche persönlichen Daten gebe ich bei einer Internetsuche preis?
- Wie groß ist die Gefahr, dass die Betreiber großer Suchmaschinen eine dominante Rolle in Gesellschaft und Wirtschaft übernehmen?
- ...

Anwendungsbezogene Perspektive:

- Welche Suchmaschine eignet sich für mein Informationsbedürfnis?
- Was sind gute Suchbegriffe, um möglichst präzise Antworten zu erhalten?
- Spielt die Groß- und Kleinschreibung bei Suchanfragen eine Rolle?
- Wie kann ich nach Bildern mit einer bestimmten Lizenz suchen?
- Wie kann ich die Suche auf Informationen in einer bestimmten Sprache einschränken?
- Welche Möglichkeiten habe ich, bei einer Internetsuche meine Privatsphäre zu schützen?
- ...

Beispiel 2: Digitale Bilder aus den Perspektiven des Dagstuhl-Dreiecks

Bilder prägen unseren Alltag in vielen Formen: Werbung, Medien und Social Media sind einige Beispiele. Sie lassen sich nach Funktion (z. B. Informationsvermittlung, Unterhaltung), Motiven (z. B. Porträts, Reportagen) oder Technik (z. B. Fotografie, Zeichnung) kategorisieren. Aus den drei Perspektiven des Dagstuhl-Dreiecks betrachtet, stellen sich Fragen wie die folgenden:

Technologische Perspektive:

- Was ist der Unterschied zwischen Raster- und Vektorgrafiken?
- Wie werden Bilder digital codiert, gespeichert und übertragen?
- Welche digitalen Bildformate eignen sich für Webseiten?
- Was ist der Unterschied zwischen verlustfreier und verlustbehafteter Bildkompression?
- Was versteht man unter Farbraum und Farbtiefe?
- Wie hängen Auflösung und Dateigröße zusammen?
- ...

Gesellschaftlich-kulturelle Perspektive:

- Welche Rolle spielen Bilder in der Geschichte?
- Was zeichnet gute Fotografien aus?

- Welche Rechte sind beim Veröffentlichen von Bildern zu beachten?
- Inwieweit kann man Bildern glauben?
- Welche Gründe gibt es für Bildmanipulationen?
- Wieso sind Stereotypen in Bildern problematisch?
- ...

Anwendungsbezogene Perspektive:

- Was sind die Vor- und Nachteile verschiedener Bildformate?
- Welche Auflösung macht für Fotos auf dem Smartphone Sinn?
- Wie kann ich die Auflösung eines Fotos reduzieren?
- Wo finde ich lizenzfreie Clipart-Bilder im Internet?
- Wie kann ich Bilder in einer Präsentation animieren?
- Wie kann ich eine Bildschirmaufnahme machen und bearbeiten?
- Welches einfach zu bedienende Onlinebildbearbeitungsprogramm eignet sich für mein Vorhaben am besten?
- ...

Beispiel 3: Generative Sprachmodelle aus den Perspektiven des Dagstuhl-Dreiecks

Generative Sprachmodelle sind wichtige Helfer im Alltag, indem sie zum Beispiel das Verfassen von E-Mails, Berichten und anderen Texten, aber auch das Erstellen von Bildern oder Videos unterstützen. Sie können Sprachbarrieren abbauen und die globale Kommunikation vereinfachen. Genauso einfach können sie Falschinformationen produzieren und helfen, Menschen in die Irre zu führen. Der Berücksichtigung ethischer und datenschutzrechtlicher Aspekte kommt ein großer Stellenwert zu. Aus den drei Perspektiven des Dagstuhl-Dreiecks betrachtet, ergeben sich Fragen wie die folgenden:

Technologische Perspektive:

- Wie generieren Sprachmodelle zur Eingabe (Prompt) einen möglichst passenden Text?
- Wie kann ein Sprachmodell den Kontext einer Frage abstecken?
- Was versteht man unter den Begriffen „Tokens“, „Temperatur“ oder „Pre-Trained Transformer“ bei Sprachmodellen?
- Wie können Sprachmodelle neben Texten auch Bilder generieren?
- Wie funktionieren neuronale Netze?
- ...

Gesellschaftlich-kulturelle Perspektive:

- Geht mit von Sprachmodellen erzeugten Texten unsere menschliche Kreativität verloren?
- Wie kann man der Gefahr durch Sprachmodelle erzeugter Stereotypen, Gender Bias oder Fake News begegnen?

- Wie verändern generative Sprachmodelle die Berufswelt?
- Wo liegt die Grenze zum Plagiat bei der Nutzung von generativen Sprachmodellen?
- Wie viel Energie verbrauchen Training und Nutzung von großen Sprachmodellen?
- ...

Anwendungsbezogene Perspektive:

- Wo finde ich für meine Zwecke geeignete generative Sprachmodelle?
- Wie kann ich präzise und klare Anfragen formulieren, um von Sprachmodellen möglichst gute Antworten zu erhalten?
- Wie kann ich von Sprachmodellen eine Rückmeldung zu von mir verfassten Texten erhalten?
- Wie kann ich die gelieferten Antworten kritisch auf ihre Glaubwürdigkeit hinterfragen?
- Muss ich die Nutzung beim Verfassen einer Arbeit ausweisen und wie mache ich das?
- Wie können mich generative Sprachmodelle beim Finden kreativer Ideen unterstützen?
- Welche persönlichen Daten gebe ich bei der Nutzung generativer Sprachmodelle preis?
- ...

Literatur

GI (Gesellschaft für Informatik) (2016). *Dagstuhl-Erklärung – Bildung in einer digitalen vernetzten Welt*. <https://dagstuhl.gi.de/>. Zugegriffen am 05.03.2025.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: L. und F. diskutieren, ob sie im nächsten Schuljahr Chemie oder Informatik als Schwerpunkt wählen sollen. Beiden gefällt im Chemieunterricht die angewandte Arbeit im Labor. Im Informatikunterricht wird zwar ebenfalls in Projekten an der Lösung von konkreten Problemen gearbeitet, doch L. ist unsicher, ob Informatik nicht nur etwas für Nerds sei. Gemeinsam mit ihrer Informatiklehrerin diskutieren die beiden, was Informatikunterricht vom Unterricht in anderen Fächern unterscheidet.

Im Gegensatz zu traditionellen Fächern wie Mathematik, Geschichte oder Biologie, die auf über Jahrhunderte gereiftem Wissen und Methoden aufbauen, ist die Informatik eine vergleichsweise junge Disziplin. Der Informatikunterricht unterscheidet sich von anderen Fächern durch seinen Fokus auf das abstrakte Denken, die Problemlösung durch algorithmische Ansätze und die produktorientierte Gestaltung digitaler Technologien.

Überall steckt Informatik drin. Es ist faszinierend, wie sich mit Nullen und Einsen als simplen Bausteinen der digitalen Welt komplexe und vielseitige Anwendungen realisieren lassen. Aufgabe des Informatikunterrichts ist es, die Funktionsweise moderner Informatiksysteme verständlich zu vermitteln.

Abb. 2.1 zeigt ohne Anspruch auf Vollständigkeit eine Reihe von Aspekten, die den Informatikunterricht besonders kennzeichnen. Manche dieser Aspekte treffen auch für andere Fächer zu.

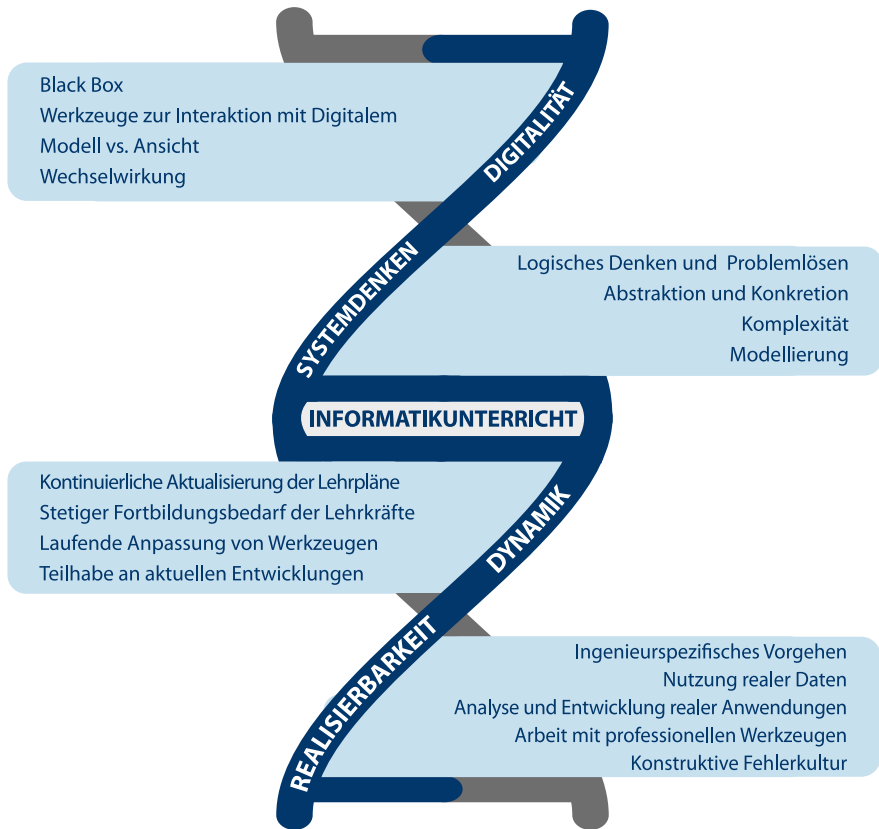


Abb. 2.1 Charakteristische Merkmale des Informatikunterrichts

Digitalität

Im Zentrum der Informatik stehen die digitale Darstellung, Erkennung, Verarbeitung, Speicherung und Übertragung von Information. Die Digitalität stellt eine spezifische Herausforderung im Informatikunterricht dar, da Lerngegenstände und Phänomene oft nicht direkt beobachtbar sind.

Black Box Prozesse wie die Ausführung eines Programms von der höheren Programmiersprache bis zum Maschinencode, der Kontrollfluss in Programmen, die Speicherverwaltung, Netzwerkprotokolle oder die Realisierung von Mikroprozessoren sind für Lernende unsichtbar und schwer nachvollziehbar. Die Lernenden begegnen vielen Objekten und Prozessen nur in Form von digitalen Artefakten auf dem Bildschirm. Der Abstraktionsgrad in der Informatik hat sich im Laufe ihrer Entwicklung kontinuierlich erhöht. Während zum Beispiel zu Beginn in Maschi-

nensprache programmiert wurde, kommen heute höhere Programmiersprachen zum Einsatz.

Werkzeuge zur Interaktion mit Digitalem Im Informatikunterricht sind digitale Medien, Werkzeuge und Informatiksysteme nicht nur Unterrichtsgegenstand, sie werden auch genutzt, um Objekte und Prozesse für den Lernprozess sichtbar und bearbeitbar zu machen. Zum einen sind das Werkzeuge wie Grafikprogramme, Programmierungsumgebungen oder Webbrowser, die digitale Artefakte visualisieren und bearbeitbar machen. Zum anderen Werkzeuge wie Mikrocontroller oder Netzwerksimulationen, die Prozesse steuerbar machen oder simulieren.

Modell vs. Ansicht Ein Tisch in einem Computerspiel sieht zwar aus wie ein physischer Tisch, besteht aber aus einer Ansammlung von Daten, die erst durch die Aufbereitung der Spiele-Engine oder des Bildbearbeitungsprogramms für einen Menschen sichtbar werden. Der Quellcode einer Webseite liegt nur als Textdatei auf einem Webserver vor und wird erst durch die Interpretation eines Webbrowsers zu einer anschaulichen Webseite. Schülerinnen und Schüler müssen lernen, mit der Abstraktheit digitaler Artefakte umzugehen und Modelle zur Lösung von Problemen zu nutzen, die in der realen Welt nur beschränkt bearbeitet werden können.

Wechselwirkung Information ist zentral für die Gesellschaft, Wirtschaft und Wissenschaft. Die Informatik steht deshalb in enger Wechselwirkung mit nahezu allen Lebensbereichen. Viele Themen des Informatikunterrichts werfen Fragen auf, die weit über das eigentliche Fachgebiet hinausgehen und ethische, ökologische, soziale, sicherheitsrelevante sowie rechtliche Dimensionen umfassen. Diese Aspekte müssen in einem übergeordneten gesellschaftlichen Kontext betrachtet und behandelt werden. Dafür benötigen Informatiklehrkräfte fachfremde Kompetenzen.

Systemdenken

Grundlegende Denk- und Problemlösefähigkeiten prägen die Informatik maßgeblich. Sie umfassen den Umgang mit komplexen Systemen, die Fähigkeit zur Abstraktion und Konkretion sowie das Verständnis und die Modellierung von realen und virtuellen Welten. Im Informatikunterricht lernen die Schülerinnen und Schüler, komplexe Probleme strukturiert anzugehen, logische Schlussfolgerungen zu ziehen und kreative Lösungen zu entwickeln.

Logisches Denken und Problemlösen In der Informatik müssen Prozesse und Daten formalisiert und in einer präzisen, eindeutigen Sprache dargestellt werden, die Computer verstehen. Logisches Denken ist unerlässlich, um klare Notationen und Semantiken zu entwickeln.

Abstraktion und Konkretion In der Informatik sind Fachinhalte oft abstrakt, während die Ergebnisse konkret sind. Die Fähigkeit, beim Lösen konkreter Probleme

zwischen abstrakten Darstellungen und realen Umsetzungen zu wechseln, ist entscheidend und muss im Unterricht besonders gefördert werden, etwa bei der Entwicklung von Algorithmen oder Datenstrukturen.

Komplexität Informatikprobleme sind oft komplex, da sie mehrere Ebenen der Abstraktion, große Mengen an Daten oder viele Interaktionen zwischen verschiedenen Komponenten umfassen. Allgemeine Problemlösungsmethoden wie Teile-und-herrsche, Visualisierung und Modellierung oder Arbeiten in Teams sind geeignete Strategien, um den Lernenden zu helfen, komplexe Probleme zu verstehen und systematisch zu lösen.

Modellierung Modellierung ist ein wichtiges Entwurfsmuster in der Informatik. Die Herausforderung besteht darin, die Komplexität der realen Welt in aussagekräftige Modelle zu übertragen. Im Unterricht müssen diese Fähigkeiten durch schrittweises Vorgehen, visuelle Hilfsmittel, spezialisierte digitale Werkzeuge und praxisnahe Projekte gezielt gefördert werden.

Realisierbarkeit

Die Möglichkeit, im Informatikunterricht theoretisches Wissen unmittelbar in praxisnahe Anwendungen zu übertragen, trägt zu einem besseren Verständnis der Konzepte und einer höheren Motivation der Schülerinnen und Schüler bei. Die praktische Umsetzung kann mehr Zeit in Anspruch nehmen als geplant, da unerwartete Fehler oder Probleme mit der technischen Infrastruktur auftreten können.

Ingenieurspezifisches Vorgehen Ansätze, Methoden und Werkzeuge aus den Ingenieurwissenschaften sind Teile des Informatikunterrichts und helfen, analytisch zu denken, technische Probleme präzise zu lösen sowie Prozesse zu optimieren. Die Vorgehensweisen aus den Ingenieurwissenschaften legen einen besonderen Fokus auf Qualitätssicherung, fördern Kreativität und Teamarbeit und erfordern spezifische Unterrichtsmethoden.

Nutzung realer Daten Der Zugang zu realen Daten ermöglicht es Lernenden, aktuelle Datensätze wie Flugbewegungen, Klimadaten oder Social-Media-Beziehungen mit geeigneten Werkzeugen zu analysieren. Sie können auch einfache KI-Modelle entwickeln und mit realen Daten trainieren. Der Umgang mit echten Datensätzen fördert sowohl das technische Verständnis als auch das Bewusstsein für ethische und sicherheitsrelevante Aspekte der Datenverarbeitung.

Analyse und Entwicklung realer Anwendungen Im Informatikunterricht werden Anwendungen analysiert und auch entwickelt. Das Entwickeln eigener Spielprogramme oder Apps, das Gestalten von Websites oder das Konstruieren kleiner Gadgets zur Überwachung der eigenen Fitness mithilfe von Sensoren und Mikrocontrollern fördern ein vertieftes Verständnis.

Arbeit mit professionellen Werkzeugen Im Informatikunterricht wird neben der Nutzung von Lernumgebungen auch mit professionellen Werkzeugen gearbeitet, zum Beispiel Programmierungsumgebungen oder Netzwerkanalysertools. Weil solche Werkzeuge eine große Einarbeitungszeit erfordern, wird oft mit didaktisch aufbereiteten, einfacheren Werkzeugen begonnen, bevor später die Profiwerkzeuge zum Einsatz kommen.

Konstruktive Fehlerkultur Im Informatikunterricht ist das Scheitern aufgrund des starken Praxisbezugs ein natürlicher und wichtiger Teil des Lernens. Die Nutzung professioneller Werkzeuge kann darüber hinaus die Fehleranalyse erschweren. Die Fähigkeit, systematisch Fehler zu finden und zu beheben, muss deshalb gezielt vermittelt werden.

Dynamik

Die Informatik entwickelt sich rasant. Neben dem zeitlosen Fundament müssen technologische und methodische Fortschritte kontinuierlich integriert werden, um den Unterricht interessant und zukunftsorientiert zu gestalten.

Kontinuierliche Aktualisierung der Lehrpläne Die Lehrpläne im Fach Informatik müssen regelmäßig überarbeitet und aktualisiert werden, um den Schülerinnen und Schülern eine zeitgemäße Ausbildung zu bieten, die sich an wissenschaftlichen und technologischen Entwicklungen orientiert.

Stetiger Fortbildungsbedarf der Lehrkräfte Lehrkräfte im Fach Informatik müssen sich kontinuierlich fortbilden, um mit den Entwicklungen Schritt zu halten. Neben klassischen Weiterbildungsformaten bieten sich auch der Austausch in Erfahrungsgruppen auf sozialen Netzwerken, Onlinekurse und Webinare, kollaborative Lernplattformen und IT-Unternehmen als schulische Kooperationspartner an.

Laufende Anpassung von Werkzeugen Werkzeuge erfordern regelmäßige Updates und müssen manchmal auch durch neue Werkzeuge ersetzt werden. Für Lehrkräfte kann die ständige Anpassung des eigenen Wissens und der Unterrichtsunterlagen an neue Werkzeuge und Versionen spannend, jedoch auch zeitaufwendig und belastend sein.

Teilhabe an aktuellen Entwicklungen Lernende können im Informatikunterricht an aktuellen Diskursen und technischen Innovationen teilhaben. Technologien wie KI, Smartwatches oder RFID-Systeme eignen sich als Lernobjekte. Digitale Dienste und Open-Source-Projekte sind leicht zugänglich. Plattformen wie GitHub, Stack Overflow und MOOCs bieten kostenfreie Lernmaterialien und Communitysupport.

Der Informatikunterricht zeichnet sich durch Digitalität, Systemdenken, Realisierbarkeit und Dynamik aus. Diese Charakteristika prägen den Informatikunterricht über alle Jahrgangsstufen und Ausbildungsgänge hinweg und müssen bei der Unterrichtsplanung und Durchführung berücksichtigt werden.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Teil II

Auswahl der Unterrichtsinhalte

Auswahlkriterien für Inhalte

3

Einstieg: Während H. seinen Unterricht zu Netzwerktopologien vorbereitet, hält er inne: Ständig tauchen neue Themen auf – Künstliche Intelligenz, Large Language Models, Blockchain und Quantencomputing. Was noch vor wenigen Jahren den Lehrplan prägte, scheint oft bereits veraltet. Auch die Schüler haben andere Erwartungen. Hanna hat ihm heute stolz ihr neues Smartphone gezeigt und wollte wissen, wie das mit der KI-basierten Fotooptimierung gehe. Angesichts der ständig neuen Themen und der knappen Vorbereitungszeit fragt er sich, wie er das alles bewältigen kann.

Die Informatik entwickelt sich sehr dynamisch; neue Technologien und Produkte entstehen in kurzen Abständen. Im Informatikunterricht ist die Gefahr groß, sich zu sehr von dieser Schnellebigkeit beeinflussen zu lassen. Die Auswahl von Unterrichtsinhalten ist deshalb eine anspruchsvolle Aufgabe.

Seit jeher entwickelt sich die Informatik rasant, was dazu führt, dass Lehrpläne und Unterrichtsmaterialien schnell veralten. Technologien, die heute relevant erscheinen, können in wenigen Jahren überholt sein. Ein weiteres Problem besteht darin, dass sich der Informatikunterricht leicht in produktspezifischen Details verlieren kann. Diese Details bieten zwar kurzfristigen Nutzen, doch sie vermitteln den Schülerinnen und Schülern kein nachhaltiges Verständnis der Informatik. Die Herausforderung besteht also darin, einen Unterricht zu gestalten, der sowohl die dauerhaften, fundamentalen Konzepte der Informatik als auch aktuelle technologische Entwicklungen abdeckt, ohne den Fokus zu stark auf das eine oder das andere zu legen.

Aus der Forschung zur Curriculumentwicklung stammen verschiedene Modelle zur Auswahl von Unterrichtsinhalten. Diese Modelle unterscheiden sich nach dem zugrunde liegenden Bildungsbegriff, der Schulstufe und der Zielgruppe. Ihre theoretische und komplexe Natur macht sie für den Alltag von Schulen und Lehrkräften

nur bedingt nutzbar. Ganz bewusst gehen wir deshalb nicht auf diese Modelle ein, sondern stellen in den folgenden Kapiteln drei Sichtweisen vor, die sich in der Praxis bei der Auswahl von Unterrichtsinhalten als hilfreich erwiesen haben.

Fachsystematik Fachsystematik bezeichnet die geordnete Strukturierung und Systematisierung der Inhalte, Konzepte und Prinzipien eines Fachgebiets. Sie verdeutlicht die Zusammenhänge zwischen den verschiedenen Elementen des Fachs und bildet eine Grundlage für die didaktische Aufbereitung und Vermittlung dieser Inhalte. Die Fachsystematik dient Lehrkräften als Orientierungshilfe, um alle relevanten Inhalte angemessen im Unterricht zu berücksichtigen. Gleichzeitig besteht jedoch die Gefahr, dass ein zu streng fachsystematischer Ansatz den Unterricht trocken und theoretisch erscheinen lässt. Der Nutzen vieler Konzepte in realen Situationen bleibt oft unklar, und anwendungsorientierte Aspekte werden vernachlässigt. Zudem kann der Fokus auf etablierte Prinzipien dazu führen, dass neuere Technologien und Innovationen zu wenig Beachtung finden.

Fundamentale Ideen Für die Lernenden ist eine enge Verbindung zu ihrer eigenen Lebenswelt und zu aktuellen Themen wichtig. Insbesondere in allgemeinbildenden Schulen muss man sich immer die Frage „So what?“ stellen: Welchen Beitrag zur Allgemeinbildung leistet der Unterricht? Als geeignetes Instrument zur Beantwortung dieser Frage haben sich die fundamentalen Ideen von Bruner (1960) bewährt, die von Schwill (1993) auf den Informatikunterricht adaptiert wurden. Das Instrument zielt darauf ab, zentrale Konzepte und Prinzipien der Informatik zu identifizieren, die das grundlegende Verständnis fördern und über die Zeit und spezifische Technologien hinweg relevant bleiben. Der Fokus der fundamentalen Ideen liegt zudem stark auf der Frage, wie den Lernenden die Bedeutsamkeit der Inhalte aufgezeigt werden kann und wie die Inhalte didaktisch aufbereitet und anschaulich vermittelt werden können.

Konzept- und Produktwissen Je nach Zielgruppe und Thema kommt dem Vermitteln theoretischer Grundlagen oder der Anwendung des Wissens eine andere Bedeutung zu. Beim Festlegen der Lernziele und der Auswahl der Lerninhalte ist es hilfreich, sich Klarheit über die zu vermittelnden Konzepte und die handlungsorientierten Teile im Unterricht zu verschaffen. Für das Verständnis und die Nutzung von Informatikmitteln ist sowohl Konzeptwissen als auch Produktwissen nötig. Konzeptwissen bezieht sich auf das Verständnis von grundlegenden Prinzipien, Theorien und Konzepten, die das Fundament einer Disziplin bilden. Dazu gehören Themen wie Algorithmen und Datenstrukturen, Abstraktion und Modularität, Rekursion oder formale Sprachen und Automaten. Produktwissen bezieht sich auf das Wissen über spezifische Technologien, Werkzeuge und Anwendungen, die in der Praxis verwendet werden. Es handelt sich um anwendungsorientiertes Wissen, das typischerweise schnell veraltet, da es eng mit aktuellen Technologien verbunden ist. Beispiele sind das Wissen über bestimmte (Versionen von) Programmiersprachen, Entwicklungsumgebungen, Frameworks und Bibliotheken.

Zur gezielten Auswahl der Unterrichtsinhalte stehen Informatiklehrkräften verschiedene in der Praxis erprobte Instrumente zur Verfügung. Die Anwendung dieser Instrumente hilft neben der Auswahl relevanter Themen auch bei der Aufbereitung der Fachinhalte und der späteren Unterrichtsgestaltung.

Literatur

Bruner, J. S. (1960). *The process of education*. Harvard University Press.

Schwill, A. (1993). Fundamentale Ideen der Informatik. *Zentralblatt für Didaktik der Mathematik*, 25(1), 20–31.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: H. unterrichtet eine 8. Klasse im Fach „Medien und Informatik“. Der Lehrplan gibt vor, dass er sowohl Kenntnisse zur Funktionsweise wichtiger Informatiksysteme als auch ein Bewusstsein für gesellschaftliche Fragen vermitteln soll. H. hat die Erfahrung gemacht, dass der Unterricht in dieser Schulstufe für die Schülerinnen und Schüler stark handlungsorientiert sein muss. In den vergangenen Jahren haben seine Klassen immer stärker den Wunsch geäußert, weniger Zeit für das Programmieren, dafür mehr für Künstliche Intelligenz einzuräumen. Die Vielfalt der Informatik stellt H. vor große Probleme. In Lehrmitteln, Zeitschriften und online findet er zu KI eine riesige Menge an Unterrichtsvorschlägen: neuronale Netze, maschinelles Lernen, Ethik, Bildmanipulation, Big Data, Fake News, Sprachmodelle, Sprachübersetzung, Trainieren eines Chatbots, autonome Fahrzeuge, Datenschutz usw. Wie soll er da mit vertretbarem Zeit- und Planungsaufwand und für die begrenzte Dauer von ca. 6–8 Lektionen eine sinnvolle Auswahl treffen?

Die große Bandbreite an Themen in der Informatik, die geringe Unterrichtszeit sowie überfrachtete Lehrpläne und Kompetenzkataloge stellen für Informatiklehrkräfte eine Herausforderung bei der Planung und Gestaltung von Unterricht dar.

Die Themenvielfalt kann sowohl Lehrkräfte als auch Schülerinnen und Schüler überfordern, insbesondere in einem engen Zeitrahmen. Lehrerinnen und Lehrer müssen Inhalte sorgfältig priorisieren, ohne dabei das Gesamtbild des Fachgebiets aus den Augen zu verlieren. Hinzu kommt, dass die Vielzahl an verfügbaren Lehrmaterialien, Konzepten und Themen es schwer macht, eine konsistente und kompakte Unterrichtsstruktur zu entwickeln, die sowohl den Bildungszielen als auch den Interessen der Lernenden gerecht wird. Es braucht daher eine didaktische Strukturierung, die einerseits eine systematische Einführung in zentrale Informatik-

konzepte bietet, andererseits aber auch genügend Raum lässt für aktuelle Themen und die Interessen der Schülerinnen und Schüler.

Ein systematischer Zugang bietet einen umfassenden Überblick über ein Fachgebiet, sodass Lernende die Breite des Fachs und die Verbindung zwischen verschiedenen Themenbereichen erkennen können. Ein systematischer Zugang entspricht zudem oft den Strukturen, die auch in der Wissenschaft verwendet werden. Dies bereitet die Lernenden auf weiterführende Studien oder Berufe vor. Wir stellen nachfolgend drei Strukturierungen vor, die sich für den Informatikunterricht auf unterschiedlichen Stufen bewährt haben.

Great Principles of Computing

Als Möglichkeit zur Strukturierung auf höheren Schulstufen, insbesondere auf der Stufe Hochschule, wird oft Bezug genommen auf die „Great Principles of Computing“ von Peter Denning (siehe Tab. 4.1). Das Ziel dieser Grundprinzipien ist es, die Sichtweise auf die Informatik über die technikzentrierte Sicht mit dem Computer im Mittelpunkt hinaus zu erweitern. Obwohl die Great Principles auf die Informatik an Hochschulen ausgerichtet sind, bieten sie für Lehrkräfte eine gute, langfristig gültige Strukturierung. Denning und Martell (2015, S. 12) unterteilen die Informatik in sechs Kategorien bzw. Kernbereiche: Kommunikation, Berechnung, Speicherung, Koordination, Evaluation und Design.

Neben diesen Kernbereichen strukturieren Denning und Martell die Informatik auch aufgrund von Anwendungsbereichen (engl. „domains“). Jeder Anwendungsbereich konzentriert sich auf eine Technologie oder eine Anwendung von Technologie. Im Gegensatz zu den Kernbereichen beschreiben die Anwendungsbereiche verschiedene Interessengebiete. Die in diesen Gebieten tätigen Personen nutzen für die Lösung ihrer Probleme Methoden aus den Kernbereichen der Informatik. Zum Beispiel werden im Bereich Cloud-Computing Methoden aus allen Kernbereichen genutzt (vgl. Denning und Martell (2015), S. 31), wie in Tab. 4.2 dargestellt.

Im Informatikunterricht der Sekundarstufe I und II steht in der Regel nicht genug Zeit zur Verfügung, um alle sechs „Great Principles of Computing“ abzudecken. Die Strukturierung in Kernbereiche eignet sich aber, um als Lehrkraft Lerninhalte

Tab. 4.1 Great Principles of Computing

Communication	Reliably moving information between locations
Computation	What can and cannot be computed
Collection	Representing, storing, and retrieving information from media
Coordination	Effectively using many autonomous computing agents
Evaluation	Measuring whether systems produce intended computations
Design	Structuring software systems for reliability and dependability

Tab. 4.2 Anwendungsbereich Cloud-Computing

Communication	Error-correcting codes, compressing sound and image files, location-based identification
Computation	Map-Reduce methods, massively parallel computing
Recollection	Replicating data, atomic transactions, transaction rollback, database structures, searching Internet, naming, digital object identifiers, digital object handles
Coordination	Locking protocols, rollback protocols, file-transfer protocols, file-syncing protocols, hypertext protocols, domain name system, time stamping, version control
Evaluation	Performance of massively parallel and massively distributed memory systems
Design	Interface design, data warehouse architecture

in einen Gesamtkontext einzuordnen und Parallelen zwischen vermeintlich losgelösten Sachverhalten aufzuzeigen. Sie vermitteln auch Sicherheit, dass die Lerninhalte wirklich relevant sind. Im Unterricht helfen die Prinzipien, den Lernenden die Bedeutung der behandelten Themen zu kommunizieren und die zugrunde liegenden Konzepte deutlich zu machen. Während sich die Bedeutung der Anwendungsbereiche in der Informatik schnell ändern kann, sind die Grundprinzipien langlebiger Natur.

Bildungsstandards Informatik der Gesellschaft für Informatik

Als Strukturierung für die Sekundarstufe I und II haben sich die Bildungsstandards Informatik der Gesellschaft für Informatik (GI) (2008) etabliert. Diese fokussieren auf die Kompetenzen und Fähigkeiten, die Schülerinnen und Schüler bis zu einem bestimmten Bildungsabschnitt erwerben sollen. Sie beschreiben die angestrebten Lernergebnisse und Fertigkeiten in der Art eines übergeordneten Rahmenlehrplans. Die Bildungsstandards umfassen ein Kompetenzmodell, Inhalts- und Prozessbereiche.

Die Prozessbereiche beschreiben, auf welche Art und Weise die Schülerinnen und Schüler mit den genannten Fachinhalten umgehen sollen. Prozessbereiche sind:

- Modellieren und Implementieren
- Begründen und Bewerten
- Strukturieren und Vernetzen
- Kommunizieren und Kooperieren
- Darstellen und Interpretieren

Die Inhaltsbereiche beschreiben die zu erwerbenden fachlichen Kompetenzen:

- Information und Daten
- Algorithmen

- Sprachen und Automaten
- Informatiksysteme
- Informatik, Mensch und Gesellschaft

Das Kompetenzmodell kombiniert die beiden Dimensionen Inhaltsbereich und Prozessbereich und fügt eine dritte Dimension der Anforderungsbereiche hinzu:

- Reproduktion
- Reorganisation und Transfer
- Reflexion und Problemlösung

Die Bildungsstandards und die „Great Principles of Computing“ haben ähnliche Ziele, unterscheiden sich jedoch in ihrem Ansatz und ihrer Zielgruppe. Dennings Werk richtet sich an ein breiteres Publikum, einschließlich Fachleute und Studierende in Wissenschaft und Technik, sowie an Praktiker in der Informatik. Die Bildungsstandards hingegen sind praxisorientiert und fokussieren auf die Vermittlung konkreter Kompetenzen und Inhalte, die im schulischen Informatikunterricht erforderlich sind. Sie bilden in Deutschland das zentrale Kompetenzmodell für den Informatikunterricht.

The Big Ideas in Computer Science for K-12 Curricula

Der Ansatz „Big Ideas in Computer Science“ von Tim Bell, Paul Tymann und Amiram Yehudai (2018) unterscheidet sich stark von den Great Principles und den Bildungsstandards. Er beschäftigt sich mit der Integration der Informatik in die Lehrpläne von der Vorschule bis zur 12. Klasse weltweit. Dabei stellen die Autoren zehn zentrale Ideen der Informatik vor, die Lehrkräften sowie Schülerinnen und Schülern helfen sollen, die grundlegenden Konzepte des Fachgebiets besser zu verstehen und zu vermitteln. Besonders im Bereich der Grundschule und Primarschule, wo Lehrkräfte oft Generalisten sind, spielen einfache Erklärungen grundlegender Prinzipien der Informatik eine wichtige Rolle. Wir geben hier die zehn Ideen in stark verkürzter Form wieder:

1. Informationen werden in digitaler Form dargestellt.
2. Algorithmen interagieren mit Daten, um Berechnungsprobleme zu lösen.
3. Die Effizienz von Algorithmen kann modelliert und bewertet werden.
4. Manche Berechnungsprobleme sind algorithmisch nicht lösbar.
5. Programme bringen Algorithmen und Daten in eine für den Computer ausführbare Form.
6. Digitale Systeme werden von Menschen entworfen, um menschliche Bedürfnisse zu erfüllen.
7. Digitale Systeme erzeugen virtuelle Darstellungen natürlicher und künstlicher Phänomene.

8. Der Schutz von Daten und Systemressourcen ist in digitalen Systemen entscheidend.
9. Zeitabhängige Operationen in digitalen Systemen müssen koordiniert werden.
10. Digitale Systeme kommunizieren über Protokolle miteinander.

Diese großen Ideen sollen Lehrerinnen und Lehrern helfen, die Informatik als langfristig relevantes und stabil bleibendes Fach zu begreifen. Sie bieten eine Orientierung, die über kurzfristige technologische Entwicklungen hinausgeht, und betonen die fundamentalen Konzepte der Informatik.

Ein fachsystematischer Zugang bietet wichtige pädagogische und fachliche Vorteile. Die Übersicht zu den Kernbereichen und Anwendungsbereichen der Informatik unterstützt Lehrkräfte bei der Planung und Gestaltung eines fundierten, praxisorientierten Unterrichts. Ein ausgewogener Unterricht sollte die systematische Struktur des Fachs berücksichtigen, aber auch lebensweltbezogene Probleme und Anwendungen integrieren, um die Motivation der Lernenden zu erhöhen.

Literatur

- Bell, T., Tymann, P., & Yehudai, A. (2018). The big ideas in Computer Science for K-12 curricula. *Bulletin of EATCS*, 1(124), 2–12.
- Denning, P. J., & Martell, C. H. (2015). *Great principles of computing*. MIT Press.
- GI (Gesellschaft für Informatik) (2008). *Bildungsstandards Informatik*. <https://informatik-standards.de/>. Zugegriffen am 17.03.2025.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: J. unterrichtet am Gymnasium Mathematik und Informatik. Für sie ist klar, dass Logik von zentraler Bedeutung ist. Sie bildet das Fundament der Mathematik und ist wichtig für das Verständnis mathematischer Beweise. Auch in der Informatik ist logisches Denken gefordert, etwa beim Formulieren von Algorithmen oder bei der Analyse und Konstruktion formaler Sprachen. Strukturierte Datenbankabfragen mit SQL oder der Entwurf relationaler Datenbanken erfordern für den Prozess der Normalisierung ebenfalls ein gutes Verständnis von Logik. J. vermittelt deshalb jeweils in einem Theorieteil die Grundbegriffe der Boole'schen Aussagenlogik. Bei ihren Klassen stößt das Thema aber auf wenig Resonanz. Für die Schülerinnen und Schüler ist der Unterrichtsinhalt zu abstrakt-formal. Der wiederholte Hinweis von J., dass das Wissen später bei anderen Themen benötigt wird, kann auch nicht wirklich motivieren.

Ohne einen erkennbaren Bezug zur Lebenswelt und ohne klaren Nutzen sinken bei den Lernenden Motivation und Interesse am Lernstoff. Besonders bei abstrakten, theoretischen Inhalten ist es oft schwierig, diese anschaulich aufzubereiten und deren Relevanz aufzuzeigen.

Betrachten wir als Beispiel Boole'sche Aussagenlogik: In Lehrmitteln erfolgt der Einstieg in das Thema Logik meist abstrakt und in einem rein mathematischen Kontext. Einen alternativen Zugang mit Bezug zur Lebenswelt stellt die Steuerung der Lichtsignalanlage einer Straßenkreuzung dar. Durch aussagenlogische Formeln soll verhindert werden, dass zwischen einzelnen Spuren Kollisionen auftreten können. In diesem Kontext lassen sich Begriffe wie „Operatoren“, „Variablen“, „Wahrheitswerte“, „Formeln“, „Wahrheitstabellen“, „Äquivalenzen“ und „Normalformen“ einführen. Ausgehend von realen Situationen können über Wahrheitstabellen die Grundlagen von logischen Formeln vermittelt werden. Abb. 5.1 zeigt ein Beispiel einer Simulation.

Situation
><

Wahrheitstabelle

A	B	C	D	sicher
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

nicht sicher

Tipp Testen Löschen

$(\neg A \wedge \neg B \wedge \neg C \wedge D) \vee (A \wedge B \wedge \neg C) \vee (A \wedge B \wedge D)$

Abb. 5.1 Simulation einer Lichtsignalanlage (Lernumgebung LogicTraffic, swisseduc.ch)

Als Instrument zur Überprüfung der Relevanz von Lerninhalten und ihres Bezugs zur Lebenswelt haben sich die fundamentalen Ideen von Bruner (1960) bewährt, die von Schwill (1993) auf den Informatikunterricht adaptiert wurden. Während Bruner den Begriff der „fundamentalen Idee“ nur vage formuliert, gibt Schwill eine präzise Definition an, basierend auf vier Kriterien: Horizontal-, Vertikal-, Zeit- und Sinnkriterium. Wir übernehmen in Abb. 5.2 die Definition von Schwill und ergänzen diese durch ein fünftes Kriterium, das Repräsentationskriterium. Dieses halten wir für zentral, da gerade die enaktive Darstellung dazu anregt, sich intensiv mit dem Lerninhalt auseinanderzusetzen und bei abstrakten Themen kreative Zugänge zu entwickeln.

Auf fundamentalen Ideen abgestützter Unterricht garantiert die Auswahl von langlebigen Inhalten, die für die Lernenden relevant sind. Das Herauskrystallisieren der fundamentalen Ideen ist aber oft nicht einfach. Entsprechend groß ist die Verlockung, ein Thema ohne eingehendere Reflexion als wichtig und grundlegend einzustufen. Anhand der fünf Kriterien kann überprüft werden, ob es sich bei einem Sachverhalt tatsächlich um eine fundamentale Idee handelt.

Die Überprüfung von Inhalten anhand der fünf Kriterien ist eine große Hilfe bei der Unterrichtsvorbereitung. Die Lehrerin wird gezwungen, sich Gedanken über die Bedeutung eines Sachverhaltes zu machen. Warum ist der Stoff wichtig? Weshalb müssen die Schülerinnen ihn verstehen? Die Beantwortung dieser Fragen fließt unmittelbar in den Unterricht ein. Die Überprüfung des Repräsentationskriteriums führt zu einer intuitiven Vorstellung des Sachverhaltes. Das Horizontalkriterium deckt Alltagsanalogien auf, und das Vertikalkriterium führt zu einfach verständlichen

Eine fundamentale Idee ist ein Sachverhalt, der ...

HORIZONTALKRITERIUM



... in verschiedenen Bereichen vielfältig anwendbar oder erkennbar ist.

VERTIKALKRITERIUM



... auf jedem intellektuellen Niveau aufgezeigt und vermittelt werden kann.

ZEITKRITERIUM



... in der historischen Entwicklung deutlich wahrnehmbar ist und längerfristig relevant bleibt.

SINNKRITERIUM



... einen Bezug zur Sprache und zum Denken des Alltags und der Lebenswelt besitzt.

REPRÄSENTATIONSKRITERIUM



... sich in verschiedenen kognitiven Repräsentationsformen (enaktiv, ikonisch, symbolisch) darstellen lässt.

Abb. 5.2 Kriterien fundamentaler Ideen

Erklärungen. Die Überlegungen zu fundamentalen Ideen helfen außerdem bei der Strukturierung der Unterrichtsunterlagen: Diese können in die langlebigen und in die kurzlebigen, oft produktspezifischen Inhalte aufgeteilt werden. Die kurzlebigen Inhalte müssen häufiger aktualisiert werden als die langlebigen. Dank der Aufteilung müssen jeweils nur Teile der Unterlagen überarbeitet werden.

Die Betonung der fundamentalen Ideen beugt der Gefahr vor, sich in kurzlebigen, produktspezifischen Details zu verlieren. Allerdings reicht es nicht aus, in einem Themengebiet einfach möglichst alle fundamentalen Ideen zu identifizieren. Je nach Zielgruppe können grundlegende Konzepte eines Themas für den Unterricht sehr relevant oder vollkommen belanglos sein. Es ist Aufgabe des Lehrers, aus den fundamentalen Ideen eine Auswahl zu treffen, die für die Zielgruppe von Bedeutung ist. Diskussionen mit Kolleginnen und Kollegen eignen sich sehr gut, um fundamentale Ideen zu identifizieren. Auch Fachliteratur, speziell die Klassiker in einem Themenbereich, geben oft nützliche Hinweise auf fundamentale Ideen. Auf höheren Schulstufen kann man auch zusammen mit leistungsstarken Schülerinnen

und Schülern fundamentale Ideen eines Themas erarbeiten und so Metareflexion zum eigenen Unterricht betreiben.

In der Informatikausbildung müssen die langlebigen Inhalte – grundlegende Konzepte und Methoden – im Mittelpunkt des Unterrichts stehen. Die fundamentalen Ideen sind ein Instrument, mit dem die Bedeutsamkeit eines Themas oder eines Sachverhaltes überprüft werden kann. Diese Überprüfung liefert wichtige Anhaltspunkte für die Aufbereitung der Unterrichtsinhalte und für die Unterrichtsgestaltung.

Nachfolgend geben wir ohne Anspruch auf Vollständigkeit eine Reihe von fundamentalen Ideen an. Die ersten zwei fundamentalen Ideen führen wir detaillierter aus.

Beispiel 1: Fundamentale Idee bei Grafikformaten: Raster- und Vektorgrafiken

Eine Rastergrafik (auch Pixelgrafik oder Bitmap) speichert die Information für jeden erfassten Punkt (Pixel) eines Bildes. Typische Formate für Rastergrafiken sind BMP, GIF, JPEG, TIFF und PNG. Eine Vektorgrafik dagegen beschreibt ein Bild durch mathematische Funktionen: Vektoren definieren Linien, Kurven oder Flächen. Um das Bild eines Kreises zu speichern, benötigt eine Vektorgrafik vier Angaben: die Lage des Kreismittelpunkts, den Kreisdurchmesser, die Farbe der Kreislinie und ihre Strichstärke. Vektorgrafiken können im Gegensatz zu Rastergrafiken ohne Qualitätsverlust stufenlos skaliert und verzerrt werden. Bei Vektorgrafiken bleiben die Eigenschaften einzelner Linien, Kurven oder Flächen erhalten und können nachträglich verändert werden. Demgegenüber sind Vektorgrafiken ungeeignet für die Darstellung von Fotos und vergleichbaren Grafiken, weil diese sich kaum mathematisch modellieren lassen.

Anhand der fünf Kriterien überprüfen wir, ob es sich bei der Einteilung in Raster- und Vektorgrafiken wirklich um eine fundamentale Idee handelt:

Horizontalkriterium: Eine fundamentale Idee muss in verschiedenen Bereichen anwendbar sein. In der Architektur wird schon lange mit Planskizzen gearbeitet, die sich als Vektorgrafiken beschreiben lassen. In der Kunst gibt es den Malstil des Pointillismus, der mit Malern wie Georges Seurat und Camille Pissarro seinen Höhepunkt hatte. Der Pointillismus ist charakteristisch für seine Bilder aus einzelnen, kleinen Farbtupfern, die ein schönes Beispiel für Rasterbilder darstellen. Während Darstellungsformen in der Architektur oder Kunst nahe am Thema Grafik liegen, sind besonders überzeugende Beispiele solche, die aus völlig anderen Bereichen stammen. Ein solches Beispiel ist die digitale Musikproduktion: Hier werden Musikstücke oft durch MIDI-Daten beschrieben. MIDI ist ein Format, das Noten und Spielinformationen (z. B. Tonhöhe, Anschlagstärke, Dauer) speichert. Diese Daten können ohne Qualitätsverlust bearbeitet und skaliert werden. Ein Musikstück kann transponiert, das Tempo verändert oder die Instrumentierung geändert werden,

ohne dass die Klangqualität leidet. MIDI entspricht somit einer Vektorgrafik, da es sich um eine flexible und skalierbare Beschreibung handelt. Demgegenüber steht die Audioaufnahme eines Musikstücks in einem Format wie WAV oder MP3. Hier wird das Signal in diskrete Zeitabstände unterteilt, ähnlich wie Pixel in einer Rastergrafik. Änderungen wie die Verlangsamung oder Beschleunigung eines Audiotracks führen oft zu Verzerrungen oder Qualitätsverlust, weil die diskreten Samples nicht stufenlos skaliert werden können. Eine Audioaufnahme entspricht also einer Rastergrafik.

Vertikalkriterium: Eine fundamentale Idee muss auf jedem intellektuellen Niveau aufgezeigt werden können. Ob das Vertikalkriterium erfüllt ist oder nicht, lässt sich einfach überprüfen: Man überlegt sich, ob man die Kernidee eines Sachverhaltes einem Kind erklären könnte. Bei der Unterscheidung von Raster- und Vektorgrafiken zum Beispiel so: Ein Kind kennt Rastergrafiken von Steckmosaiken her. Und Vektorgrafiken von Aufgaben, bei denen durch das Verbinden von Zahlen Zeichnungen in Form von Polygonzügen entstehen (siehe Abb. 5.3).

Die Überlegungen zum Vertikalkriterium zeigen den Wert des Kriteriums für die Lehrerin auf: Die auf Kinder ausgerichtete Erklärung führt nahtlos zu den Vor- und Nachteilen der beiden Formate. Bilder im Rasterformat ermöglichen detailreiche und realitätsnahe Darstellungen, benötigen aber mehr Speicherplatz als Vektorgrafiken und lassen sich nicht einfach vergrößern und verkleinern. Im Unterschied dazu brauchen Vektorgrafiken kaum Speicherplatz und lassen sich fast beliebig skalieren.

Zeitkriterium: Eine fundamentale Idee muss vor zehn Jahren gültig gewesen und in zehn Jahren noch relevant sein. In der Informatik haben die beiden Grafikformate eine lange Tradition.

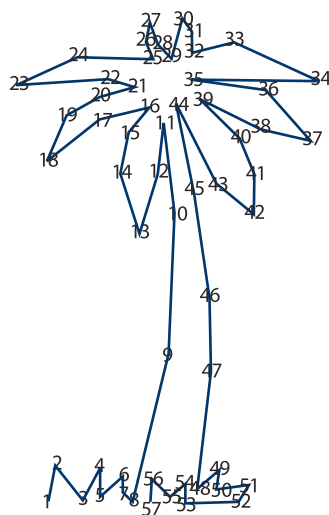
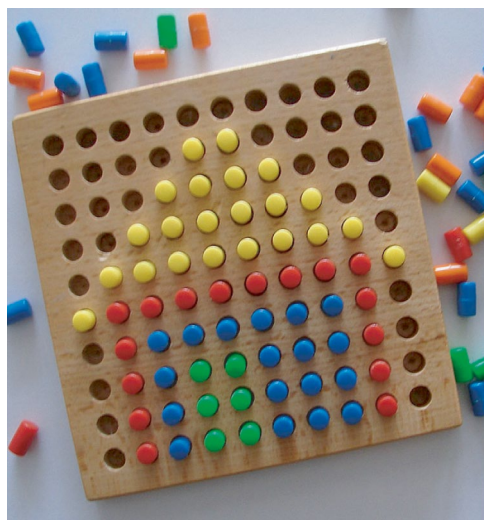


Abb. 5.3 Raster- und Vektorgrafiken kindgerecht erklärt

Sinnkriterium: Eine fundamentale Idee muss für die Lebenswelt der Lernenden relevant sein. Es muss klar sein, weshalb sich die Auseinandersetzung mit dem Sachverhalt für die Lernenden lohnt. Im Beispiel liegt der Nutzen auf der Hand: Als Computeranwenderin wird man mit beiden Grafikformaten konfrontiert. Digitalkamera und Scanner produzieren Rastergrafiken, Illustrationswerkzeuge oder Werkzeuge für den 3D-Druck arbeiten mit Vektorgrafiken.

Repräsentationskriterium: Eine fundamentale Idee lässt sich in der Regel enaktiv durch aktives Tun begreifen, ikonisch visualisieren und in kompakter Form symbolisch beschreiben. Enaktive Repräsentationen sind manchmal schwierig zu finden, sind aber sehr wertvoll, denn sie liefern Hinweise auf gute Analogien. Für Raster- und Vektorgrafiken sind beim Vertikalkriterium Hinweise auf enaktive Repräsentationen angeführt: Steckmosaike und Malen-nach-Zahlen.

Beispiel 2: Einschränkung einer Suche auf eine Teilkollektion mittels Metadaten

Bei einer Internetrecherche ist es oft sinnvoll, die unüberschaubare Menge zur Auswahl stehender Dokumente aufgrund übergeordneter Kriterien (Metadaten) zuerst auf eine Teilmenge zu reduzieren, wie in Abb. 5.4 dargestellt. Beispielsweise kann die Suche auf Dokumente einer bestimmten Sprache, eines bestimmten Dateityps oder auf Dokumente von einer bestimmten Website eingeschränkt werden. Die eigentliche Suche wird erst auf dieser Teilmenge durchgeführt.

Im Folgenden prüfen wir, ob es sich bei der Einschränkung der Suche auf eine Teilkollektion um eine fundamentale Idee handelt:

Horizontalkriterium: Sucht ein Patient ein Krankenhaus auf und beklagt sich über akute Schmerzen, wird man zuerst versuchen, den Schmerz zu lokalisieren,

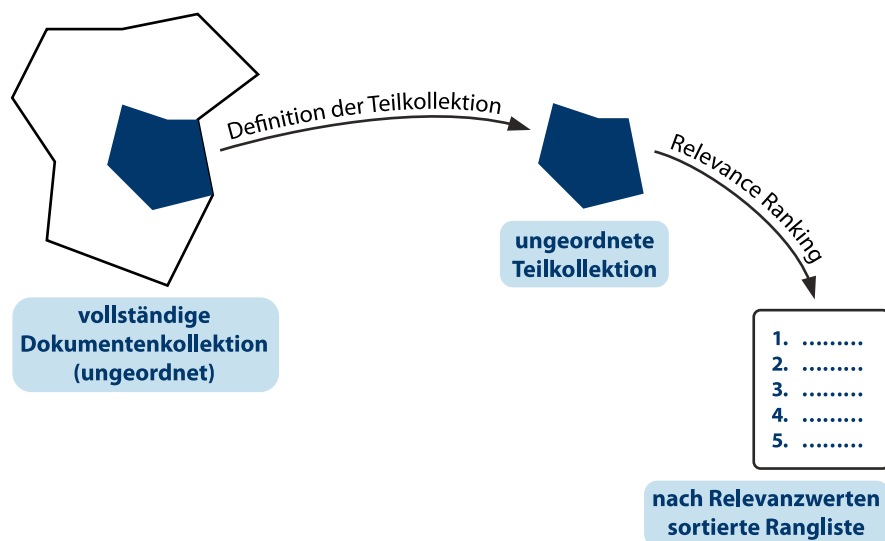


Abb. 5.4 Einschränkung einer Suche auf eine Teilkollektion

und dann den Patienten an den zuständigen Spezialisten auf der entsprechenden Abteilung überweisen.

Vertikalkriterium: Beim Kauf neuer Kinderschuhe wird die Suche im Schuhgeschäft zunächst anhand der gewünschten Schuhgröße, der Schuhart und weiterer Kriterien wie Farbe, Material oder Hersteller eingeschränkt. Diese Kriterien fungieren als Metadaten, die die Auswahl eingrenzen. Erst nachdem diese Filter angewendet wurden, schaut man sich die verbleibenden Optionen genauer an.

Zeitkriterium: Die Einschränkung auf eine Teilkollektion entspricht dem Einsatz eines Filters. Filtertechniken spielen in vielen Bereichen seit Langem eine Rolle. Auch in der Informatik kommen Filter zum Einsatz, etwa Dateifilter auf der Stufe Betriebssystem oder Textfilter zur Spamerkenkung. Filtern begegnen wir im Alltag auch in Onlineshops, bei Streamingdiensten, Bibliothekskatalogen oder Navigations-Apps, um nur einige Beispiele zu nennen.

Sinnkriterium: Bei der Einschränkung auf eine spezifische Teilmenge liegt der Vorteil auf der Hand. Anstatt die berühmte Nadel im gesamten Heuhaufen zu suchen, sucht man gezielt im relevanten Teil des Heuhaufens.

Repräsentationskriterium: Die Einschränkung einer Recherche auf eine Teilkollektion lässt sich gut anhand des Zusammensetzens eines Puzzles veranschaulichen, beispielsweise für das Motiv eines Sonnenuntergangs am Sandstrand. Eine gute Strategie kann hier sein, zum einen alle Randteile zu sammeln und zum anderen alle Teile mit der blauen Farbe des Himmels.

Beispiel 3: Teile-und-herrsche als algorithmisches Prinzip

Teile-und-herrsche („divide and conquer“) ist eine der wichtigsten fundamentalen Ideen der Informatik. Zuerst wird ein komplexes Problem in kleinere und überschaubare Probleme aufgeteilt. Dann werden die kleinen Probleme gelöst und anschließend die Teillösungen zur Gesamtlösung zusammengefügt. Wiederholtes Anwenden von Teile-und-herrsche führt zur Rekursion, einer weiteren fundamentalen Idee der Informatik. Die Überprüfung der fünf Kriterien halten wir kurz:

Horizontalkriterium: Teile-und-herrsche regiert die Welt. In der Politik hat sich die Methode „divide et impera“ seit Gaius Julius Cäsar bewährt. Wer im Alltag Arbeiten an andere delegieren will, kommt ebenfalls nicht um die Aufteilung der Arbeit in kleinere Stücke herum.

Vertikalkriterium: Teile-und-herrsche erleben wir schon im Kindergarten: Man stelle sich vor, 250 Kinder einer Gemeinde würden den Kindergarten in einem einzigen Raum besuchen.

Zeitkriterium: „Divide and conquer“ ist eine zentrale Problemlösungsstrategie in der Informatik, bei der ein Problem rekursiv in kleinere Teilprobleme zerlegt wird, bis diese einfach zu lösen sind. Anschließend werden die Teillösungen kombiniert, um die Lösung des Gesamtproblems zu erhalten. Dieses Prinzip kommt beispielsweise bei effizienten Algorithmen wie Quicksort oder der Fourier-Transformation zum Einsatz.

Sinnkriterium: Wenn uns die Arbeit über den Kopf wächst, ist Teile-und-herrsche ratsam.

Repräsentationskriterium: Teile-und-herrsche lässt sich gut veranschaulichen. Geschicktes Vorgehen beim gemeinsamen Lösen eines Puzzles ist eine Möglichkeit.

Beispiel 4: Dilemma zwischen Sicherheit und Benutzerfreundlichkeit

Sicherheit und Benutzerfreundlichkeit bei Informatiksystemen stehen oft in einem Spannungsverhältnis. Mehr Sicherheit führt in der Regel zu einer Einbuße an Benutzerfreundlichkeit und umgekehrt. Sehr schön zeigt sich das Dilemma bei Authentifizierungsmethoden: Starke Passwörter, Multifaktorauthentifizierung (MFA) und unterschiedliche Passwörter für verschiedene Dienste bieten erhöhten Schutz gegen unbefugten Zugriff auf Benutzerkonten. Diese Sicherheitsmaßnahmen können jedoch die Benutzerfreundlichkeit beeinträchtigen. Lange und komplexe Passwörter sind schwer zu merken, und die Notwendigkeit, ein zusätzliches Gerät für die MFA bereitzuhalten, kann als lästig empfunden werden. Ein Passwortmanager erleichtert zwar die Verwaltung verschiedener Passwörter, setzt aber voraus, dass man sich mit einem weiteren Informatikwerkzeug auseinandersetzt. Die Überprüfung der fünf Kriterien halten wir auch hier kurz:

Horizontalkriterium: Die Ausgestaltung der Zutrittskontrolle in Firmen, Hotels oder anderen Betrieben ist immer ein Kompromiss zwischen Sicherheit und Benutzerfreundlichkeit.

Vertikalkriterium: Klettergerüste auf einem Kinderspielplatz sollten möglichst sicher sein, aber auch ein Spannungsmoment beinhalten. Bei betagten Leuten steigt oft die Sturzgefahr, aber der gänzliche Verzicht auf Fortbewegung reduziert die Beweglichkeit weiter.

Zeitkriterium: Die Römer bauten ein großes Straßennetz, das neben militärischen Zwecken auch dem Handel diente. Gleichzeitig machten die guten Verkehrsverbindungen das Römische Reich für feindliche Angriffe anfällig. Das Dilemma zwischen Sicherheit und Benutzerfreundlichkeit begleitet auch technische Entwicklungen seit Langem. In der Kryptografie sorgten komplexe Verschlüsselungsverfahren für höhere Sicherheit, machten jedoch den Datenaustausch und die Integration in bestehende Systeme oft schwieriger. Sicherheitsgurte in Autos erhöhen die Sicherheit, wurden aber anfangs als unbequem empfunden.

Sinnkriterium: Sicherheit im Leben und gleichzeitig Spaß und Lebensfreude sind für die meisten Menschen wichtig. Eine Kreditkarte ohne PIN-Eingabe (z. B. kontaktloses Bezahlen) ist bequemer, aber unsicherer, falls sie gestohlen wird. Eine Zwei-Faktor-Authentifizierung für Onlinezahlungen erhöht die Sicherheit, macht den Prozess aber komplizierter. Kameras in öffentlichen Räumen erhöhen die Sicherheit, schränken aber die Privatsphäre der Menschen ein. Strenge Sicherheitskontrollen an Flughäfen erhöhen die Sicherheit der Passagiere, führen aber zu längeren Wartezeiten und Unannehmlichkeiten.

Repräsentationskriterium: Im Internet finden sich viele Comics und Video-clips, die auf mehr oder weniger lustige Art die Folgen von übertriebenem Sicherheitsdenken zeigen. Als enaktive Umsetzung könnte man einen Dominostein-Wettbewerb inszenieren. Welches Team schafft in einer bestimmten Zeit die längste Kette? Ohne eingebaute Sicherheitslücken ist die Gefahr groß, dass die Kette vor-

zeitig fällt. Der Einbau von Sicherheitsmechanismen verzögert aber den Aufbau der Kette.

Beispiel 5: Great Principles und fundamentale Ideen

Das Instrument der fundamentalen Ideen kann auf alle Ausbildungsstufen angewendet werden und zum Beispiel in Zusammenhang mit der fachsystematischen Strukturierung nach den Great Principles of Computing gesetzt werden. Wir zeigen dies stellvertretend für die Zielgruppen ICT-Anwender, IT-Techniker und Softwareentwickler. In Tab. 5.1 geben wir zu jeder fundamentalen Idee einige inhaltliche Stichwörter und begründen im Anschluss, wieso und in welchem Zusammenhang die fundamentale Idee für das betreffende Zielpublikum von Bedeutung ist.

Communication

- *Synchrone und asynchrone Kommunikation:* Anwender sind mit einer Vielfalt von Kommunikationsmitteln konfrontiert: E-Mail, SMS, Chat, soziale Netzwerke, Foren usw. Die Nutzung des richtigen Werkzeuges für den richtigen Zweck setzt ein Verständnis für grundlegende Prinzipien der Kommunikation wie synchrone versus asynchrone Kommunikation oder Push- versus Pull-dienst voraus.
- *Fehlerkorrigierende Codes:* Bei der Übertragung von Daten erhält der Empfänger aufgrund von fehlerbehafteten Übertragungskanälen häufig verfälschte Da-

Tab. 5.1 Fundamentale Ideen und Great Principles of Computing

	ICT-Anwender
Communication	Synchrone und asynchrone Kommunikation
Computation	Was ist ein Programm?
Recollection	Index einer Suchmaschine
Coordination	Versionierung
Evaluation	Bewertung von Internetverbindungen
Design	Standard-Interaktionselemente
	IT-Techniker
Communication	Fehlerkorrigierende Codes
Computation	Boolesche Aussagenlogik
Recollection	Zwischenspeicher
Coordination	Workflows organisieren
Evaluation	Evaluierung von Server-Hardware für Unternehmensnetzwerke
Design	Systemdesign für Netzwerkinfrastrukturen
	Softwareentwickler
Communication	Abtasttheorem von Shannon
Computation	Turingmaschinen
Recollection	Metadaten
Coordination	Gemeinsame Ressourcen
Evaluation	Optimierung von Algorithmen
Design	Design Patterns für Software-Architektur

ten. IT-Techniker sollten verstehen, wie man durch Hinzufügen von Redundanz fehlerentdeckende und fehlerkorrigierende Codes erzeugen kann.

- *Abtasttheorem von Shannon:* Die Umwandlung von analogen in digitale Signale und umgekehrt ist zentral für viele Technologien. Informatikerinnen sollten wissen, wie man Funktionen aus Abtastwerten rekonstruiert und interpoliert, und eine Vorstellung über den Zusammenhang zwischen der Abtastung und dem Spektrum haben.

Computation

- *Was ist ein Programm?* Anwender sollten eine Vorstellung davon haben, was ein Programm ist. Ein Programm kann definiert werden als eine statische Spezifikation dynamischer Abläufe. Diese Definition ist viel zu abstrakt, als dass ein Anwender damit etwas anfangen könnte. Die Idee dahinter lässt sich aber an einfachen Beispielen illustrieren.
- *Boole'sche Aussagenlogik:* Logik spielt in vielen Bereichen eine Rolle, bei Datenbankabfragen, beim Programmieren oder beim Konfigurieren von Regelwerken für Spamfilter oder Firewalls. IT-Techniker müssen deshalb die Grundlagen Boole'scher Aussagenlogik kennen.
- *Turing-Maschinen:* Aussagen über Berechenbarkeit oder Komplexität beziehen sich auf ein mathematisch definiertes Modell, das die Daten und erlaubten Operationen festlegt. Turing-Maschinen, rekursive Funktionen oder Lambda-Kalkül sind solche Berechnungsmodelle.

Recollection

- *Index einer Suchmaschine:* Die effiziente und effektive Nutzung von Suchmaschinen ist nur auf den ersten Blick kinderleicht. Will man bei einer Suche die Ausbeute nicht unnötig einschränken, kommt man nicht um ein Grundverständnis für die Funktionsweise einer Suchmaschine herum, etwa um den Prozess der Indexierung und Normalisierung von Textdokumenten.
- *Zwischenspeicher:* IT-Techniker sollten Zwischenspeicher wie Caching und Proxies als allgemeines Prinzip zur Effizienzsteigerung verstehen und die wichtigsten Anwendungsgebiete in der Informatik kennen.
- *Metadaten:* Das Erfassen von Daten über Daten ist ein grundlegendes Prinzip, das man bei Betriebssystemen, Datenbanken oder der Architektur großer Websites antrifft. Softwareentwicklerinnen sollten vertiefte Kenntnisse zu Metadaten besitzen und die Unterschiede zwischen normalisierten und nicht normalisierten Metadaten oder kontrollierten und nicht kontrollierten Thesauri kennen.

Coordination

- *Versionierung:* Sobald mehrere Leute gemeinsam an einem Dokument arbeiten, ist der Einsatz eines Versionskontrollsystems sinnvoll. Anwender werden schon bei Verwendung des Überarbeitungsmodus in einer Textverarbeitung mit den Vor- und Nachteilen solcher Systeme konfrontiert. Ein Verständnis für typische

Probleme bei parallelen Prozessen wie Deadlocks oder Inkonsistenzen aufgrund fehlerhafter Synchronisation ist dabei hilfreich.

- *Workflows organisieren:* IT-Techniker müssen oft zeitkritische Arbeitsabläufe planen, etwa beim Austausch von Server- und Netzinfrastruktur. Grundlagen zu Scheduling und Workflowmanagement sind bei solchen Aufgaben eine große Hilfe.
- *Gemeinsame Ressourcen:* Bei Kommunikationsprotokollen oder beim Programmieren nebenläufiger Prozesse spielen Synchronisationsprobleme und entsprechende Lösungen wie atomare Operationen, Write Locks oder Monitore eine wichtige Rolle und gehören deshalb zum Wissen jeder Informatikerin.

Evaluation

- *Bewertung von Internetverbindungen:* Die Qualität der Internetverbindung lässt sich anhand von Bandbreite und Latenz einschätzen. Bandbreite ist entscheidend für Streaming, da sie die maximale Datenrate angibt. Latenz ist besonders wichtig für Echtzeitanwendungen wie Onlinegaming, da sie die Verzögerung misst. Ein Grundverständnis dieser Faktoren hilft, die Ursache möglicher Verbindungsprobleme zu verstehen.
- *Evaluierung von Serverhardware für Unternehmensnetzwerke:* IT-Techniker sollten Server nach Kriterien wie Rechenleistung, Stromverbrauch, Ausfallsicherheit und Wartungsfreundlichkeit bewerten können. So können sie geeignete Hardware auswählen, um gegebene Projektanforderungen zu erfüllen.
- *Optimierung von Algorithmen:* Softwareentwickler bewerten die Effizienz von Algorithmen anhand ihrer Laufzeit und des Ressourcenbedarfs, um mögliche Optimierungspotenziale zu erkennen. Durch diese Analyse lassen sich ineffiziente Algorithmen identifizieren und durch leistungsfähigere Alternativen ersetzen, was insbesondere bei der Verarbeitung großer Datenmengen zu erheblichen Verbesserungen führen kann.

Design

- *Standardinteraktionselemente:* Ein Verständnis für Standardinteraktionselemente wie Buttons, Menüs, Gesten oder Tastenkürzel erleichtert die Nutzung von Betriebssystemen und Anwendungssoftware. IT-Anwender werden dank deren Kenntnis effizienter oder lernen bislang unbekannte Funktionen kennen.
- *Systemdesign für Netzwerkinfrastrukturen:* Effektives Netzwerkdesign erfordert den Einsatz von Methoden wie Load Balancing zur Lastverteilung, Redundanz für Ausfallsicherheit und Maßnahmen zur Netzwerksicherheit. IT-Techniker sollten mit diesen Methoden vertraut sein, um skalierbare und robuste Systeme bauen oder anpassen zu können.
- *Design Patterns für Softwarearchitektur:* Design Patterns bieten bewährte Lösungen für wiederkehrende Probleme in der Softwareentwicklung. Muster wie Singleton, Observer oder Model-View-Controller (MVC) erleichtern die Entwicklung strukturierter, wartbarer und erweiterbarer Softwarearchitekturen und sind ein essenzieller Bestandteil moderner Softwareprojekte.

Literatur

Bruner, J. S. (1960). *The process of education*. Harvard University Press.

Schwill, A. (1993). Fundamentale Ideen der Informatik. *Zentralblatt für Didaktik der Mathematik*, 25(1), 20–31.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Dozentin H. plant eine firmeninterne Fortbildung mit dem Ausschreibungstitel „Effizient arbeiten mit MS 365: Entdecken Sie die Power von Teams, OneDrive und Co.“ und dem Kursinhalt „Nutzung von Teams für virtuelle Besprechungen, gemeinsame Bearbeitung von Dokumenten in Echtzeit via OneDrive, Nutzung von SharePoint. Hilfreiche Tastenkombinationen und versteckte Funktionen in Word, Excel und PowerPoint sowie Tipps zur Verwaltung von E-Mails und Kalendern in Outlook.“ Die Ausschreibung könnte aber auch so lauten: Titel: „Unter der Oberfläche von MS 365: Einblicke in die Technologien, die moderne Arbeitswelten antreiben.“ Kursinhalt: „Wir vermitteln Ihnen die Konzepte von Cloud-Services und ihre effiziente Nutzung in einer modernen Arbeitsumgebung. Ein besonderes Augenmerk gilt dabei der Datensicherheit und dem Datenschutz sowie dem Einsatz der integrierten KI-Tools.“

In der Informatik spielen sowohl das Konzeptwissen als auch das Produktwissen eine große Rolle. Konzentrieren sich Lehrerinnen und Kursleiter einseitig auf das Vermitteln von Konzeptwissen, fehlt die Handlungsorientierung: Die Lernenden können das Gelernte nicht in die Praxis umsetzen. Liegt der Fokus stattdessen zu sehr auf der Vermittlung von Produktwissen, ist der Unterricht nicht nachhaltig: Die Lernenden können das Gelernte nicht auf neue Situationen übertragen und anwenden.

Es gibt verschiedene Möglichkeiten, Wissen zu kategorisieren. Für unsere Zwecke genügt eine einfache Klassifizierung: Konzeptwissen umfasst die längerfristig gültigen, grundlegenden Zusammenhänge eines Sachgebiets. Produktwissen umfasst die Kenntnisse, die zur Bedienung eines konkreten Produkts nötig sind, zum Beispiel einer Softwareanwendung oder einer Hardwarekomponente. Konzeptwissen ist genereller, denn es gilt für alle Produkte und Produktversionen. Die folgende Gegenüberstellung in Tab. 6.1 charakterisiert die beiden Begriffe:

Tab. 6.1 Produkt- und Konzeptwissen im Vergleich

Produktwissen	Konzeptwissen
Wie funktioniert es?	Warum funktioniert es?
Produktbezogen	Produktunabhängig
Kurzlebig	Langlebig
Auswendig lernen, wiedergeben	Verstehen und einordnen
Isolierte Fakten	Zusammenhänge
Wenig Transfer möglich	Transfer möglich
Konkret	Abstrakt
Handlungsorientiert	Verstehensorientiert

Im Informatikunterricht ist es verlockend, das Schwergewicht nur auf das Produktwissen zu legen: Der rasche Wandel in der Informatik verleitet dazu, das Augenmerk in erster Linie auf die sich ändernden Produkte zu richten. Lehrkräfte haben zudem wenig Zeit, um in Ruhe die grundlegenden Konzepte zu identifizieren. Das gilt insbesondere für Themen, für die sie selbst keine Profis sind. Der Druck ist groß, das Gelernte unmittelbar einsetzen zu können. Produktwissen ist sofort nutzbar und für die Lernenden auf den ersten Blick weniger anspruchsvoll als Konzeptwissen.

Ähnlich verlockend ist es, den Fokus ausschließlich auf das Konzeptwissen zu richten. Dadurch schützt sich der Lehrer davor, sich dauernd mit neuen Entwicklungen auseinandersetzen zu müssen. Unterlagen müssen nicht aktualisiert werden, wenn neue Produktversionen auf den Markt kommen. Der Lehrer kann sich das nötige Wissen bequem aus Lehrbüchern aneignen. Nicht zuletzt tritt er weniger in Konkurrenz mit jenen Lernenden, die beim Produktwissen einen Vorsprung haben.

Die Unterscheidung zwischen Konzept- und Produktwissen ist wichtig beim Erstellen von Unterrichtsmaterialien und erhöht die Wiederverwendbarkeit von Teilen der Unterrichtsvorbereitung zu einem späteren Zeitpunkt. Der Quicksortalgorithmus kann genauso präsentiert werden wie vor zwanzig Jahren. Auch das Konzept von absoluten und relativen Adressen in einer Tabellenkalkulation hat sich nicht geändert.

Im Unterschied zum Konzeptwissen nimmt die Bedeutung von Produktwissen eher ab. Der Einsatz von generativer Künstlicher Intelligenz ersetzt in den meisten Fällen das Nachschlagen in Produktdokumentationen. Durch die Nutzung von KI-gestützten Tutoren können Lernende individuell und effizient unterstützt werden, wenn es darum geht, spezifische Konzepte in bestimmten Werkzeugen umzusetzen. Möchte man zum Beispiel wissen, wie man eine Liste in Python sortiert, liefert die Anfrage „Wie sortiere ich eine Liste in Python?“ eine direkte Antwort mit Codebeispielen. Auch bei Fragen zur Bedienung von Werkzeugen, sei es eine Office-anwendung, eine Entwicklungsumgebung oder eine spezifische Softwarelösung, bietet Künstliche Intelligenz oft wertvolle Unterstützung.

Guter Informatikunterricht vermittelt sowohl Konzeptwissen als auch Produktwissen. Das Konzeptwissen hilft beim Einordnen von Fakten in größere Zusammenhänge und erleichtert somit das Lernen. Es ermöglicht außerdem den Transfer von früher erworbenen Kenntnissen auf neue Situationen. Das Produktwissen fördert die Handlungsorientierung, damit die Lernenden ihr Wissen in der Praxis einsetzen können.

Beispiel 1: Gestaltung guter Präsentationen

Die Gestaltung guter Präsentationen mit einer Präsentationssoftware erfordert sowohl Produktwissen als auch Konzeptwissen. Zum Konzeptwissen gehört unter anderem die Bedeutung der Zielgruppenanalyse: Was sind die Bedürfnisse und Erwartungen der Zielgruppe? Aber auch die Entwicklung einer klaren Kernbotschaft und eines roten Fadens für die Präsentation. Bei der visuellen Gestaltung einzelner Folien gilt es, die Multimediadesign-Prinzipien zu berücksichtigen. Beispielsweise das Kohärenzprinzip, wonach Text, Grafiken oder Audioclips, die ausschließlich zur Unterhaltung dienen oder nur marginal relevante Informationen beinhalten, besser weggelassen werden. Oder das räumliche Kontiguitätsprinzip, wonach Text und zugehörige Bildelemente direkt nebeneinander angeordnet werden sollten. Beschriftungen sollen direkt neben den zugehörigen Elementen angeordnet werden.

Beim Produktwissen geht es darum, die Funktionen der genutzten Präsentationssoftware effektiv zu nutzen. Dazu gehören die Vertrautheit mit der Benutzeroberfläche und ihren Menüs, die verschiedenen Ansichten (Normalansicht, Folienmasteransicht, Präsentationsansicht), das Einfügen und Bearbeiten von Textfeldern, Bildern, Diagrammen, Tabellen, Videos und Audios.

Beispiel 2: Copy and Paste

Copy and Paste ist ein wichtiger, effizienzsteigernder Mechanismus in der Informatik, der vom Betriebssystem in engem Zusammenspiel mit Anwendungen ermöglicht wird. Die folgenden Aspekte gehören zum Konzeptwissen, unabhängig von der verwendeten Anwendung oder dem Betriebssystem:

- Mithilfe von Copy and Paste lassen sich Objekte von einer Stelle an eine andere kopieren. Objekte können zum Beispiel Textelemente, Bilder oder Diagramme sein. Das Kopieren funktioniert innerhalb derselben Anwendung und meistens auch zwischen verschiedenen Anwendungen. Die Variante Cut and Paste verschiebt die Objekte. Die Funktionen Copy and Paste und Cut and Paste lassen sich meistens über eine Tastenkombination oder einen Menüpunkt ansteuern.
- Copy and Paste funktioniert nur, wenn das gewünschte Objekt zuvor selektiert wurde. Manchmal geschieht die Selektion implizit, weil beispielsweise das betreffende Objekt das einzig verfügbare ist. Aufgrund dieser Voraussetzung sind die Copy-and-Paste-Funktionen in der Regel auch über das Kontextmenü erreichbar.

- Zwischen dem Zeitpunkt des Kopierens oder Ausschneidens und dem Zeitpunkt des Einfügens muss das Objekt zwischengelagert werden. Darum kümmert sich das Betriebssystem. Bei vielen Betriebssystemen kann die Zwischenablage eingesehen werden.
- Bei unterschiedlichen Anwendungstypen ist das Verhalten von Copy and Paste nicht immer vorhersehbar. Was geschieht beim Kopieren eines Textabschnitts aus einer Textverarbeitung und Einfügen in ein Malprogramm? Die beiden Anwendungen arbeiten mit unterschiedlichen Datentypen. Bei manchen Anwendungen geschieht beim Einfügen nichts, bei anderen wird der Datentyp automatisch angepasst und beispielsweise aus dem Text ein Grafikobjekt erzeugt. Für jeden Datentyp stellt sich die Frage, was genau kopiert wird. Beim Kopieren von Text aus einem Webbrowser können zum Beispiel nur die reinen Textzeichen oder aber der Text inklusive aller Formatierungsangaben kopiert werden. Manchmal hat man auch die Wahl und kann sich beim Einfügen für eine von mehreren Varianten entscheiden.

Um effizient zu arbeiten, ist es jedoch unerlässlich, sich Produktwissen anzueignen, sich also mit der Umsetzung der Konzepte auf einem konkreten Gerät und Betriebssystem vertraut zu machen: Unter Windows zum Beispiel muss man wissen, dass Copy, Cut und Paste über `ctrl + C`, `ctrl + X` und `ctrl + V` auf der Tastatur oder in Standardanwendungen über den Menüpunkt „Bearbeiten“ oder die rechte Maustaste erreichbar sind. Arbeitet man mit Apple-Geräten, lauten die Tastenkombinationen `cmd + C`, `cmd + X` und `cmd + V`. Bei der Bedienung eines mobilen Geräts (z. B. Android-Smartphones) hingegen drückt man auf das zu kopierende Objekt, bis es als markiert dargestellt wird (z. B. farblich hinterlegt). Aus dem Kontextmenü wählt man „Kopieren“ und wechselt zur gewünschten Anwendung. In dieser drückt man auf die Stelle, an welcher das Objekt eingefügt werden soll, und wählt im sich öffnenden Dialog „Einfügen“ aus.

Beispiel 3: Dateisysteme

Alle Computernutzer sind mit Betriebssystemen konfrontiert, IT-Supporter und andere Fachleute oft mit mehreren Betriebssystemen und verschiedenen Versionen. Betriebssysteme übernehmen eine Vielzahl von Funktionen. Unter anderem stellen sie mit dem Dateisystem eine Abstraktion der Festplatte und anderer Speichermedien zur Verfügung. In diesem Zusammenhang gibt es allerlei Konzeptwissen: Dateien und Ordner, der baumartige Aufbau von Ordnerstrukturen, Metadaten von Objekten im Dateisystem oder Zugriffsrechte.

Alles Konzeptwissen nützt aber nichts, wenn man unter Unix-ähnlichen Betriebssystemen mit einem Kommandozeileninterpreter konfrontiert ist, der keine grafischen Menüs kennt, sondern ausschließlich auf Tastatureingaben reagiert. In dieser Situation ist Produktwissen nötig. Man muss die wichtigsten Befehle kennen, die zur Verfügung stehen. Einige Beispiele: Den Pfad des aktuellen Verzeichnisses anzeigen mit `pwd`, in ein anderes Verzeichnis wechseln mit `cd`, alle Objekte im aktuellen Verzeichnis inklusive Details anzeigen mit `ls -al`. Es gibt Hunderte relevanter Befehle zur Steuerung des Betriebssystems. Zumindest einen Grundstock davon

muss man auswendig kennen, um mit einem Kommandozeileninterpreter arbeiten zu können.

Beispiel 4: Aufbau von Internetadressen

Es ist allgemein bekannt, dass man Zeichenfolgen wie <https://de.wikipedia.org/wiki/Paris> im Webbrowser eingeben kann und so bei der betreffenden Webseite landet. Dank der starken Standardisierung bei den Browsern wissen die meisten Leute auch, wo sie die Zeichenfolge eingeben müssen, nämlich in dem Textfeld ganz oben im Fenster. Das ist unverzichtbares Produktwissen für die Bedienung eines Webbrowsers. Gleichzeitig ist eine vernünftige Portion Konzeptwissen nützlich. Das Verständnis für den grundlegenden Aufbau einer Adresse im Internet Protokoll:// Rechneradresse/Pfad hilft in verschiedenen Situationen:

- Der Rechnername kann grundsätzlich beliebig gewählt werden. Aufgrund dieser Erkenntnis verstehen die Lernenden, dass eine Rechneradresse nicht zwingend mit www beginnen muss, sondern dass de.wikipedia.org auch eine gültige Adresse ist.
- Manchmal werden Webseiten nicht gefunden. Mit dem Wissen um den Pfad nach der Rechneradresse können die Lernenden nach einer Fehlermeldung einen Teil des Pfads oder den ganzen Pfad weglassen und es nochmals versuchen, zum Beispiel mit <https://de.wikipedia.org/>.
- Weil http und https zwei unterschiedliche Protokolle bezeichnen, die ansonsten auch dem obigen Schema folgen, kann man oft bei einer http-Adresse das s hinzufügen, um zur verschlüsselten Variante zu gelangen.

Beispiel 5: Nutzung von Git

Die effiziente Nutzung von Git oder einer anderen Versionsverwaltung setzt sowohl Konzeptwissen als auch Produktwissen voraus. Um Git in der Softwareentwicklung effektiv einzusetzen, benötigt man ein Verständnis der grundlegenden Prinzipien von Versionskontrollsystemen. Dazu zählen Konzepte wie die Verwendung von Branchingstrategien, um parallele Entwicklungszweige zu verwalten, die Handhabung von Mergekonflikten und der Ablauf von Pull Requests für Codereviews. Diese Konzepte erleichtern die Zusammenarbeit im Team und sorgen für einen strukturierten Entwicklungsprozess.

Für die konkrete Nutzung von Git braucht es Kenntnisse der wichtigsten Befehle wie git clone, git commit, git push, git pull, git branch und git merge. Zusätzlich ist das Verständnis von grafischen Benutzeroberflächen für Git hilfreich, die den Umgang mit Repositories erleichtern. Ohne dieses Wissen ist eine effiziente Arbeit mit Git kaum möglich, auch wenn man die Konzepte dahinter versteht.

Das Konzeptwissen vermittelt den größeren Zusammenhang, während Produktwissen die konkrete Umsetzung ermöglicht. Beides ist notwendig, um Git effektiv im Alltag einzusetzen.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Teil III

Unterrichtsvorbereitung

Zielgruppengerechte Zugänge

7

Einblick: In der 8. Klasse steht für die nächsten sechs Stunden das Erstellen von Webseiten auf dem Programm. Dabei sollen die Grundlagen von HTML und CSS vermittelt werden. Das Ziel: Die Schülerinnen und Schüler erstellen eine kleine Website über sich selbst – mit einem Porträtfoto, einigen Angaben zu ihren Interessen und mehr. Eigentlich ein tolles Thema für den Unterricht, denkt sich Lehrer P. Umso überraschter ist er, dass bei einigen die Motivation für die Aufgabe sehr gering ist. „Das ist doch Schnee von gestern, das können wir längst“, meinen sie. Dabei ist sich P. sicher, dass viele die grundlegenden Konzepte wie die Trennung von Inhalt, Struktur und Layout noch nicht wirklich verstanden haben. Andere haben bereits Mühe, mit einem Texteditor eine einfache HTML-Datei zu erstellen und diese im Browser anzuzeigen. Und eine Schülerin kritisiert grundsätzlich: Sie würde das sowieso ganz anders machen und ein Content-Management-System nutzen.

Das Zielpublikum im Informatikunterricht ist sehr breit gefächert. Informatik wird in vielen Schulstufen unterrichtet, von der Primarschule bis zur Hochschule. Die Lernenden haben oft stark unterschiedliche Vorkenntnisse – vom Nerd bis zu denen, die Informatik lieber meiden. Diese Heterogenität von Zielgruppen und Lernenden muss bei der Unterrichtsplanung berücksichtigt werden.

Im Mittelpunkt des Informatikunterrichts steht oft die Auseinandersetzung mit Informatiksystemen. Dabei geht es entweder um deren Analyse, Erstellung oder Nutzung zur Lösung konkreter Aufgaben. Je nach Zielgruppe liegt der Schwerpunkt eher auf theoretischen Aspekten oder praktischen Fertigkeiten. Die Lernenden bringen oft stark unterschiedliche Interessen und Vorkenntnisse mit. Manche beschäftigen sich möglicherweise bereits in ihrer Freizeit intensiv mit Themen wie Programmierung oder Videobearbeitung. Auch die private digitale Ausstattung der Lernen-

den variiert in der Regel stark. Die Anpassung des Unterrichts an die Bedürfnisse der Zielgruppe ist in allen Fächern wichtig, stellt aber in der Informatik eine besondere Herausforderung dar.

Im Folgenden beschreiben wir ohne Anspruch auf Vollständigkeit einige wichtige Aspekte, die bei einem zielgruppengerechten Zugang berücksichtigt werden sollten.

Berücksichtigung von Vorgaben Je nach Schulart und -stufe hat die Lehrkraft unterschiedlich viel Freiraum bei der Auswahl der Inhalte. Im Bereich der obligatorischen Schule inklusive des Gymnasiums geben Lehrpläne die Inhalte vor. Diese sind oft überladen, sodass die Lehrkraft eine sinnvolle Auswahl treffen und dabei das Schulprofil sowie regionale Gegebenheiten berücksichtigen muss. In Berufsschulen oder bei einheitlichen Abschlussprüfungen besteht hingegen kaum Spielraum, und in firmeninternen Kursen oder Zertifizierungsfortbildungen sind häufig sogar die Prüfungsfragen vorgegeben.

Berücksichtigung formal-operationaler Entwicklungsstadien Gemäß der kognitiven Theorie von Jean Piaget entwickeln Kinder etwa ab dem zwölften Lebensjahr die Fähigkeit zu abstraktem und systematischem Denken nach formal-logischen Regeln. Diese Entwicklung ist sowohl für den Mathematik- als auch Informatikunterricht in unteren Schulstufen von großer Bedeutung. Unterrichtsmethoden in der Primar- und frühen Sekundarstufe sollten verstärkt auf konkrete Beispiele setzen. Hier bietet sich zum Beispiel das PRIMM-Modell (Sentance & Waite, 2017) an, das die Phasen Predict (Vorhersagen), Run (Ausführen), Investigate (Untersuchen), Modify (Verändern) und Make (Erstellen) umfasst. Ein Beispiel hierfür könnte die Analyse eines kleinen in Scratch programmierten Spiels sein, das anschließend modifiziert und erweitert wird.

Berücksichtigung unterschiedlicher Infrastruktur Die Infrastruktur spielt im Informatikunterricht eine wichtige Rolle. Ähnlich wie in den Naturwissenschaften Chemie, Biologie oder Physik ist der Informatikunterricht eng mit einem „Labor“ verknüpft. In der Laborumgebung sammeln die Schülerinnen und Schüler praktische Erfahrungen und wenden theoretische Konzepte direkt an. In den unteren Schulstufen stehen oft weniger technische Ressourcen zur Verfügung. Daher müssen Unterrichtsmethoden gewählt werden, die ohne spezialisierte Hard- oder Software auskommen. Beispielsweise können in der Primarschule kompakte und kostengünstige Roboter mit webbasierten Programmierungsumgebungen eingesetzt werden. Dies ermöglicht einen spielerischen Einstieg in die Programmierung ohne hohen technischen Aufwand.

In Gymnasien, Berufsschulen und Technikerschulen sind die Ressourcen in der Regel umfangreicher. Schülerinnen und Schüler haben Zugang zu Hardware wie Arduino-Boards oder Raspberry Pi, mit denen sie tiefere Einblicke in die Programmierung und Elektronik gewinnen können. Auch firmeninterne Kurse profitieren von spezialisierter Ausrüstung, die auf die beruflichen Anforderungen zugeschnitten ist.

Berücksichtigung heterogener Vorkenntnisse Privathaushalte verfügen in der Regel über kein eigenes Chemie- oder Physiklabor. Daher bringen Schülerinnen und Schüler in diesen Fächern meist ähnliche Vorkenntnisse und Erfahrungen mit. Im Informatikunterricht hingegen gibt es große Unterschiede im privaten Zugang zu IT-Infrastruktur, was zu einer höheren Heterogenität der Vorkenntnisse und Erfahrungen führt. Doch nicht nur in Bezug auf die private Ausstattung unterscheidet sich der Informatikunterricht von anderen „Laborfächern“. Es gibt Schülerinnen und Schüler, die sich in ihrer Freizeit intensiv mit digitalen Medien und Informatik beschäftigen und dadurch umfangreiche Anwenderkenntnisse und Produktwissen mitbringen. Es gibt aber auch Lernende, die lieber die Finger von allen Dingen rund um Informatik lassen und wenig Interesse und Motivation für den Informatikunterricht zeigen.

Berücksichtigung von Genderaspekten Im Informatikunterricht spielen auch Genderaspekte eine Rolle. Schülerinnen und Schüler haben teils unterschiedliche Interessen und Zugänge zur Informatik. So neigen Mädchen dazu, Computer als Werkzeuge für spezifische Anwendungen zu betrachten, während Jungen sie eher als Spielzeug oder technisches Gerät wahrnehmen. Lehrkräfte sollten diesen Aspekten bei der Gestaltung des Unterrichts Rechnung tragen. Die Unterrichtsthemen sollten eine Vielfalt von Interessen ansprechen und sowohl technische als auch anwendungsbezogene Aspekte abdecken. Beispielsweise können Projekte, die gesellschaftliche oder kreative Anwendungen der Informatik betonen, Mädchen besonders ansprechen.

Eine bewusste Planung und Gestaltung des Unterrichts, die die unterschiedlichen Voraussetzungen und Bedürfnisse der Zielgruppe einbezieht, ist entscheidend für einen erfolgreichen Informatikunterricht.

Literatur

Sentance, S., Waite, J. (2017). PRIMM: Exploring pedagogical approaches for teaching text-based programming in school. In: *Proceedings of the 12th workshop on Primary and Secondary Computing Education* (S. 113–114).

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: In der Kaffeepause verschafft W. seinem Frust Gehör: „Ich hatte heute vor, den Schülern die Bedeutung von Formatvorlagen bei der Trennung von Inhalt, Struktur und Layout zu verdeutlichen. Da sie alle Webseiten erstellen möchten, habe ich XML und CSS als Beispiele gewählt. Bei den Übungen bereiteten ihnen aber vor allem die CSS-Selektoren Schwierigkeiten. Dass sich dieselben Konzepte in den Formatvorlagen von Textverarbeitungen wiederfinden, hat am Ende niemand verstanden.“

Kollegin H. versteht die Probleme nur zu gut: „Ich habe auch immer wieder den Eindruck, dass die Schülerinnen sich bei Übungen in den Details verlieren. Vor lauter Bäumen sehen sie den Wald nicht mehr. Und in den Prüfungen sind sie dann überfordert.“

Im Informatikunterricht werden Konzept- und Produktwissen vermittelt und praktische Fertigkeiten gefördert. Für die Lernenden bleibt dabei zum Teil unklar, was die wesentlichen Ziele des Unterrichts sind; sie laufen Gefahr, sich in Details zu verlieren.

Klare Zielsetzungen sind ein wesentlicher Bestandteil guten Unterrichts; Kompetenzen und Lernziele bilden das Fundament jeder allgemeinen Didaktik. Im Informatikunterricht, in dem die Digitalität, rasche technologische Entwicklungen und ingenieurwissenschaftliche Methoden eine wohlstrukturierte Planung erfordern, ist die Festlegung präziser Ziele von besonderer Bedeutung. Vor diesem Hintergrund gliedern wir das Vorgehen bei der Planung und Umsetzung von Informatikunterricht in sechs Ebenen (siehe Abb. 8.1).

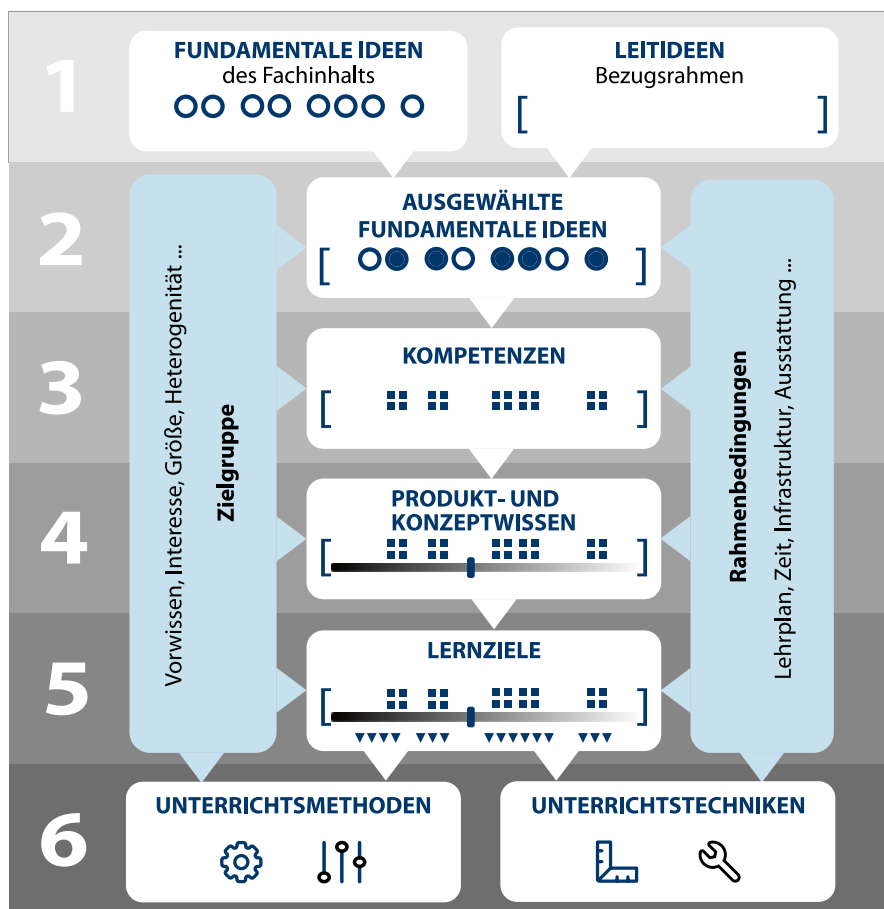


Abb. 8.1 Sechs Ebenen bei der Planung und Umsetzung von Informatikunterricht

Leitideen Leitideen stellen einen Bezugsrahmen dar. Sie begründen, warum etwas gelernt werden soll, und halten als Konsequenz daraus fest, was gelernt werden soll. Die Leitideen zeigen die Relevanz eines Themas auf und betten das Thema in einen umfassenderen fachlichen, gesellschaftlichen und wissenschaftlichen Kontext ein. Fachsystematische Strukturierungen wie die „Great Principles of Computing“ oder die Bildungsstandards unterstützen bei der Formulierung von Leitideen.

Fundamentale Ideen Zu jedem Thema gibt es fundamentale Ideen. Die Leitideen sowie Rahmenbedingungen und die Zielgruppe bestimmen, welche fundamentalen Ideen für den Unterricht ausgewählt werden.

Kompetenzen Bildung umfasst mehr als nur die Vermittlung von Fachwissen. Lernprozesse sollten so gestaltet werden, dass erworbenes Wissen auch im Alltag angewendet werden kann. Für den Begriff der Kompetenz gibt es viele unterschiedliche Definitionen. Etabliert hat sich die Definition von Weinert (2001): Er bezeichnet Kompetenzen als „die bei Individuen verfügbaren oder durch sie erlernbaren kognitiven Fähigkeiten und Fertigkeiten, um bestimmte Probleme zu lösen, sowie die damit verbundenen motivationalen, volitionalen und sozialen Bereitschaften und Fähigkeiten, um die Problemlösungen in variablen Situationen erfolgreich und verantwortungsvoll nutzen zu können“. Weinerts Kompetenzdefinition unterstreicht die Bedeutung der Bereitschaft und Fähigkeit, Problemlösungen in unterschiedlichen Situationen effektiv anzuwenden. Das Konzept der Disposition als innere Bereitschaft zur Verhaltensänderung spiegelt sich bereits in Blooms (1956) Lernzielklassifikation wider.

Konzept- und Produktwissen Sowohl Konzeptwissen als auch Produktwissen sind nötig, um Informatikmittel nutzbringend einsetzen zu können. Je nach Zielgruppe und Rahmenbedingungen bietet sich eine andere Gewichtung an.

Lernziele Während Kompetenzen beschreiben, was sich die Lernenden über eine längere Unterrichtssequenz hinweg aneignen sollen, beschreiben Lernziele konkret und überprüfbar, was die Schülerinnen und Schüler am Ende einer Unterrichtsstunde können sollten. Das Formulieren und Kommunizieren von Kompetenzen und Lernzielen hat positive Effekte sowohl für die Lernenden als auch für die Lehrkräfte. Die Lernenden wissen, warum sie etwas lernen sollen und welche Lernleistungen von ihnen erwartet werden.

Beim Planen von Unterricht helfen eine klare Struktur und ein geordneter Ablauf, Unterrichtsinhalte und angestrebte Kompetenzen umfassend zu berücksichtigen und den Unterricht effizient sowie schülergerecht zu gestalten.

Beispiel 1: Formulieren guter Prompts bei KI-Werkzeugen

Suchmaschinen liefern zu einer Suchanfrage eine Liste von Treffern in Form von Links auf mögliche relevante Dokumente, aus denen man sich die benötigten Informationen selbst zusammenstellt. Generative KI-Werkzeuge stellen ein Antwortdokument zu einer Anfrage zusammen. Sowohl bei Suchmaschinen als auch generativen KI-Werkzeugen ist das Formulieren einer guten Suchanfrage bzw. guter Prompts wichtig für die effiziente und effektive Recherche.

- *Leitidee:* Generative Sprachmodelle – wir sprechen im Folgenden von deren Anwendung in Form von Chatbots – werden in verschiedenen Bereichen genutzt. Dabei ist das Formulieren der Prompts für die Kommunikation mit einem Chatbot entscheidend. Je genauer die Prompts den Kontext der Anfrage und die Anforderungen an die Antwort beschreiben, desto hilfreicher sind in der Regel die Antworten des Chatbots.
- *Fundamentale Ideen:* Das Formulieren von Prompts für generative Sprachmodelle lässt sich gut mit der Kommunikation zwischen Menschen vergleichen. Genauso wie man einem Menschen (also einer natürlichen Intelligenz) präzise Fragen stellen muss, um eine hilfreiche Antwort zu erhalten, erfordert auch ein Sprachmodell (eine Künstliche Intelligenz) klare und gut strukturierte Prompts. Dabei hilft es, den Kontext zu geben, ähnlich wie man einem Freund zunächst die nötigen Hintergründe liefert, bevor man eine spezifische Frage stellt. Wie in einem Gespräch, bei dem man durch Nachfragen und Klarstellungen eine genauere Antwort erhält, können auch Prompts iterativ verfeinert werden, um das gewünschte Ergebnis zu erzielen. Und schließlich ist es wichtig, die Grenzen der Modelle zu kennen, so wie man auch die Fachgebiete und Einschränkungen eines menschlichen Gesprächspartners berücksichtigt.
- *Kompetenzen:* Die Lernenden reflektieren die Chancen und Herausforderungen beim Einsatz von Chatbots und entwickeln eine aufgeschlossene, aber zugleich reflektierte Einstellung gegenüber KI-Werkzeugen. Die Schülerinnen und Schüler erarbeiten selbst Ansätze zur steten Verbesserung der eigenen Nutzung von KI-Werkzeugen.
- *Lernziele:* Die Lernenden beschreiben in eigenen Worten, wie KI-Werkzeuge funktionieren. Sie formulieren zu vorgegebenen Fragestellungen zielführende Prompts und beurteilen die Antworten von Chatbots für unterschiedliche Eingaben hinsichtlich ihrer Genauigkeit und Nützlichkeit.

Beispiel 2: Betrug im Internet

Wir nutzen Onlineplattformen, um etwas zu kaufen, Dinge zu ersteigern, oder für Finanztransaktionen. Dabei steht Geld auf dem Spiel, weshalb Anwender ein Grundverständnis für die wichtigsten Bedrohungen und Sicherheitsmaßnahmen haben sollten. Ein wichtiger Aspekt dabei sind Betrügereien.

- *Leitidee:* Wie überall im Alltag besteht auch bei der Nutzung des Internets die Gefahr, auf Betrüger hereinzufallen. Für einige Betrügereien können Internetnutzer konkrete Maßnahmen treffen, für andere ist nur Sensibilisierung möglich.
- *Fundamentale Ideen:* Die Motivation der Betrüger ist im Internet nicht anders als in der Offlinewelt: Sie wollen an Geld kommen mithilfe von geeigneten

Tricks. Unterschiede gibt es bei den Techniken, die sich die besonderen Eigenschaften des Internets zunutze machen. Zum Beispiel die Automatisierbarkeit und die gute Erreichbarkeit von großen Mengen von Nutzern. Das führt dazu, dass sich auch Betrügereien mit kleinen Geldbeträgen lohnen, falls genügend Leute darauf reinfallen.

- *Kompetenzen:* Die Lernenden erkennen die wichtigsten Betrugsmaschinen, denen sie im Internet begegnen können. Sie übertragen dieses Wissen auf ihr eigenes Nutzungsverhalten und lassen entsprechend Vorsicht walten oder ergreifen geeignete Sicherheitsmaßnahmen.
- *Lernziele:* Die Lernenden erläutern wesentliche Betrugsarten wie Phishingmails, Social Engineering, Keylogger Malware und Identity Spoofing. Sie beschreiben für jede dieser Betrugsarten eine Verhaltensregel, die sie zukünftig vor solchen Angriffen schützen soll.

Beispiel 3: Adressierung in Netzprotokollen

Bei den Absolventen der Fachrichtung Systemtechnik an einer Berufsschule handelt es sich um Personen, die später unter anderem mit dem Aufbau und der Administration eines lokalen Netzes oder mit der Installation und der Wartung eines Web-servers betraut werden. Dazu benötigen sie einen Einblick in die Funktionsweise verschiedener Netzprotokolle, insbesondere der TCP/IP-Protokollfamilie.

- *Leitidee:* Die Adressierung stellt einen wichtigen Aspekt in Netzprotokollen dar und bildet eine Grundvoraussetzung für funktionierende Kommunikation. Angehende Systemtechniker müssen die theoretischen Grundlagen der Adressierung bei Netzprotokollen beherrschen und die Zusammenhänge auf den unterschiedlichen Ebenen verstehen (MAC-Adressen, IP-Adressen, TCP-Ports).
- *Fundamentale Ideen:* Die Grundlage der Adressierung in Netzprotokollen ist die Kapselung, also das Delegieren von Aufgaben an unterschiedliche Schichten und Instanzen.
- *Kompetenzen:* Die Lernenden unterscheiden in ihrem beruflichen Alltag Adressierung in verschiedenen Schichten und grenzen darauf aufbauend beispielsweise Fehler systematisch ein.
- *Lernziele:* Die Lernenden skizzieren ohne Hilfsmittel verschiedene Protokollschichten, ordnen jeder Schicht die entsprechende Adressierung zu und erläutern deren Funktion (z. B.: MAC-Adressen identifizieren Netzkarten im lokalen Netzwerk, IP-Adressen identifizieren Netzschnittstellen über größere Distanzen und TCP-Ports identifizieren Applikationen). Außerdem ermitteln sie eigenständig auf einem Windows- oder Linux-Rechner die MAC- und IP-Adresse und lassen die Routingtabelle anzeigen.

Beispiel 4: Grundlagen von Compilern

Eine der ersten Hürden beim Einstieg ins Programmieren sind die Fehlermeldungen des Compilers. Für viele Neulinge ist es frustrierend, Fehler wie vergessene Semikolons im Programmtext beheben zu müssen, bevor das Programm endlich ausgeführt wird. Angehende Programmierer sollten deshalb ein Grundverständnis der Funktionsweise eines Compilers haben.

- *Leitidee:* Programmierer sollten den Weg vom Quelltext zum ausführbaren Programm kennen und wissen, welche Stationen der Quelltext bis zur Umwandlung in Maschinencode durchläuft, beispielsweise die Analysephase mit der lexikalischen, syntaktischen und semantischen Analyse.
- *Fundamentale Ideen:* Zum Beispiel Parsingstrategien (top-down, bottom-up etc.), Unterschied zwischen Syntax und Semantik, andere Aspekte von Grammatiken oder die Entkopplung der einzelnen Komponenten eines Compilers (Parser, Optimierer, Code-Generator) durch Zwischencode.
- *Kompetenzen:* Programmierer ordnen Compiler-Fehlermeldungen ein und reagieren angemessen darauf. Sie betrachten Fehlermeldungen als einen normalen Bestandteil ihrer Arbeit und reagieren nicht mit Frustration.
- *Lernziele:* Programmierer beschreiben ohne Hilfsmittel die einzelnen Phasen der Quellcode-Übersetzung durch einen Compiler (Scanning, Parsing, Code-Generierung, Optimierung). Zudem ordnen sie gängige Fehlermeldungen des Java-Compilers (z. B. „} missing“ oder „method indexOf in the type String not applicable for argument (int)“) den jeweiligen Phasen zu und beheben die damit verbundenen Fehler.

Literatur

- Bloom, B. S. (1956). *Taxonomy of educational objectives*. Longmans.
Weinert, F. E. (2001). *Leistungsmessungen in Schulen*. Beltz.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Planung von Informatikschulungen

9

Einstieg: B. ist Dozent an einer Fachhochschule, sein Schwerpunkt sind Internet-technologien. Dieses Wissen möchte er auch im Rahmen von Weiterbildungen vermitteln. Er bietet Schulungen zur Funktionsweise, Installation und Konfiguration moderner Webserver an. Als sich der erste Tag dem Ende zuneigt, ist B. frustriert: Er hat die Zeit völlig unterschätzt und konnte nur zwei Drittel der Inhalte behandeln. Zudem stürzte der Server während einer wichtigen Übung ab. Das Feedback der Teilnehmer fällt entsprechend negativ aus, und B. fragt sich, wie er es beim nächsten Mal besser machen kann.

Informatiklehrkräfte unterrichten nicht nur an ihren Schulen, sondern geben auch Schulungen. Aufgrund der unterschiedlichen Rahmenbedingungen erfordern Schulungen eine andere Herangehensweise als Unterricht, der sich über mehrere Wochen erstreckt.

Ein- oder mehrtägige Schulungen in der Fortbildung oder innerbetrieblichen Weiterbildung können nicht in gleicher Weise geplant werden wie Informatikunterricht an einer Schule. Die wichtigsten Unterschiede:

Zielgruppe Im Unterricht trifft man auf Lernende, die je nach Schulstufe mehr oder weniger freiwillig teilnehmen. Das Fach Informatik wird in der Schule meist nicht aus besonderem Interesse, sondern als Teil des Curriculums besucht. An Schulungen hingegen nehmen in der Regel Erwachsene teil, oft aus freiem Antrieb, um sich weiterzubilden. Im Gegensatz zu Schülerinnen und Schülern müssen sie für die Schulung bezahlen oder die Vorgesetzten von der Relevanz für das Unternehmen überzeugen.

Lerninhalt In der Schule gibt der Lehrplan die grobe Richtung der Lerninhalte für einen längeren Zeitraum vor. Diese Inhalte sind meist allgemeinbildend, ihre

Relevanz wird längerfristig betrachtet. In einer Schulung dagegen erwarten die Teilnehmerinnen und Teilnehmer, zu einem spezifischen Thema etwas Konkretes zu lernen, das ihnen unmittelbar im Beruf nützlich ist. Die Lehrkräfte sind deshalb versucht, sich auf die Vermittlung spezifischer Fertigkeiten und Techniken zu konzentrieren. Dennoch kann es von Vorteil sein, den Spagat zwischen Konzepten einerseits und dem unmittelbar nutzbaren Produktwissen andererseits zu wagen.

Lernziele Im Schulunterricht werden Lernziele über einen längeren Zeitraum verfolgt. In Schulungen hingegen ist der Zeitraum begrenzt, weshalb die Planung detaillierter Lernziele besonders wichtig ist. Plastisch formulierte Lernziele in der Ausschreibung können außerdem die Erwartungen der Teilnehmenden steuern.

Zeitmanagement Erstreckt sich der Unterricht über mehrere Wochen, kann bei mangelhafter Zeitplanung, technischen Problemen oder anderen Unvorhersehbarkeiten in einer späteren Stunde fortgefahren werden. Bereitet ein Thema besondere Schwierigkeiten, kann das Tempo angepasst werden. Demgegenüber findet eine Schulung einmalig statt, der zeitliche Rahmen ist fest vorgegeben. Bei der Vorbereitung müssen daher verschiedene Szenarien bedacht werden: Welche Inhalte sind unverzichtbar, welche können notfalls verkürzt oder ganz gestrichen werden? Ist genügend Pufferzeit eingeplant für unvorhergesehene Störungen?

Vorkenntnisse In Schulungen ist wenig über die Vorkenntnisse der Teilnehmenden bekannt, sie sind oft stark heterogen. Die Ausschreibung sollte deshalb die notwendigen Vorkenntnisse präzise beschreiben. Einige Anbieter stellen Onlineselbsttests zur Verfügung. Die Planung sollte die Bandbreite der Vorkenntnisse berücksichtigen. Das bedeutet insbesondere, praktische Übungen individualisiert zu gestalten, also zugeschnitten auf den Kenntnisstand der einzelnen Teilnehmenden. Ziel sollte sein, dass jeder Teilnehmer bei den meisten Aufgaben ein Erfolgserlebnis erreichen kann.

Sozialer Rahmen Im Schulunterricht entwickelt sich über die Zeit eine Vertrautheit zwischen den Lehrkräften und den Schülerinnen und Schülern sowie unter den Lernenden selbst. Es entstehen unbewusste oder bewusste Spielregeln. Bei einer Schulung kennt die Lehrkraft die Teilnehmer nicht. Diese wiederum kennen höchstens einzelne andere. In firmeninternen Weiterbildungen gibt es unter Umständen hierarchische Verflechtungen, die der Lehrkraft nicht bekannt sind. Die Gestaltung der sozialen Interaktion sollte deshalb in die Planung einbezogen werden. Wie soll der Einstieg gestaltet werden? Was teilt die Lehrkraft über sich mit? Stellen sich die Teilnehmer gegenseitig vor? Wie werden Gruppen für Partner- oder Gruppenarbeiten gebildet? Wie kann in der kurzen Zeit ein offenes Klima geschaffen werden, in dem sich die Teilnehmenden wohlfühlen und Fragen stellen?

Infrastruktur Technische Ausfälle wie ein defekter Beamer oder eine unterbrochene Internetverbindung sind in einer eintägigen Schulung gravierend. Bei der Vorbereitung sollte eingeplant werden, wie man mit einer plötzlichen technischen

Störung umgeht. Wo gibt es eine Back-up-Präsentationsmöglichkeit? Wo einen Ersatzrechner? Sind die Folien auf dem Kursleiter-Laptop gespeichert? Was tun, wenn die Internetverbindung ausfällt oder die Übungsrechner nicht funktionieren?

Die Lehrkräfte müssen sich mit den besonderen Eigenheiten und Erfordernissen von Schulungen auseinandersetzen. Die Lernziele und der zeitliche Ablauf sind sorgfältig zu planen und Übungen am Computer individualisiert zu gestalten. Zudem sind hohe Anforderungen an die Infrastruktur zu stellen. Mögliche Probleme und Komplikationen sollten bereits im Vorfeld berücksichtigt und entsprechende Lösungsansätze entwickelt werden.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Teil IV

Unterrichtsmethoden

Einstieg: M. ist stolz auf ihre Unterrichtseinheiten. Sie sind immer nach dem gleichen, erprobten Muster aufgebaut: Zuerst gibt es eine Einführung ins Thema. Worum geht es, warum sind die Inhalte wichtig, was wird man nachher können? Dann folgen ein Beispiel und sorgfältig vorbereitete Übungen am Computer. Zum Abschluss werden Fragen diskutiert und Musterlösungen verteilt. Während viele Schüler diese klare Struktur schätzen, beklagen sich einige über Monotonie. Kürzlich kritisierte eine ganze Klasse, der Unterricht sei langweilig und eintönig.

Informatikunterricht besteht häufig aus Theoriephasen und praktischer Umsetzung. Im Theorieteil kommen Methoden wie der fragend-entwickelnde Ansatz oder Lehrervorträge zum Einsatz. Auf die Dauer kann diese Unterrichtsgestaltung monoton werden. Zudem können je nach zu vermittelnden Kompetenzen andere Methoden geeigneter sein.

Schülerinnen und Schüler haben unterschiedliche Lernpräferenzen: Einige bevorzugen klar strukturierten Unterricht, während andere Freiraum zum Entdecken benötigen. Unterschiede in Auffassungsgabe, Denkstilen und Geschlechtern erfordern eine Vielfalt an Methoden. Ein einziges Unterrichtskonzept kann diesen Ansprüchen nicht gerecht werden. Daher ist es wichtig, dass Lehrkräfte über ein Repertoire an Unterrichtsmethoden verfügen. Wir gehen hier nicht näher auf den Begriff „Unterrichtsmethode“ ein und führen auch keine Liste der gängigen Unterrichtsmethoden samt Vor- und Nachteilen an. Dazu verweisen wir auf die Fachliteratur, zum Beispiel die gute und prägnante Übersicht in Wiechmann und Wildhirt (2015), das Standardwerk zu Unterrichtsrezepten von Grell und Grell (2010) oder die praxisnahen Ausführungen von Meyer und Junghans (2022). Zender (2018) stellt 20 Unterrichtsmethoden samt Beispielen für den Informatikunterricht steckbriefartig vor.

Faktoren wie außerschulisch erworbene Informatikkenntnisse, Angebote in den vorangehenden Schulstufen, private IT-Infrastruktur, ICT-Bedienfertigkeiten und geschlechtsspezifische Interessen führen zu großer Heterogenität im Informatikunterricht, weshalb der geschickten Wahl der Unterrichtsmethode eine besondere Bedeutung zukommt. Ein lehrerzentrierter Ansatz kann dazu führen, dass einige Schülerinnen überfordert sind, während andere sich langweilen. Bei der praktischen Arbeit am Computer ist ein einheitliches Lerntempo kaum möglich. Daher ist Individualisierung unerlässlich.

Ein weiterer Aspekt ist die Detailvielfalt vieler Inhaltsbereiche: Bei Betriebssystemen oder Programmiersprachen, aber auch bei Anwendungsprogrammen macht es wenig Sinn oder ist es schlicht unmöglich, im Unterricht alle Details zu behandeln. Vielmehr muss den Lernenden die Methodenkompetenz vermittelt werden, sich das notwendige Wissen „just in time“ anzueignen. Lehrervorträge können diese Kompetenz schwer vermitteln, weshalb andere Unterrichtsformen gefragt sind.

Prinzipiell eignen sich alle gängigen Unterrichtsmethoden auch für den Informatikunterricht. Wir stellen in den folgenden Kapiteln sieben Unterrichtsmethoden vor, die den spezifischen Anforderungen des Informatikunterrichts besonders gut gerecht werden:

Lernaufgabe Lernaufgaben sind Unterrichtssequenzen, die behandelte Inhalte vertiefen und erweitern. Sie eignen sich besonders für individualisierte Übungen am Computer.

Puzzlemethode Diese Gruppenarbeitsmethode fördert Teamwork und eignet sich insbesondere für Themen, die nicht in der ganzen Breite behandelt werden können oder müssen.

Pair Learning Pair Learning reduziert die kognitive Belastung bei komplexen Problemen, indem die beiden Lernenden unterschiedliche Perspektiven einnehmen. Durch den sozialen Austausch können effizientere und kreativere Lösungen als beim individuellen Arbeiten erzielt werden.

Rollenspiel Rollenspiele ermöglichen das Verstehen komplexer informatischer Prinzipien durch Perspektivenwechsel und helfen, abstrakte Konzepte oder gesellschaftliche Wechselwirkungen zu erforschen.

Leitprogramm Leitprogramme sind umfangreiche Selbststudienmaterialien, die auf dem Mastery-Learning-Prinzip basieren. Sie eignen sich für heterogene Lerngruppen oder anspruchsvolle Themen im Informatikunterricht.

Entdeckendes Lernen Diese Methode legt Wert auf selbstständiges Arbeiten, Kreativität und kritische Reflexion. Entdeckendes Lernen ergänzt die Theorievermittlung im Informatikunterricht.

Projektunterricht Projekte bieten Einblicke in die Entwicklung und Wartung komplexer Informatiksysteme und sind ideal, um praktische Erfahrungen zu sammeln und Problemlösungsfähigkeiten zu entwickeln.

Für den Informatikunterricht speziell geeignet sind Methoden zur Individualisierung des Unterrichts und Unterrichtsmethoden, die schwierigen, abstrakten Sachverhalten sowie Themen mit einer großen Detailvielfalt Rechnung tragen.

Literatur

- Grell, J., & Grell, M. (2010). *Unterrichtsrezepte*. Beltz Verlag.
- Meyer, H., & Junghans, C. (2022). *Unterrichtsmethoden*. Theorie- und Praxisband.
- Wiechmann, J., & Wildhirt, S. (2015). *Zwölf Unterrichtsmethoden – Vielfalt für die Praxis*. Beltz Verlag.
- Zendler, A. (2018). *Unterrichtsmethoden für den Informatikunterricht*. Springer Vieweg.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: T. legt großen Wert auf einen engen Praxisbezug und führt deshalb regelmäßig praktische Übungen durch. Angehende IT-Supporter bauen selbst Festplatten oder Speicherchips ein und aus, messen den Datendurchsatz im Netzwerk und konfigurieren Router oder Firewalls. Die Übungen sind regelmäßig mit Stress verbunden: Einige Kursteilnehmende kommen schnell voran, andere kämpfen mit unerwarteten Problemen. Versuche, die Übungen zu synchronisieren, scheitern regelmäßig. Und sobald die Kursteilnehmerinnen in ihre Aufgaben vertieft sind, werden die von T. nachgereichten Erläuterungen kaum mehr zur Kenntnis genommen. In einer der letzten Stunden forderte ein Teilnehmer verärgert, seine Aufgaben in Ruhe selbst bearbeiten zu können.

Praktische Übungen am Computer sind ein fester Bestandteil im Informatikunterricht. Unterschiedliche Arbeitstempi und unerwartete Probleme erschweren jedoch die Durchführung. Lernende schätzen es zudem nicht, wenn sie durch nachgereichte Erklärungen in ihrer Konzentration gestört werden.

Praktische Übungen müssen individualisiert gestaltet werden und dürfen nicht einfach aus dem schrittweisen Abarbeiten einer Bedienungsanleitung oder aus dem Abtippen von Programmcodes bestehen. Lernaufgaben individualisieren praktische Übungen, indem sie in kleine, selbstständig bearbeitbare Sequenzen unterteilt werden. Sie fördern Motivation und Eigenverantwortung, da sie Neues beinhalten und so gestaltet werden, dass möglichst alle Lernenden zu einem Erfolgserlebnis kommen. Die Lehrerin erklärt den Stoff nur so weit, dass die Lernenden ihn vertiefen und teilweise selbst erarbeiten können. Sie steht nicht mehr im Mittelpunkt und hat mehr Zeit für individuelle Fragen. Lernaufgaben haben zudem eine positive Nebenwirkung: Sie können das Selbstvertrauen steigern und die Selbsthilfe fördern. In der Informatik ist es wichtig, sich selbstständig in neue Themen einarbeiten zu können,

zum Beispiel anhand eines Tutorials. Diese Fähigkeit wird mit Lernaufgaben trainiert.

Die folgende Checkliste (angelehnt an Frey & Frey-Eiling, 2010) fasst wichtige Punkte für die Erstellung von Lernaufgaben zusammen:

Etwas Neues Lernaufgaben sind nicht einfach Übungs- oder Repetitionsaufgaben. Die Schülerinnen und Schüler müssen dabei etwas Neues lernen. Das Neue sollte eine kleine „Entdeckung“ sein.

Schriftliche Aufgabenstellung Lernaufgaben müssen schriftlich vorbereitet sein. Die Anweisungen sollten klar und prägnant sein und alle zur Lösung notwendigen Informationen beinhalten.

Hinweise zum Vorgehen Aufgabenstellung, Ziel und, falls angebracht, Tipps für den Lösungsweg werden vorgegeben.

Zeitdauer Lernaufgaben dauern in der Regel 10–30 min und fügen sich gut in Lektionen von 45–60 min ein. Im Idealfall enthält eine Lernaufgabe für die Schnellsten eine oder mehrere zusätzliche Aufgabenstellungen, die thematisch ergänzend, vertiefend oder kognitiv anspruchsvoller sein können.

Formale Antwortstruktur Die Lernenden sollten genau wissen, welche Leistung von ihnen in der Lernaufgabe erwartet wird. Eine klar vorgegebene Antwortstruktur unterstützt die Lernenden beim Bearbeiten der Aufgaben.

Schwierigkeitsgrad Die Lernaufgabe soll vom Großteil der Lernenden selbstständig und erfolgreich gelöst werden können.

Das Entwerfen eigener Lernaufgaben eignet sich auch als Auftrag in der Ausbildung angehender Informatiklehrkräfte. Eine kleine Sammlung von Lernaufgaben ist hilfreich beim Einstieg in eine Unterrichtstätigkeit.

Individualisierung ist im Informatikunterricht wichtig, besonders bei der Gestaltung von praktischen Übungen. Mit Lernaufgaben können kurze Unterrichtssequenzen von 10–30 min individualisiert werden: Die Lehrperson unterrichtet den ersten Teil der Stunde und stellt anschließend eine Lernaufgabe.

Beispiel 1: QR-Codes

Codes begegnen uns überall: auf der Verpackung unserer Lebensmittel, in Büchern, auf Eintrittskarten oder in Werbeanzeigen. Eine weitverbreitete Art ist der QR-Code. Im ersten, von der Lehrperson bereitgestellten Teil der Lernaufgabe werden der Aufbau und die Funktionsweise von QR-Codes detailliert beschrieben. Im zweiten

Abb. 11.1 Lernaufgabe
zu Rangierung von
Textdokumenten

INFORMATIONSBEDÜRFNIS:

Sie suchen nach Informationen zu einem Flugzeugunglück in Lockerbie.

Sie stellen die unten stehende Anfrage an das Suchsystem und erhalten ein relevanteres Dokument (links) und ein weniger relevantes (rechts).

ANFRAGE:

Flugzeug Unglück Lockerbie

RELEVANTERES DOKUMENT	WENIGER RELEVANTES DOKUMENT
Flugzeug bei Lockerbie abgestürzt. Die Behörden sprechen von einer Katastrophe.	Flugzeug bei Phnom Penh, Kambodscha, abgestürzt. Das Unglück sei auf menschliches Versagen zurückzuführen.

Notieren Sie eine Regel, die in diesem Fall ein relevanteres von einem weniger relevantem Dokument unterscheidet.

Wichtig dabei: Ein Suchsystem kann nur auf statistische Informationen zurückgreifen und nicht die gleichen kognitiven Gedankengänge anstellen wie ein Mensch.

Teil der Lernaufgabe recherchieren die Schülerinnen und Schüler selbst, wo QR-Codes bevorzugt zum Einsatz kommen. Sie verwenden Online-QR-Generatoren, um eigene QR-Codes zu erstellen, und untersuchen deren Anfälligkeit gegenüber Beschädigungen und Missbrauch sowie mögliche Schutzmaßnahmen. Für die schnelleren Schülerinnen und Schüler können zusätzliche Aufgaben gestellt werden: Sie beschäftigen sich damit, wie QR-Codes manuell decodiert werden können, oder entwickeln Ideen für innovative Anwendungen von QR-Codes an ihrer Schule.

Beispiel 2: Indexierung von Textdokumenten durch Suchmaschinen

Diese Lernaufgaben stammen aus Kompaktkursen zum Thema Information Retrieval. Die erste, in Abb. 11.1 verkürzt dargestellte Lernaufgabe beschäftigt sich mit der Rangierung von Dokumenten. Die Teilnehmer erhalten ein Arbeitsblatt mit einer Suchanfrage und zwei Dokumenten, von denen eines relevant ist. Sie sollen eine Regel entwickeln, die das relevantere Dokument identifiziert.

Die zweite, in Abb. 11.2 dargestellte Lernaufgabe geht auf die Indexierung von Textdokumenten ein. Bei dieser Lernaufgabe experimentieren die Kursteilnehmer am Rechner mit einem realen Suchsystem. Sie sollen herausfinden, wie die Indexierung bei einem Suchsystem ihrer Wahl funktioniert: Wird Groß- und Kleinschreibung unterschieden? Wie werden Umlaute behandelt? Werden Platzhalter unterstützt? Macht das Suchsystem eine Wortzerlegung? Dieses Wissen hilft den Kursteilnehmern, ihre Suchanfragen präziser zu stellen und die Resultate genauer zu analysieren.

Die formale Antwortstruktur dieser Lernaufgabe wird anhand einer Tabelle vorgegeben. Die erste Tabelle ist bereits mit einem fiktiven Beispiel ausgefüllt. Die zweite Tabelle ist leer und soll durch Experimentieren ausgefüllt werden.

Abb. 11.2 Lernaufgabe
zu Indexierung von
Textdokumenten

SUCHSYSTEM OMNISEARCH	
Groß-/Kleinschreibung	wird ignoriert ► <i>Bern, bern, bERn</i> finden immer dieselben Dokumente
Umlaute	Umlaute werden nicht besonders bearbeitet ► <i>münchen</i> findet andere Dokumente als <i>muenchen</i>
Platzhalter(Wildcards)	werden nicht unterstützt ► <i>bahnstei*</i> findet keine Dokumente
Wortzerlegung	keine Wortzerlegung ► <i>dampfschiffdeck</i> findet nichts; <i>dampf schiff deck</i> findet vieles

SUCHSYSTEM: _____	
Groß-/Kleinschreibung	
Umlaute	
Platzhalter(Wildcards)	
Wortzerlegung	

Beispiel 3: Quizgenerator in Scratch

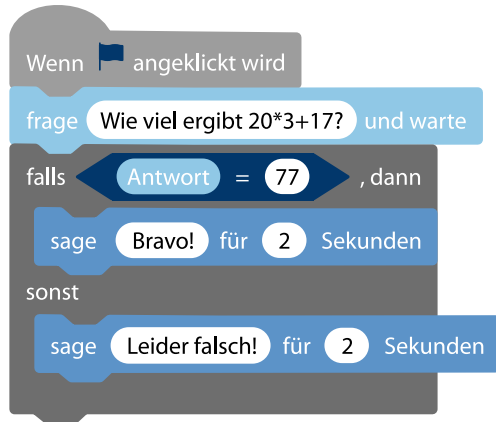
Scratch bietet eine visuelle Programmierumgebung, die besonders für den Einstieg ins Programmieren geeignet ist. Lernaufgaben eignen sich, um die verschiedenen Befehle der Programmiersprache kennenzulernen und selbst auszuprobieren. Ziel dieser Aufgabe ist es, ein interaktives Quiz zu programmieren und dabei das Verständnis für bedingte Anweisungen zu vertiefen. Zunächst wird wie in Abb. 11.3 ein einfaches Quiz mit nur einer Frage und einer richtigen Antwort vorgegeben.

Die Lernenden sollen das Quiz eigenständig um weitere Fragen erweitern. Schnelle Schülerinnen erhalten die Zusatzaufgabe, den Punktestand des Quiz mit einer Variablen zu erfassen und anzuzeigen.

Beispiel 4: Kompression von Daten

Lernaufgaben eignen sich gut in Kombination mit Lernumgebungen, mit denen die Lernenden experimentieren können. In einer Unterrichtseinheit zum Thema Datenkompression wird beispielsweise nach einer Einführung durch den Lehrer der Huffman-Algorithmus genauer unter die Lupe genommen: Die Lernenden erhalten

Abb. 11.3 Beispielcode eines einfachen Quizgenerators



mehrere, aufeinander aufbauende Lernaufgaben, die sie mithilfe der zugehörigen Lernumgebung beantworten sollen.

Ein erster Teil der Lernaufgabe könnte wie folgt aussehen:

„Beim Kofferpacken können wir durch Druck die Luft herauspressen und bringen so mehr Kleidungsstücke in den Koffer. Der Huffman-Code funktioniert nach dem gleichen Prinzip.

Aufgabe:

Wie muss der Eingabetext beschaffen sein, damit er sich gut komprimieren lässt?

Vorgehen:

Machen Sie sich zuerst mit der Lernumgebung vertraut. Komprimieren Sie verschiedene Wörter, um so auf die Lösung zu kommen. Zum Beispiel mit: können, Schifffahrtsgesellschaft, Griffbrett, Maus, AAAAAAABBCED, ABCDEFGHIJKL. Formulieren Sie Ihre Antwort in höchstens drei Sätzen.“

Literatur

Frey, K., & Frey-Eiling, A. (2010). *Ausgewählte Methoden der Didaktik*. vdf Hochschulverlag an der ETH Zürich.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Es steht einfach nie genügend Zeit zur Verfügung: In sechs Lektionen soll der effiziente Einsatz einer Präsentationssoftware vermittelt werden. Welche Funktionen der Präsentationssoftware soll man detailliert behandeln? Das Konzept der Masterfolien? Oder benutzerdefinierte Animationen? Einige Kursteilnehmerinnen interessieren sich spezifisch für das Einbinden von Audiodateien in verschiedenen Formaten. Und neben dem Produktwissen zu den Funktionen der verwendeten Software sollte auch das Konzeptwissen nicht zu kurz kommen: Was macht eine gute Präsentation aus? Welche Prinzipien des Multimediadesigns sollte man unbedingt berücksichtigen? Kurz: Der Funktionsumfang der Präsentationssoftware ist riesig und das Spektrum der Vorkenntnisse der Kursteilnehmer breit.

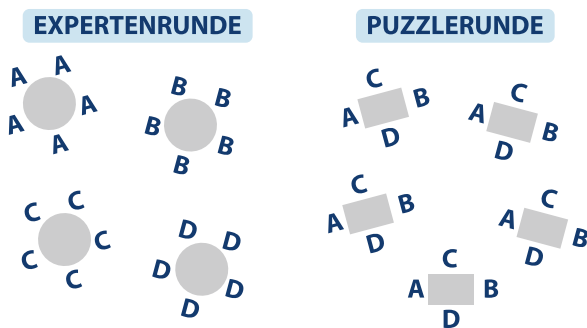
Die meisten Informatiksysteme zeichnen sich durch eine Fülle von Details aus, die nur ansatzweise im Unterricht behandelt werden können. Insbesondere beim Produktwissen variieren die Vorkenntnisse der Lernenden stark und es ist nicht leicht, den Unterricht so zu gestalten, dass er dem unterschiedlichen Wissensstand der Lernenden gerecht wird.

Aus Zeitgründen kann man im Unterricht in der Regel nicht alle Funktionen einer Software ausführlich behandeln. Produktabhängige Kenntnisse auf Vorrat zu vermitteln, macht wenig Sinn. Viele Lernende werden einen Teil der Funktionen erst zu einem späteren Zeitpunkt verwenden oder möglicherweise nie benötigen. Hinzu kommen neue Softwareversionen, in denen Funktionen anders umgesetzt sind.

Die Fülle an Details stellt aber nicht nur bei eher produktorientierten Schulungen eine Herausforderung dar. Auch Unterrichtsinhalte, die durch eine Fülle an Detailwissen gekennzeichnet sind, wie verschiedene Verschlüsselungs- und Sortieralgorithmen, können mangels Zeit meist nicht in der ganzen Breite behandelt werden.

Wichtiger als den Anspruch, die Funktionen einer Software oder alle Facetten eines Themas detailliert zu unterrichten, ist die Förderung der Methodenkompetenz

Abb. 12.1 Expertengruppen (links) und Puzzlegruppen (rechts)



des selbstständigen Lernens. Die Lernenden sollen sich im Bedarfsfall mittels geeigneter Informationsquellen die benötigten Kenntnisse eigenständig aneignen können. Die Unterrichtsmethode Gruppenarbeit ist eine Möglichkeit, den Unterricht individualisierend zu gestalten und selbstständiges Lernen zu fördern. Gruppenarbeiten fördern neben anderen Sozialkompetenzen das Vertrauen in die eigenen Fähigkeiten. Allerdings reicht dazu nicht einfach die Aufforderung der Lehrerin oder des Kursleiters, ein Thema in Gruppen zu bearbeiten. Ohne geeignete Maßnahmen übernimmt häufig das stärkste Mitglied der Gruppe die Verantwortung und erledigt die Arbeit selbst. Guter Gruppenunterricht muss deshalb dafür sorgen, dass alle Lernenden ihren Beitrag leisten. Dazu speziell geeignet ist die Puzzlemethode.

Wir erläutern die Methode anhand eines Beispiels mit vier Themen und 20 Lernenden. Die Lernenden werden wie in Abb. 12.1 dargestellt in vier sogenannte Expertengruppen mit je fünf Lernenden aufgeteilt. Jede Gruppe erhält das vollständige Material zu einem der vier Themen. Die Zuteilung der Lernenden zu den Themen kann von der Lehrkraft vorgenommen werden, oder die Lernenden teilen die Themen selbstständig unter sich auf. Berücksichtigt werden können dabei beispielsweise unterschiedliche Leistungsfähigkeiten und Interessen der Lernenden sowie ihre kommunikativen sozialen Kompetenzen.

In einer ersten Phase beschäftigen sich alle Mitglieder einer Expertengruppe selbstständig mit dem Thema. In der zweiten Phase kommen alle Mitglieder einer Expertengruppe zusammen, diskutieren offene Fragen und überlegen sich, wie sie das Thema den anderen Lernenden vermitteln können. Für die dritte Phase bilden sich neue Gruppen, die sogenannten Puzzlegruppen. Alle Lernenden unterrichten ihr Thema den drei anderen Lernenden ihrer Puzzlegruppe. Als Abschluss empfiehlt es sich, wenn die Lehrkraft offengebliebene Fragen aus der Puzzlerunde aufnimmt und gemeinsam mit den Lernenden klärt.

Die Lernenden bearbeiten bei der Puzzlemethode selbstständig ein überschaubares Thema und bilden sich zu Expertinnen und Experten aus. Genau diese Fähigkeit ist im Umgang mit Informatiksystemen und Informatikthemen gefragt. Durch das Vermitteln des erarbeiteten Wissens an andere wird zudem das Selbstwertgefühl gesteigert. Nicht zu unterschätzen ist, dass die Erklärungen von Schülerinnen in der Sprache der Schüler formuliert werden und deshalb oft besser verständlich und weniger abstrakt sind als die Erklärungen der Lehrkraft.

Wichtig beim Einsatz der Puzzlemethode ist, dass alle Lernenden ihre Aufgaben selbstständig bewältigen können. Allenfalls empfiehlt es sich, den schwächeren Lernenden einfachere Themen zuzuteilen. Besonders geeignet sind Themen, die sich in ungefähr gleich große Teile aufteilen lassen. Die Lernenden sollten mit der Weitergabe des erworbenen Wissens erst beginnen, wenn sie ihre Themen wirklich verstanden haben. Die Lehrkraft kann das Verständnis mit Kontrollfragen überprüfen.

Ein Nachteil der Puzzlemethode ist, dass nicht alle den gesamten Stoff gleich gut bearbeiten. Die Experten werden das eigene Thema besser verstehen. Möglicherweise schleicht sich beim Vermitteln in der Puzzlegruppe der eine oder andere Fehler ein. Geeignet ist die Puzzlemethode deshalb bei Themen mit einer Fülle von Einzelaspekten, die nicht vollständig bearbeitet werden müssen. Als Einführung in die grundlegenden Konzepte eines Themenbereichs empfiehlt sich die Puzzlemethode nicht.

Achtung: Die Puzzlemethode kann zu lebhafter Kommunikation führen, die chaotisch wirken mag, aber von den Lernenden als strukturiert und hilfreich empfunden wird.

Die Puzzlemethode ist eine kooperative Lernform, die sich speziell für den Informatikunterricht eignet. Sie empfiehlt sich bei Themen mit vielen Teilaspekten und fördert die Fähigkeit der Lernenden, sich selbstständig in neue Themen einzuarbeiten.

Beispiel 1: Computer – Schicht um Schicht

Beim Kauf eines Computers oder Smartphones wird man mit Begriffen wie „Hardware“ und „Software“, „Betriebssystem“, „Prozessor“, „Arbeitsspeicher“ usw. konfrontiert. Ein Verständnis für das Innenleben eines Computers und die verschiedenen Schichten seines Aufbaus sind hilfreich. In der Primarschule kann dieses Wissen mit der Puzzlemethode vermittelt werden, indem der Computer wie in Abb. 12.2 dargestellt mit einer Küche verglichen wird: Der Prozessor ist für die Verarbeitung im Computer zuständig, die Köchin in der Küche. In einer Restaurantküche fällt viel Arbeit an und es braucht flinke und meistens auch mehrere Köchinnen. Ebenso benötigt ein Computer je nach Einsatzgebiet mehrere oder leistungsfähigere Prozessoren. Zum Kochen braucht es Zutaten. Salz, Pfeffer und andere Gewürze sollten schnell greifbar sein, Fleisch findet sich im Kühlschrank, Kartoffeln und Gemüse lagert man vielleicht im Keller. Analog sind beim Computer Daten je nach Zugriffsart im Arbeitsspeicher oder auf der Festplatte als Datenspeicher gespeichert. Das Betriebssystem des Computers stellt eine Grundausrüstung dar, vergleichbar mit dem Kochherd, Pfannen, Kellen und mehr in der Küche.

Jeder Expertengruppe wird eines der vier Themen zugeteilt. Die Aufgabe besteht darin, die Begriffe „Prozessor“, „Arbeitsspeicher“, „Datenspeicher“ und „Betriebssystem“ zu verstehen und den Mitschülern in der Puzzlegruppe zu erläutern.

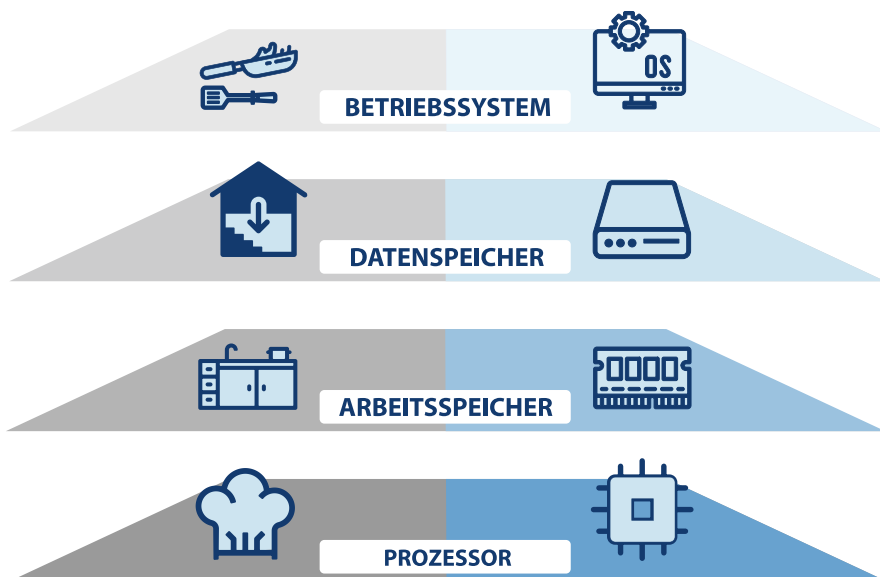


Abb. 12.2 Schichtenmodell zum Aufbau eines Computers; Illustration angelehnt an Hartmann (2019)

Ebenfalls sollen Spezifikationen gängiger Produkte (z. B. Prozessortyp und Leistung oder Speichergrößen) in Onlineshops recherchiert werden.

Nach dem Austausch zum Aufbau eines Computers in der Puzzlegruppe kann in einer zweiten, offeneren Puzzlerunde auch der Kauf eines geeigneten Computers, Tablets oder Smartphones bearbeitet werden. Den einzelnen Expertengruppen wird je ein Kundenprofil zugeteilt. Beispiele: Die Buchhalterin und Protokollführerin eines Vereins; ein Historiker, der die Geschichte einer Stadt mit viel Bildmaterial dokumentiert; ein im Fußball engagierter E-Sportler; eine Figurentheater-Spielerin, die in ihren Stücken oft kleine Videoanimationen einbaut.

Beispiel 2: Browsereinstellungen

Beim Thema Internet sollen in einer Schulung verschiedene Einstellungen wie Cachespeicher, Proxyserver, Verlauf, Cookies usw. thematisiert werden. Stellt der Kursleiter die Details selbst vor, ist die Gefahr groß, dass die Kursteilnehmerinnen nur wenig davon aufnehmen. Da es nicht entscheidend ist, dass alle Kursteilnehmer jedes Detail gleich gut verstehen, ist die Puzzlemethode geeignet, um einen Überblick zu den Browsereinstellungen zu geben.

Bei diesem Thema ist eine flexiblere Form des Puzzleunterrichts möglich: Der Kursleiter übergibt nicht jeder Expertengruppe das vollständige Material, sondern jede Expertengruppe trägt selbst Hintergrundinformationen zu ihrem Thema zusammen und klärt ab, wo die entsprechenden Einstellungen in den jeweiligen Browsern vorgenommen werden. Diese offene Form eines Puzzles vermittelt die nötige

Methodenkompetenz, um bei anderen Programmen analoge Abklärungen selbstständig vorzunehmen.

Beispiel 3: Sortierverfahren

Es gibt verschiedene Sortierverfahren, die je nach Problemstellung unterschiedlich geeignet sind. Im Informatikunterricht steht nicht genügend Zeit zur Verfügung, um alle gängigen Sortieralgorithmen im Detail zu behandeln und als Programme zu implementieren. Die Behandlung einzelner Verfahren in Gruppen nach der Puzzlemethode drängt sich geradezu auf: Jeder Expertengruppe wird ein Verfahren zugewiesen. Je nach Altersstufe und Zielsetzung kann der Auftrag an die Expertengruppe verschieden formuliert werden. So kann eine kurze Beschreibung des Verfahrens abgegeben oder von der Gruppe selbst recherchiert werden. Neben dem Algorithmus können die Laufzeit- und Speicherplatzkomplexität betrachtet werden. Zusätzlich zu gängigen Verfahren wie Bubblesort oder Quicksort können auch Verfahren wie Countingsort oder Bucketsort mit linearer Laufzeit untersucht werden, die auf spezifischen Eigenschaften der Eingabedaten beruhen.

Beispiel 4: Informationssysteme in der Luftfahrt

Die Stärken der Puzzlemethode zeigen sich auch in Schulungen für technologisch hochkomplexe Berufsfelder wie die Luftfahrt. Die Digitalisierung hat das Umfeld und die Erwartungen an die Pilotinnen und Piloten stark verändert: Digitale Systeme haben analoge Instrumente abgelöst. Autopilotensysteme automatisieren viele Funktionen, müssen aber von den Pilotinnen weiter überwacht und verstanden werden. Piloten müssen auch wissen, wie sie auf Fehlermeldungen der Kontrollsysteme reagieren sollen und wie sie schnell in den elektronischen Informationssystemen Herstellerhinweise zur Problembehandlung finden.

Früher orientierte sich das Training von Cockpitcrews an von den Flugzeugherstellern zur Verfügung gestellten Handbüchern. Diese sind heute abgelöst durch digitale Informationssysteme, sodass es in der Aus- und Fortbildung gar nicht möglich ist, alle denkbaren Sicherheitsprobleme zu behandeln. Von den Piloten wird eine hohe Eigenverantwortung und Selbstregulation verlangt, um etwa in Notfällen gute Entscheidungen zu treffen. Hier bietet sich die Puzzlemethode an: Im Training werden die Expertengruppen mit unterschiedlichen sicherheitsrelevanten Problemen konfrontiert. Sie diskutieren mögliche Vorgehensweisen, wie die verfügbaren Informationssysteme genutzt werden können, und evaluieren die Problemlösungen (z. B. im Flugsimulator). In den Puzzlegruppen werden dann die Erkenntnisse und Erfahrungen ausgetauscht.

Im Vordergrund einer solchen Schulung steht nicht das Wissen um das Lösen aller denkbaren Probleme, was angesichts der Komplexität schlichtes Wunschdenken darstellt. Vielmehr vermittelt die Puzzlemethode die nötige Methodenkompetenz zum Umgang mit Problemen und stärkt das Selbstvertrauen der Pilotinnen, auch schwierige Situationen angemessen meistern zu können.

Beispiel 5: Nachhaltigkeit und Sozialverträglichkeit von IT

Die Herstellung und Nutzung von Computern, Smartphones und anderen IT-Geräten haben weitreichende Auswirkungen auf Umwelt und Gesellschaft. Im Rahmen der Puzzlemethode können Lernende sich mit verschiedenen Aspekten der Nachhaltigkeit und Sozialverträglichkeit in der IT-Branche beschäftigen:

Abbau und Gewinnung von Rohstoffen: Zur Herstellung von Computern und Smartphones werden seltene Erden wie Kobalt und Lithium benötigt. Der Abbau dieser Rohstoffe ist mit erheblichen ökologischen Belastungen und sozialen Herausforderungen verbunden. Die Expertengruppe untersucht die Umweltfolgen des Rohstoffabbaus, wie etwa die Zerstörung von Lebensräumen und Wasserverschmutzung, sowie die oft prekären Arbeitsbedingungen in den Abbaubereichen.

Herstellung und globale Lieferketten: Die Produktion elektronischer Geräte ist auf komplexe globale Lieferketten angewiesen, die von wenigen dominanten Herstellern und Monopolen kontrolliert werden. Die Expertengruppe analysiert die wirtschaftlichen Abhängigkeiten, die sich daraus ergeben, und diskutiert die sozialen und ökologischen Kosten der globalen Produktion.

Energieverbrauch und Wasserbedarf von IT-Geräten und Rechenzentren: Der Betrieb von IT-Geräten und Rechenzentren verursacht einen erheblichen Energieverbrauch. Große Cloud-Dienste sind energieintensiv, und viele Rechenzentren nutzen Wasser zur Kühlung ihrer Server. Dies führt in manchen Regionen zu einer Konkurrenz um Wasserressourcen und belastet die lokalen Gemeinschaften. Die Expertengruppe analysiert den Strom- und Wasserverbrauch von IT-Geräten und Rechenzentren.

Lebensdauer und Entsorgung von Elektronikgeräten: Die begrenzte Lebensdauer vieler Elektronikgeräte trägt zur wachsenden Menge an Elektroschrott bei. Dieser wird häufig unter unsachgemäßen Bedingungen entsorgt, was die Umwelt mit giftigen Substanzen belastet. Die Expertengruppe untersucht die Herausforderungen des Recyclings und stellt Ansätze vor, um die Nutzungsdauer von Geräten zu verlängern, etwa durch Reparierbarkeit, Wiederverwendung und die Förderung einer Kreislaufwirtschaft, in der Materialien wiederverwertet werden.

In den Expertengruppen erarbeiten die Lernenden jeweils vertiefte Kenntnisse zu einem dieser Themen. In den Puzzlegruppen geben die Lernenden ihr Wissen an ihre Kolleginnen und Kollegen weiter und entwickeln gemeinsam ein Verständnis für die komplexen Herausforderungen im Bereich der nachhaltigen und sozialverträglichen Herstellung und Nutzung von IT-Systemen.

Literatur

Hartmann, W., et al. (2019). *connected 02 – Lehrmittel Medien und Informatik*. Lehrmittelverlag Zürich.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Im Ausbildungsgang „Datenbankmanagement“ entwirft Berufsschüler F. eine Datenbank für Autovermietungen. Er legt Tabellen für Fahrzeuge, Kunden und Reservierungen an, definiert Felder für Automarken, Mietpreise und Buchungszeiträume. Die Struktur sieht auf den ersten Blick sauber aus, doch bereits beim ersten Test treten Probleme auf: Das System kann keine Fahrzeuge verwalten, die in mehreren Filialen verfügbar sind, und es fehlt eine Möglichkeit, Zusatzleistungen wie Kindersitze oder Navigationssysteme zu erfassen. Zudem erlaubt das Design nicht, mehrere Fahrer für eine Miete einzutragen, was in der Praxis häufig vorkommt. Die Datenbank ist funktional, aber nicht praxistauglich.

Bei der Entwicklung von Informatiklösungen gilt es, eine Menge an Details zu berücksichtigen. Verschiedene Ansätze müssen abgewogen und Entscheidungen getroffen werden. Eine Person allein übersieht dabei schnell wichtige Aspekte oder fühlt sich von der Aufgabe überfordert.

Informatiklösungen sind meist vielschichtig und gerade in der Konzeptionsphase mit Unsicherheiten verbunden. Verschiedene Methoden helfen, mit dieser Komplexität umzugehen. Eine bewährte Methode ist, zu zweit einander zu unterstützen und so ein Problem besser zu lösen. In der Softwareentwicklung ist diese Methode als „Pair Programming“ seit Langem etabliert und wird als eine effektive Form der Zusammenarbeit geschätzt. Beim Pair Programming arbeiten zwei Entwicklerinnen gemeinsam an einem Stück Code. Die Idee dahinter: Vier Augen sehen mehr als zwei. Durch die Zusammenarbeit werden Fehler schneller erkannt, was zu einer höheren Codequalität und weniger Bugs führt. Zudem empfinden viele Entwickler die Methode als motivierend, da sie weniger isoliert sind und aktiv Ideen und Lösungen austauschen können. Im Pair Programming gibt es zwei Rollen: den „Driver“, der den Code schreibt, und den „Navigator“, der den Überblick behält und strategische Hinweise gibt. In regelmäßigen Abständen werden die Rollen getauscht – ähnlich

wie Fahrer und Beifahrer beim Autofahren. In der Praxis der Softwareentwicklung findet dieser Rollentausch meist nicht wirklich statt, entscheidend ist die gemeinsame Diskussion. Im Unterricht macht der regelmäßige Rollentausch Sinn und fördert den Austausch zwischen den zwei Lernpartnern.

Die Unterrichtsmethode „Pair Learning“ lehnt sich an den bewährten Ansatz des Pair Programming an und ist in der Didaktik als Partnerarbeit, die kleinste Form der Gruppenarbeit, bestens verankert. Die Methode kann nicht nur beim Programmieren, sondern in vielen Bereichen des Unterrichts eingesetzt werden, besonders für Aufgaben, die viel Denkarbeit und Unsicherheit mit sich bringen. Zu zweit fühlt man sich beim Fällen von Entscheidungen sicherer und die kognitive Last der Aufgaben wird geteilt. Während eine einzelne Person dazu neigt, sich auf ihre vertrauten Werkzeuge und Ansätze zu verlassen, eröffnet die gemeinsame Diskussion über Methoden und Strategien oft neue, effizientere Wege. Praktisch umsetzbar ist Pair Learning vor allem im Kleinen bei anspruchsvollen Aufgaben. Sobald die Aufgaben klarer definiert und leichter aufteilbar sind, kann die intensive Zusammenarbeit enden – es geht nicht um „Pairing um des Pairings willen“.

Natürlich gibt es immer wieder Lernende, die Vorbehalte gegenüber dieser Methode haben, oft aus Angst, sich vor dem Partner zu blamieren. Deshalb ist es wichtig, solche Ängste gezielt anzusprechen und bei der Bildung der Paare darauf zu achten. Pair Learning funktioniert dann besonders gut, wenn die Aufgabe so gestellt wird, dass beide Lernenden bei der Bearbeitung aufeinander angewiesen sind. Auch muss die Lehrkraft Maßnahmen ergreifen, um sicherzustellen, dass sich beide Partner wirklich aktiv beteiligen und sich nicht einer in die passive Rolle zurückzieht.

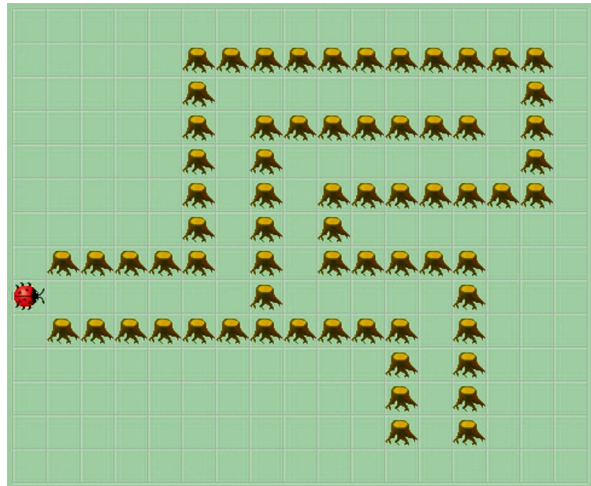
Erfordert eine Aufgabe viel Denkarbeit, bringt sie viel Unsicherheit mit sich und müssen laufend Entscheidungen getroffen werden, reduziert Pair Learning die kognitive Last beim Lösen der Aufgabe. Gleichzeitig wird der soziale Austausch gefördert und die Verantwortung für die Lösung geteilt, was für viele Lernende eine Erleichterung darstellt.

Beispiel 1: Pair Programming beim Steuern eines Roboters

Einführende Programmierumgebungen wie Karol the Robot, Hamster oder Kara bieten eine stark vereinfachte, visuell unterstützte Plattform, in der ein steuerbarer Akteur (z. B. ein Roboter oder Käfer) auf einem rasterartigen Spielfeld agiert. Die Schülerinnen und Schüler schreiben einfache Programme, um den Akteur zu bewegen, Hindernisse zu erkennen oder Objekte zu manipulieren. Dadurch werden grundlegende Konzepte wie Sequenzen, Schleifen und Bedingungen auf eine intuitive Weise vermittelt, ohne dass komplexe Syntax oder abstrakte Programmierparadigmen im Vordergrund stehen. In der Aufgabe in Abb. 13.1 soll der Marienkäfer durch Tunnel mit beliebig komplexen Formen gesteuert werden.

Die scheinbar einfache Aufgabe hat ihre Tücken. Schnell übersieht man eine mögliche Tunnelform, und der Akteur bewegt sich wieder rückwärts oder verharrt an

Abb. 13.1 Steuerung eines Aktors durch einen Tunnel. (Programmierung KaraToJava, swisseduc.ch)



Ort und Stelle. Zu zweit können die verschiedenen Tunnelformen gründlicher getestet und mögliche Fehler schneller erkannt werden.

Beispiel 2: Planung einer Website für ein mittelgroßes Unternehmen

Die Planung einer Website für ein mittelgroßes Unternehmen erfordert sorgfältige Überlegungen und eine klare Strategie. Es gilt, die Zielgruppe und deren Erwartungen an die Website zu verstehen, aber auch welche Dienstleistungen und Produkte das Unternehmen anbietet. Sowohl die Sicht des Unternehmens als auch die der Websitebesucher beeinflussen die Struktur der Website und die Navigation. Auch eher technische Aspekte gilt es zu berücksichtigen, etwa wer die Website warten und aktualisieren wird, oder die Optimierung für Suchmaschinen. Dazu müssen die finanziellen Mittel und personellen Ressourcen des Unternehmens für die Umsetzung und den späteren Betrieb der Website ausreichend sein.

Im Unterricht des Ausbildungsganges „Mediamatik und Webdesign“ einer Berufsschule teilt die Lehrerin die Klasse in Zweiergruppen auf. In einer ersten Phase geht es um das Festlegen der Grobstruktur der Website mittels eines Mock-up-Tools. Eine Lernende übernimmt dabei die Rolle der Kundschaft des Unternehmens, die andere die Sichtweise des Unternehmens. In einer zweiten Phase wird der Entwurf der Website zu zweit hinterfragt. Eine Lernende macht sich dabei vielleicht eher Gedanken zur technischen Umsetzung und den resultierenden Kosten. Die andere Lernende wirft die Frage auf, ob sich die Struktur für laufende Aktualisierungen durch Mitarbeitende des Unternehmens eignet.

Beispiel 3: P-AI-R Programming mit generativem Sprachmodell

In einer abgewandelten Version des Pair Programming wird eine Person durch einen KI-Assistenten ersetzt, der speziell für Programmieraufgaben trainiert wurde. Der Vorteil: Der KI-Assistent steht jederzeit zur Verfügung und agiert immer freundlich.

In der Softwareentwicklung werden KI-Assistenten häufig genutzt, um Programmfragmente zu erstellen, die zwar keine große Herausforderung darstellen, deren Entwicklung aber erheblichen Zeitaufwand beansprucht. In fortgeschrittenen Projekten können Schülerinnen und Schüler ähnlich vorgehen.

Auch beim Einstieg ins Programmieren können KI-Assistenten Unterstützung bieten. Eigene Codestücke können einem KI-Assistenten zur Begutachtung vorgelegt werden oder der KI-Assistent hilft beim Debuggen. Eine Schülerin hat zum Beispiel ein Python-Programm geschrieben, das den fortlaufenden Wurf eines Würfels simuliert und zählt, wie lange es jeweils dauert, bis eine Sechs gewürfelt wird:

```
from random import randint
augenzahl=randint(1,6)
zaehler=0
while augenzahl!= 6:
    augenzahl = randint(1,6)
    zaehler = zaehler+1

print('Anzahl der Versuche:',zaehler)
```

Der KI-Assistent meldet zurück, dass das Programm grundsätzlich richtig ist, mit einer Ausnahme: Falls die erste geworfene Augenzahl bereits eine Sechs ist, wird dieser Wurf nicht gezählt und als Anzahl Versuche „Null“ ausgegeben. Außerdem gibt der KI-Assistent den Hinweis, dass mit der Methode `seed()` der Zufalls-generator initialisiert werden kann, sodass nicht standardmäßig die Systemzeit genutzt wird.

Die Nutzung eines KI-Assistenten als „Pair Programmer“ hat aber Grenzen und muss besonders im Anfängerunterricht mit Bedacht gewählt werden. Für erfahrene Softwareentwickler kann die Nutzung von KI-Assistenten zu einer Effizienzsteigerung führen: schnelles Erstellen von Prototypen oder Codestücken, die sie nur noch leicht überarbeiten müssen; Ausprobieren alternativer Lösungen; Automatisieren von Routineaufgaben. Dabei können sich die Softwareentwicklerinnen auf ihre langjährigen Erfahrungen abstützen. Beim Einstieg ins Programmieren im Unterricht ist die Gefahr groß, dass die Lernenden Codevorschläge einfach ungeprüft übernehmen oder mangels Verständnisses der Programmstruktur nicht in der Lage sind, die Codevorschläge zu debuggen und zu einem lauffähigen Programm zu bringen. Vielleicht haben die Lernenden mithilfe des KI-Assistenten sogar ein funktionierendes Programm für die konkrete Aufgabe erstellt, aber dabei nicht wirklich ein vertieftes Verständnis erworben.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: U. legt Wert darauf, Unterrichtsthemen realitätsnah und aktuell zu gestalten, um das Interesse seiner Lernenden zu wecken. Im Rahmen einer Unterrichtseinheit über die Funktionsweisen von KI-Werkzeugen am Beispiel von Inhaltsfiltern in sozialen Netzwerken haben die Schülerinnen und Schüler Methoden zur Erkennung unangemessener Inhalte kennengelernt. Sie verstehen nun, wie KI-Modelle mithilfe von Deep Learning und Natural Language Processing auf großen Datenmengen trainiert werden, um solche Inhalte zu identifizieren.

U. möchte außerdem die Praktiken von Betreibern sozialer Netzwerke bei der Inhaltsmoderation aufzeigen. Ein konkretes Beispiel ist die Beschäftigung von Arbeitnehmern in Ländern mit niedrigem Lohnniveau, die gezwungen sind, stundenlang teils äußerst verstörende Inhalte zu klassifizieren. Die Frage ist nur, wie man dieses Thema für die Schülerinnen und Schüler nachvollziehbar machen kann.

Kritisches Denken ist im Informatikunterricht unerlässlich, um vielfältige Fragestellungen zu bearbeiten. Ob ethische, ökologische, wirtschaftliche oder rechtliche Aspekte – die Antworten hängen oft von persönlichen Überzeugungen und Meinungen ab. Daher sind Methoden gefragt, die es den Lernenden ermöglichen, Probleme umfassend zu verstehen und zu diskutieren.

Bei der Nutzung von Informatiksystemen sowie bei der Analyse ihrer gesellschaftlichen Auswirkungen ist es wichtig, verschiedene Sichtweisen zu berücksichtigen. Die Einnahme verschiedener Perspektiven ist aber anspruchsvoll und muss im Unterricht gezielt geübt werden. Eine fundierte Analyse der Auswirkungen von Informatiksystemen auf unsere Gesellschaft erfordert, sich mit unterschiedlichen Meinungen und Argumenten auseinanderzusetzen. Wenn Lernende die Folgen, Möglichkeiten und Risiken nur aus ihrer eigenen Sicht betrachten, können sie keine umfassende Bewertung vornehmen. Auch bei der Entwicklung von Informatiksystemen und dem Verständnis ihrer Funktionsweise müssen meist verschiedene

Sichtweisen berücksichtigt werden, wie zum Beispiel die Benutzerfreundlichkeit einer Anwendung, die Entwicklungskosten und der Wartungsaufwand.

Bei Themen, die den Blick aus verschiedenen Perspektiven erfordern, eignen sich Rollenspiele als Unterrichtsmethode. Die Schülerinnen und Schüler schlüpfen in verschiedene Rollen und erleben diese aus der jeweiligen Sicht. Rollenspiele aktivieren Vorstellungskraft, Empathie, Kreativität und Fantasie. Sie unterstützen Lernende dabei, konfliktbeladene Situationen wirkungsvoll nachzuvollziehen und gemeinsam Lösungsansätze aus verschiedenen Blickwinkeln zu finden. Allerdings können Rollenspiele für zurückhaltende Schülerinnen und Schüler eine Herausforderung darstellen, ein Aspekt, den die Lehrkraft berücksichtigen muss.

Für das Fach Informatik betrachten wir im Folgenden verschiedene Rollenspielarten: zum einen freie Rollenspiele in Form von kontextbasierten Rollenspielen und Rollengesprächen – zum anderen strukturierte Rollenspiele, bei denen die Teilnehmer entweder als Objekte bzw. Komponenten eines Informatiksystems oder als digitale Akteure agieren.

Freie Rollenspiele

Freie Rollenspiele bieten den Teilnehmenden viel Raum für eigene Ideen und Improvisation (siehe Tab. 14.1). Die Schülerinnen und Schüler übernehmen dabei unterschiedliche Rollen ohne vorgegebene Handlungsstränge.

Beispiel 1: Kontextbasiertes Rollenspiel zur Bewertung von selbst entwickelten Apps

Als kleines Abschlussprojekt zur Softwareentwicklung sollen die Schülerinnen und Schüler in einer vorgegebenen Entwicklungsumgebung eigene Smartphone-Apps erstellen. Das Ziel ist es, ein Lernspiel zu entwickeln, das sie auch selbst nutzen können. Schnell entstehen im Brainstorming allerlei Ideen. Der geplante Kopfrechen-Trainer, die Geografie-App für stumme Karten, der Vokabeltrainer, der Notenbestimm-Helfer für das Fach Musik und der Formeltrainer Chemie können mit vertretbarem Aufwand umgesetzt und später selbst verwendet werden. Allerdings ist der Perspektivwechsel zwischen der Rolle der Produktentwicklerin und der Nutzerin für die Lernenden nicht einfach. Allein aus Entwicklersicht ist eine realistische Einschätzung der Qualität der eigenen Software kaum möglich. Faktoren wie eine klar definierte Benutzerschnittstelle, eine intuitive Bedienung oder die zielgruppengerechte Barrierefreiheit gilt es zu berücksichtigen. Dies gelingt besonders gut aus der Fremdperspektive. Deshalb schlüpfen die Lernenden nach der Modellierungs- und Implementierungsphase der App aus der Rolle der Softwareentwickler in die Rolle der Softwaretester für die Apps der Mitschülerinnen und Mitschüler. Sie testen je zwei andere Apps und hinterlassen ihre Kritikpunkte und Verbesserungsvorschläge auf einem Feedbackbogen. So erhalten die Schülerinnen und Schüler nicht nur gezielte Rückmeldungen zur eigenen App-Entwicklung, sondern erleben auch selbst die Rolle als Usabilitytester.

Tab. 14.1 Freie Rollenspiele

	Kontextbasiertes Rollenspiel	Rollengespräch
Charakteristik	Übernahme bestimmter Tätigkeiten in einem realitätsnahen Szenario	Diskussion zu einem Thema mit Akteuren unterschiedlicher Perspektiven
Ziele	Erforschen, Anwenden und Vertiefen informatischer Abläufe, Prinzipien und Zuständigkeiten	Kritische Auseinandersetzung mit verschiedenen Standpunkten und Zielen, um eigene Ansichten und Lösungsansätze zu entwickeln
Beispiele (Thema und mögliche Rollen)	Digitale Kommunikation Gesprächspartner – Man in the Middle	E-Patientenakte Krankenkasse, Hausarzt, Fachärztin, Patient, Rettungssanitäterin, Forschungsinstitut
	Datenschutz Hackerin – Nutzer/Opfer	Interessengruppen des Urheberrechts Künstler, Verlag, Nutzer, Managerin, Verwertungsgesellschaft
	Softwareentwicklung Programmierer – Produkttesterin	KI in der Medizin Juristin, Patientin, Krankenversicherung, Softwareentwicklerin, Arzt, Medical-Device-Regulation-Experte
	Webentwicklung Entwicklerin – Nutzer	Entwicklung autonomer Fahrzeuge Informatikprofessorin, Programmiererin, Kundin, Netzaktivist, Ethikdozentin, technischer Sachverständiger, Fahrzeughersteller

Beispiel 2: Rollengespräch zu digitalem Kolonialismus

Im einleitenden Beispiel könnte ein sogenanntes Rollengespräch eingesetzt werden. Dabei bieten sich folgende Rollen an:

Aktivistin: Als angesehene Informatikprofessorin aus einem Land im globalen Süden engagiert sie sich leidenschaftlich für Bildungsarbeit, digitale Souveränität, lokale Technologieentwicklung und Gerechtigkeit.

CEO des Tech-Giganten „Omega“: Sein Unternehmen aus dem globalen Norden kontrolliert wichtige Datenzentren und entwickelt Algorithmen, die den Gewinn steigern. Sein soziales Netzwerk „Socialgram“ implementiert KI-Moderationstools.

Social-Media-Influencerin: Sie ist eine aktive Nutzerin verschiedener sozialer Netzwerke und erstellt Inhalte zu den neuesten Modetrends auf diesen Plattformen. Ihre meist freizügigen Outfits und Stylingtipps zielen darauf ab, ein breites Publikum anzusprechen und zu beeinflussen.

Content-Moderatorin: Als alleinerziehende Mutter erstellt sie in einem afrikanischen Land für den internationalen Dienstleister „AI ClearView“ KI-Trainingsdaten. Sie leidet unter dem hohen Druck und den menschenunwürdigen Arbeitsbedingungen. Besonders belastend ist für sie das Taggen von pornografischen Bildern und Videos.

Arbeitsgruppenmitglied „Digital Services Act“: Als Vertreter der Europäischen Kommission hat er an der Entwicklung der gemeinsamen Regulierung für die Moderation von strafbaren Inhalten durch Plattformunternehmen mitgewirkt und setzt sich für deren strikte Einhaltung ein.

Die Schülerinnen und Schüler erhalten außerdem die Möglichkeit, eine zusätzliche, selbst entwickelte Rolle in ihr Gespräch einzubringen. Dies könnte beispielsweise der Strategiemanager des Diensteanbieters „AI ClearView“, ein Power-User der Socialgram-Plattform oder ein besorgter Elternteil sein. Jede Gruppe wählt zudem einen Moderator und zwei Beobachterinnen für ihr Gruppengespräch.

Im Verlauf der Rollengespräche kann der Begriff „Digitaler Kolonialismus“, die Dominanz und Kontrolle multinationaler Unternehmen aus dem globalen Norden über das digitale Ökosystem der Länder des globalen Südens, erörtert werden. Die neutralen Moderatoren sorgen für die Einhaltung der vereinbarten Diskussionsregeln. Für die anschließende Reflexion sammeln die Beobachter während des Gesprächs eine Vielzahl von Problemen und möglichen Lösungsansätzen.

Strukturierte Rollenspiele

Strukturierte Rollenspiele folgen einem klaren Rahmen und haben vorgegebene Rollen, Szenarien und Ziele (siehe Tab. 14.2). Die Teilnehmenden müssen sich an bestimmte Regeln oder Aufgaben halten.

Tab. 14.2 Strukturierte Rollenspiele

	Rolle eines Objekts bzw. einer Komponente eines Informatiksystems	Rolle eines digitalen Akteurs
Charakteristik	Nachbilden von Funktionen, Komponenten und Interaktionen innerhalb von Informatiksystemen	Agieren als Roboter oder Maschine innerhalb strukturierter und definierter Handlungsabläufe
Ziele	Erfassen und Analysieren von Modellen, Eigenschaften, Prozessen und Aufgaben verschiedener Komponenten sowie deren Zusammenspiel	Agieren als Maschinen oder Roboter und Erleben der Aufgaben und Prozesse aus der Sicht einer ausführenden Einheit
Beispiele (Thema und mögliche Rollen bzw. Handlungen)	Netzwerke Netzwerkkomponenten und deren Interaktionen	Programmablauf Schrittweises Verfolgen der Zustände eines Programms
	Betriebssystem Anwendungsprogramm, Benutzer, Hardwarekomponente, Treiber, Scheduler, Speicherverwaltung	Programmausführung Nachvollziehen von Befehlsfolgen in einer Miniwelt
	Objektorientierte Modellierung Verkörperung von Objekten und Eigenschaften	Textgeneratoren Agieren als Chatbot in einer Unterhaltung
	Schichtenmodell Anwendung, Transport, Vermittlung, Netzzugang, Daten(paket)	Sprache eines Getränkeautomaten Verschiedene Münzen als Eingabe akzeptierter Getränkepreise

Beispiel 3: Rolle von Komponenten zum Aufbau des Internets

Die Infrastruktur des Internets kann auf verschiedenen Abstraktionsstufen betrachtet werden. Im Alltag begegnen uns Begriffe wie „Provider“, „Router“, „WLAN“, „Access-Point“ oder „DNS-Server“. Fehlermeldungen wie „Error 404 page not found“ oder „Der DNS-Server antwortet nicht“ erscheinen vielen Heimanwendern kryptisch.

In einer rund neunzigminütigen Unterrichtseinheit können die wichtigsten Komponenten des Internets durch zwei aufeinanderfolgende Rollenspiele veranschaulicht werden. Ausgehend von der Frage „Wie kommt eine Webseite von einem Webserver auf meinen Computer?“ könnte das Ziel in einer 6. Klasse darin bestehen, grundlegende Funktionsweisen der Komponenten Computer, Server, Router, WLAN, Webserver, Provider, IP-Adresse, Domain und DNS zu vermitteln. Dafür wird im Klassenraum zunächst ein analoges Internet aufgebaut: Die Schülerinnen und Schüler übernehmen die Rollen von Haushalt, Provider und Webserver. Die einzelnen Netzwerkkomponenten können aus Pappe hergestellt und mit Schnüren verbunden werden (siehe Abb. 14.1). Anschließend wird ein Webseitenaufruf vom Computer zum Webserver und dessen Antwort über die verschiedenen Zwischenstationen (Provider) mit einem Briefumschlag nachgespielt.

Im Anschluss an das analoge Rollenspiel simulieren die Schülerinnen und Schüler selbst ein Netzwerk in einer Onlineumgebung. Im Internet finden sich wie in Abb. 14.2 gezeigt Simulationsumgebungen, mit denen kollaborativ ein Netzwerk aufgebaut werden kann. Die Schülerinnen und Schüler übernehmen erneut bestimmte Rollen (Haushalt, Provider, Websiteanbieter), um jeweils einen Teil des virtuellen Netzwerks selbst aufzubauen und sich mit den Mitschülern zu vernetzen. Die Arbeit in der Simulationsumgebung erfordert einen regen Austausch unter den Teilnehmenden, um Abonnements bei Providern zu lösen, IP-Adressen zu erfragen



Abb. 14.1 Netzwerkkomponenten aus Pappe

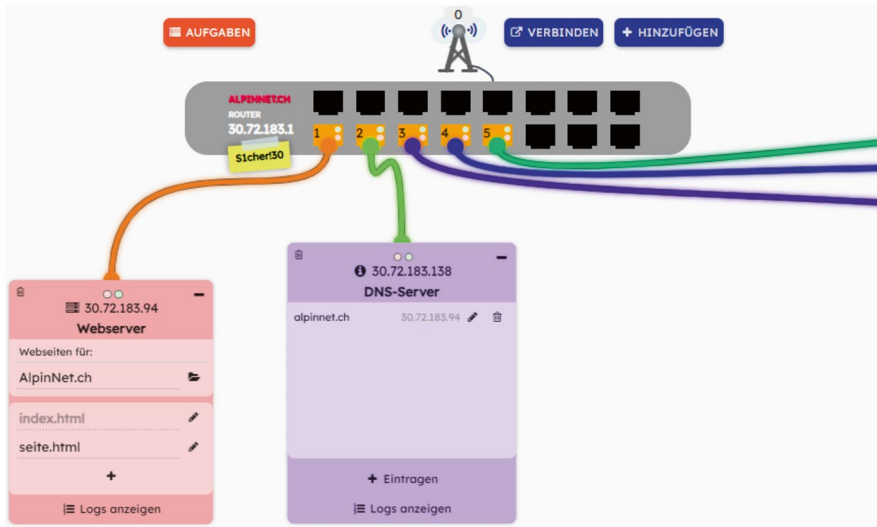


Abb. 14.2 Rolle eines Providers. (Simulationsumgebung WebNetSim, webnetsim.de)

oder DNS-Einträge zu erfassen und neue Websitedomains zu kommunizieren. Die Simulation führt die dezentrale Arbeitsweise des Internets direkt vor Augen.

Beispiel 4: Rolle des digitalen Akteurs in Miniwelt oder Struktogramm

Mit Programmierneulingen wird mittels geklebter Malerkrepp-Felder auf den Fußboden des Klassenraums eine Miniwelt erstellt. Die Schülerinnen und Schüler sollen in den Feldern gemäß einem vorgegebenen Programm einfache Bewegungs- und Drehbefehle sowie Aufhebe- und Ablegebefehle ausführen und so die Funktionsweise eines Programms erleben. Die Miniwelt kann auch als begehbare Labyrinth aus Tischen gestaltet werden. Die Schülerinnen werden dann mit verbundenen Augen von Mitschülern anhand eines vorgelesenen Programms durch das Labyrinth gesteuert. So können beispielsweise logische Fehler im Programm enaktiv aufgezeigt werden.

Auch für das Verständnis von Algorithmen ist diese Form eines Rollenspiels geeignet. Dazu wird das Struktogramm als Variante des Programmentwurfs mit Straßenmalkreide auf dem Schulhof aufgezeichnet. Die Schülerinnen und Schüler agieren als Figur in den Anweisungsfeldern des Struktogramms. Hierfür wurden in der Vorbereitung gemeinsam Rollenkarten entwickelt. Jeder Schüler arbeitet das Struktogramm als Akteur ausschließlich in seinem Anweisungsfeld ab. Mit Zetteln werden Werte zur Belegung der Variablen in das Struktogramm gegeben, jeder Akteur entscheidet, wie mit der Variable weiter verfahren wird, und gibt den berechneten Zwischenwert an den nächsten Akteur im Struktogramm weiter. Durch diese sehr kleinschrittige, interaktive Vorgehensweise fällt es den Schülerinnen und Schülern leichter, Algorithmen nachzuvollziehen, um sie später selbst zu entwickeln. Das Beispiel beruht auf Unterrichtsideen von Tobias Meyer-Hörth, Fabian

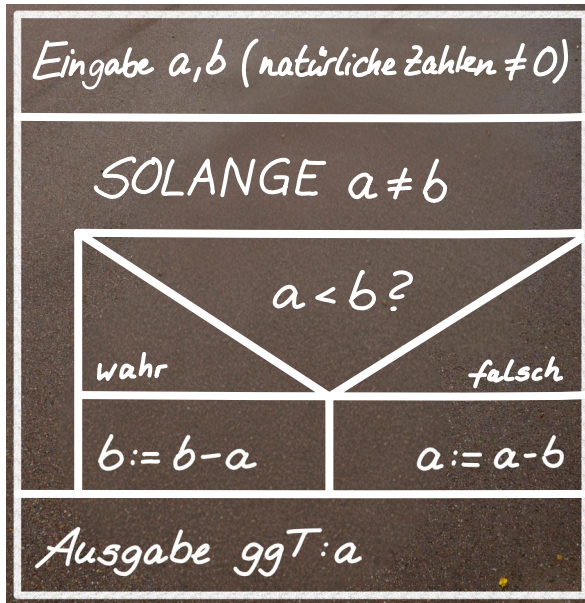


Abb. 14.3 Erlebbares Struktogramm

Graap und Johannes Struzek. Das abgebildete Struktogramm in Abb. 14.3 veranschaulicht die einfache Variante des euklidischen Algorithmus, die ausschließlich durch Subtraktion den größten gemeinsamen Teiler (ggT) zweier natürlicher Zahlen ermittelt.

Rollenspiele können dazu beitragen, komplexe und vielschichtige oder abstrakte Themen zugänglicher zu machen. Durch das Einnehmen verschiedener Perspektiven wird das Verständnis für verschiedene Aspekte der Informatik und deren Anwendung in der realen Welt gefördert.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Ohne Betriebssysteme läuft kein Computer. Für angehende IT-Supporter sind vertiefte Kenntnisse dieses Bereichs unerlässlich. An der IT-Berufsschule ist es Lehrer W. ein Anliegen, dass seine Schülerinnen und Schüler sowohl die grundlegenden Konzepte von Betriebssystemen als auch die praktische Anwendung kennenlernen. Aber die Vorkenntnisse seiner Lernenden sind gerade in diesem Bereich sehr unterschiedlich. Einige sind richtige Linux-Cracks, die mühelos neue Betriebssystemversionen installieren. Andere tun sich schon mit der grundlegenden Bedienung eines Betriebssystems schwer.

Im Informatikunterricht ist das Spektrum der Vorkenntnisse der Lernenden oft sehr breit, und es ist schwierig, Unterricht zu gestalten, der dem unterschiedlichen Wissensstand der Lernenden Rechnung trägt.

Die Beherrschung vorangegangener Lerninhalte ist der wichtigste Faktor für den Erfolg beim Erschließen neuer Wissensgebiete. Trotzdem ist die Verlockung groß, mit dem nächsten Thema fortzufahren, sobald ein Teil der Schülerinnen und Schüler den Lerninhalt verstanden hat: Der Lehrplan setzt die Lehrkräfte unter Druck. Beginnen Lehrkräfte mit dem neuen Inhalt, bevor der alte verstanden wird, entsteht ein Teufelskreis: Je mehr Wissenslücken sich auftun, desto schwerer fällt es, Neues zu begreifen. Guter Unterricht orientiert sich deshalb nicht an den schnellsten 30 % einer Klasse, sondern stellt das Mastery-Learning-Prinzip in den Mittelpunkt: Fortgefahren wird erst, wenn 80–90 % der Schülerinnen und Schüler den Lerninhalt verstanden haben.

Im Informatikunterricht variieren Lerngeschwindigkeit und -fortschritt der Schülerinnen und Schüler stärker als in anderen Fächern: Während das Konzeptwissen oft auf einem ähnlichen Niveau liegt, können die Fertigkeiten im Umgang mit dem Computer und das Produktwissen zu einzelnen Programmen oft erheblich voneinander abweichen. Leitprogramme schaffen hier Abhilfe, indem sie den Unterricht

Abb. 15.1 Mastery-Learning-Prinzip bei Leitprogrammen

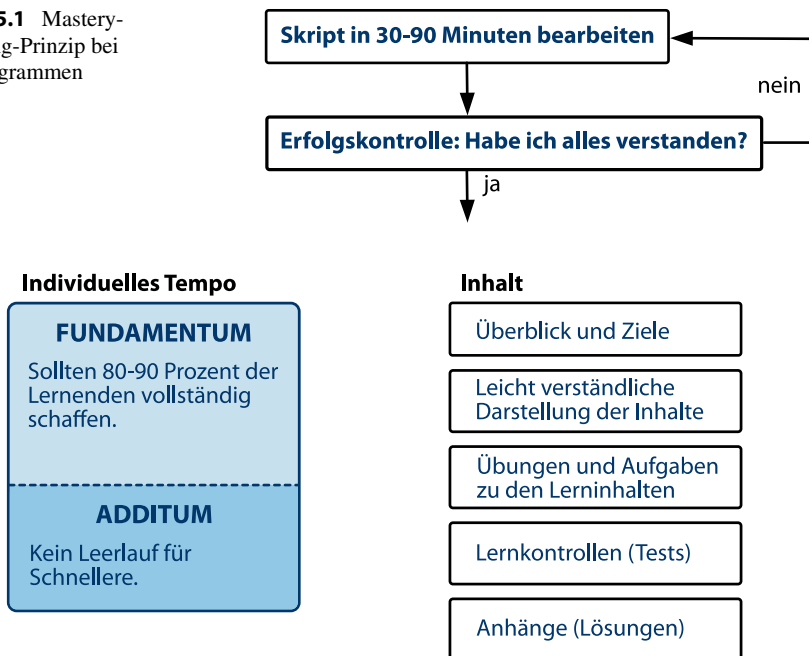


Abb. 15.2 Aufbau der einzelnen Kapitel eines Leitprogramms

individualisieren. Die Unterrichtsmethode geht zurück auf das Konzept des Keller-Plans (1968). Dabei werden die Unterrichtsinhalte in einzelne Pakete aufgeteilt und mit detaillierten Anleitungen für das selbstständige Lernen versehen. Das Besondere an diesem Ansatz: Nach jedem abgeschlossenen Paket müssen die Schülerinnen und Schüler ihr Verständnis in einem Test unter Beweis stellen. Erst nach erfolgreichem Bestehen dürfen sie zur nächsten Einheit übergehen. Dieses Vorgehen stellt sicher, dass die Lernenden die Lernziele wirklich erreicht haben, bevor sie fortfahren (siehe Abb. 15.1).

Ein Leitprogramm muss wie in Abb. 15.2 dargestellt alle wesentlichen Informationen für die Lernenden beinhalten. Es trägt den unterschiedlichen Vorkenntnissen und Lerntempi der Schüler Rechnung, indem es ein Fundamentum und ein Additum enthält. Das Fundamentum enthält die Teile, die von allen erfolgreich bearbeitet werden müssen. Das Additum ist für die Schnellen gedacht und enthält weiterführende Lernangebote. Diese müssen attraktiv sein, damit sich die schnellen oder besonders leistungsstarken Lernenden damit auseinandersetzen. Für das Additum eignen sich beispielsweise Informationen zu aktuellen Produkten, unterschiedliche Umsetzungen von technischen Lösungen, weitere Lösungsvarianten, Zusatzbetrachtungen zur Laufzeit von Algorithmen oder kleine Experimente sehr gut.

Ein gutes Leitprogramm zeichnet sich durch eine klare Struktur und motivierende Elemente aus. Jedes Kapitel beginnt mit einer prägnanten Übersicht und klar definierten Lernzielen, gefolgt von einer verständlichen Darstellung der Lern-

inhalte. Statt langer, monotoner Texte setzt ein gutes Leitprogramm auf Abwechslung und regt die Lernenden zu aktivem Denken und Handeln an. Die Gestaltung eines Leitprogramms erfordert mehr Sorgfalt als die Planung einer herkömmlichen Unterrichtsstunde. Die Kapitel des Leitprogramms müssen so aufgebaut sein, dass die Lernenden motiviert sind, selbstständig die Inhalte in ihrem eigenen Tempo zu erarbeiten. In der allgemeinen Didaktik gibt es verschiedene Modelle zur Gestaltung von Lernprozessen. Gerade für angehende Lehrkräfte lohnt sich die Orientierung an einem klar strukturierten Modell. Als geeignetes Instrumentarium bietet sich hier zum Beispiel das LUKAS-Modell (Luthiger et al., 2014) an. Das Modell ist stark auf die Förderung von Kompetenzen ausgerichtet und bietet konkrete Anleitungen zum Entwickeln von Aufgaben.

In der Informatik lassen sich in einem Leitprogramm vielfältige praktische Übungen am Computer integrieren. So können Lernende zum Beispiel in einem Netzwerkkurs verschiedene Aspekte der Vernetzung im Schulnetz erkunden oder in einer Anwendungsschulung die Grundlagen der Tabellenkalkulation praktisch vertiefen. Um selbstständiges Arbeiten zu ermöglichen, müssen zu allen Aufträgen ausführliche Lösungen abgegeben werden.

Jedes Kapitel schließt mit einer Selbstlernkontrolle ab, gefolgt von einem Kapiteltest, der entweder schriftlich oder im mündlichen Gespräch mit der Lehrkraft durchgeführt wird. Dieser Test bietet die Gelegenheit, offene Fragen im direkten Gespräch zu klären, und hebt das Leitprogramm von anderen Formen des Selbststudiums ab.

Leitprogramme eignen sich besonders für eine heterogene Gruppe von Lernenden und für anspruchsvolle Themen. In beiden Fällen trägt die Individualisierung den unterschiedlichen Lerntempi Rechnung: Langsamere Lernende können sich Zeit nehmen und zurückblättern; schnelle Lernende können Zusatzerklärungen überspringen und haben die Möglichkeit, im Additum zusätzliches Material kennenzulernen. Bei schwierigen Themen bietet die schriftliche Form zudem den Vorteil einer präzisen Darstellung. Nicht zuletzt stärkt die erfolgreiche selbstständige Erarbeitung schwieriger Inhalte das Selbstvertrauen der Lernenden.

Leitprogramme haben auch Nachteile: Aus Sicht der Lehrkraft stellt der erhebliche Zeitaufwand für das Erstellen die größte Hürde dar. Im mündlichen Unterrichtsgespräch geschieht vieles ad hoc während des Unterrichts. Beim Leitprogramm muss jedes Detail geplant sein; die Lehrkraft muss im Voraus abwägen, welche Aspekte besonders anspruchsvoll sind und detaillierter dargestellt werden müssen. Leitprogramme sollten deshalb in erster Linie für wiederkehrende Unterrichtsthemen und von verschiedenen Lehrerinnen und Lehrern gemeinsam erstellt werden. Aus Sicht der Lernenden bedeutet die Arbeit mit Leitprogrammen eine Verschiebung der Verantwortung. Sie müssen aktiv die Steuerung ihres Lernprozesses übernehmen und können sich nicht länger in die Anonymität des Klassenverbands zurückziehen. Doch gerade die Nachteile machen Leitprogramme zu einer der effektivsten Unterrichtsmethoden: Die sorgfältig aufbereiteten Inhalte in Kombination mit der notwendigen intensiven Auseinandersetzung der Schüler fördern ein tiefgreifendes Verständnis.

Die Vorkenntnisse hinsichtlich Produktwissen und die Fertigkeiten im Umgang mit Produkten sind im Informatikunterricht stark heterogen. Jede Form von lehrerzentriertem Unterricht gibt ein gemeinsames Lerntempo vor und führt dazu, dass viele Lernende über- oder unterfordert sind. Abhilfe schaffen individualisierende, auf die Lernenden zentrierte Unterrichtsformen. Leitprogramme als individualisierende, schülerzentrierte Unterrichtsform bieten hier einen vielversprechenden Lösungsansatz.

Im Internet findet sich inzwischen eine beachtliche Auswahl an Leitprogrammen zu unterschiedlichen Themengebieten der Informatik: von formalen Sprachen und Automatentheorie über Sortierverfahren, Bäume und Backtracking bis hin zu rekursiver Programmierung oder Huffman-Codierung. Aufgrund des umfassenden Charakters von Leitprogrammen, die oft ganze Unterrichtseinheiten abdecken, verzichten wir an dieser Stelle bewusst auf Beispiele.

Literatur

- Keller, F. S. (1968). Good-bye, teacher. *Journal of Applied Behaviour of Analysis*, 1, 79–89.
- Luthiger, H., Wilhelm, M., & Wespi, C. (2014). Entwicklung von kompetenzorientierten Aufgabensets – Prozessmodell und Kategoriensystem. *Journal für LehrerInnenbildung*, 3, 56–66.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Lehrerin S. führt ihre 6. Klasse in die Grundlagen des Programmierens ein. Sie startet mit einer blockbasierten Programmiersprache, erklärt die Benutzeroberfläche und führt Sequenz, Schleifen und Bedingungen anhand einfacher Beispiele ein. Jede Schülerin erhält kurze Übungsaufgaben. Nach dieser strukturierten Einführung startet sie ein kleines Projekt: ein Jump-and-Run-Spiel. Doch schnell wird klar, dass die Schüler bereits bei den einfachsten Befehlen Schwierigkeiten haben.

Offene Problemstellungen werden im Informatikunterricht teilweise vernachlässigt und Aspekte wie selbstständiges Arbeiten, Kreativität und kritische Reflexion kommen zu kurz. Dabei sind gerade diese Fähigkeiten im späteren Berufsalltag von großer Bedeutung, insbesondere wenn es darum geht, sich eigenständig in neue Themen einzuarbeiten.

Das abstrakte Wesen der Informatik stellt für den Unterricht eine besondere Herausforderung dar: Schülerinnen und Schüler müssen nicht nur die Fähigkeit zur Abstraktion entwickeln, sondern auch lernen, gleichzeitig auf verschiedenen Abstraktionsebenen zu denken. Zudem können Schülerinnen und Schüler viele Konzepte wie Algorithmen für effizientes Sortieren (mit Quicksort oder Heapsort), Verschlüsselungsverfahren (z. B. RSA) oder die formalen Definitionen regulärer und kontextfreier Sprachen nicht selbst herausfinden. Daher dominiert oft die Theorievermittlung mit anschließenden Übungen.

Dies steht im Gegensatz zum Berufsalltag von Informatikern: Das selbstständige Erschließen neuer Inhalte gehört zur täglichen Arbeit. Diese wichtige Methodenkompetenz sollte daher bereits im Unterricht gefördert werden. Offene Unterrichtssituationen geben den Schülerinnen und Schülern die Gelegenheit, selbst neue Inhalte zu erarbeiten. Eine dafür geeignete Unterrichtsmethode ist das entdeckende Lernen: Entdeckendes Lernen fördert Kreativität, Selbstständigkeit, kritisches Den-

ken und den gegenseitigen Austausch. Es ermöglicht ein hohes Maß an Individualisierung, was es der Lehrkraft erlaubt, besser auf die Bedürfnisse der einzelnen Schüler einzugehen.

Die Grundidee beim entdeckenden Lernen ist folgende: Die Lernenden erlangen neues Wissen, indem sie persönliche Erfahrungen machen und Dinge hinterfragen. Sie entwickeln durch Staunen, Wundern und Zweifeln ihre eigenen (vielleicht naiven) Theorien und müssen dabei mitunter alte Vorstellungen aufgeben und zuvor aufgestellte Hypothesen verwerfen. Für den Lernerfolg ist es oft wirksamer, wenn die Schülerinnen aktiv Erklärungen generieren, um beobachtete Phänomene zu begründen, als wenn sie Merksätze und Theorien vorgetragen bekommen und auswendig lernen müssen.

Entdeckendes Lernen braucht Zeit und Freiräume. Es gibt a priori kein Richtig oder Falsch auf der Suche nach Neuem. Wichtig ist, dass alle Erkenntnisse festgehalten und geordnet werden. Dokumentation und Präsentation von Ergebnissen helfen bei der Ordnung der eigenen Gedanken und sind ein wichtiges Mittel zur Entwicklung und kritischen Reflexion der eigenen Ideen. Nicht zielführende Herangehensweisen oder entstandene Fehler müssen in der Reflexion aufgegriffen und analysiert werden, um daraus zu lernen.

Der Erfolg von entdeckendem Lernen hängt vom gewählten Thema und der Aufgabenstellung ab. Die Selektion passender Inhalte obliegt der Lehrkraft, ebenso die Anleitung zum entdeckenden Lernen. Dabei sind drei Anforderungen zentral:

Offenheit des Themas Das Gebiet muss eine gewisse Offenheit besitzen. Entdeckendes Lernen bedeutet, ein Thema explorativ zu erkunden, Hypothesen aufzustellen, zu überprüfen und sich mit anderen auszutauschen. Das Lösen vorgegebener Aufgaben ist kein entdeckendes Lernen. Der Lerngegenstand muss mehrschichtig oder vielfältig sein und verschiedene Entdeckungswege zulassen. Zum Beispiel lassen sich der Fließkommazahlen-Standard IEEE 754 oder der Quicksortalgorithmus nicht entdecken, wohl aber die Darstellung von Fließkommazahlen im Binärsystem oder einfache Sortierverfahren. Offenheit kann auf zwei Arten erfolgen: Entweder sind verschiedene Lösungen eines Problems möglich, oder die Lernenden wählen selbstständig den zu explorierenden Aspekt eines Themas.

Vollständiges Material Das notwendige Material wird von der Lehrkraft zur Verfügung gestellt und so aufbereitet, dass alle Lernenden die Informationen ohne zusätzliche Betreuung verstehen und verarbeiten können. Ziel ist die Entwicklung eigener Ideen auf Grundlage des vorgegebenen Materials und eigenen Vorwissens und nicht das bloße Herausfiltern und Strukturieren von Informationen. Selbstständige Informationsbeschaffung kann durchaus ein Ziel sein, erfordert jedoch oft mehr Zeit, als in Präsenzveranstaltungen verfügbar ist, und muss in einem größeren Zeitrahmen stattfinden.

Aufgabe und Bewertung Aufgabenstellungen zum entdeckenden Lernen sollten verschiedene Lösungen, Herangehensweisen und Perspektiven zulassen. Sie müssen genau wie das Thema eine gewisse Offenheit besitzen, damit die Lernenden die

Freiheit haben, eigene Entdeckungen zu machen. Bei der Beurteilung von Beiträgen ist es wichtig, alle Vorschläge und Ideen ernst zu nehmen. Die Lehrkraft sollte bei halb richtigen Lösungen nicht sofort intervenieren: Es braucht ein gesundes Maß an „Laissez-faire“.

Wie könnte ein Einstieg ins Programmieren in der Primarschule mittels entdeckenden Lernens erfolgen? Die Lehrerin könnte den Schülerinnen und Schülern den Programmcode eines rudimentären Jump-and-Run-Spiels zur Verfügung stellen. Die Aufgabe: das Spiel nach eigenen Ideen weiterentwickeln. Dabei erkunden die Schülerinnen und Schüler zuerst die Benutzeroberfläche und versuchen, die Logik hinter der Spielmechanik zu verstehen. Durch schrittweise Änderungen am Code lernen sie die Bedeutung der Befehle und Kontrollstrukturen. Tooltips erleichtern das Verstehen der Programmierkonzepte. Die offene Formulierung des Auftrags motiviert sie, kreative und vielfältige Weiterentwicklungen des Spiels zu gestalten.

Entdeckendes Lernen ermutigt Schülerinnen und Schüler, neue Inhalte durch eigene Erfahrungen und kreatives Ausprobieren zu erschließen. Statt einer rein theoretischen Einführung erhalten die Lernenden die Gelegenheit, Konzepte eigenständig zu entdecken und anzuwenden. Dabei werden nicht nur Problemlösungskompetenzen gestärkt, sondern auch Fähigkeiten wie individuelles Vorgehen, selbstständiges Arbeiten und kritisches Reflektieren gefördert.

Beispiel 1: Gestaltung sehbehindertengerechter Websites

Wie „sieht“ ein Blinder eine Website? Worauf muss man achten, wenn man barrierefreie Websites erstellt? Können Blinde und Sehbehinderte die vielfältigen Informationen im Internet überhaupt nutzen oder werden sie benachteiligt? Welche technischen Hilfsmittel stehen Blinden heute zur Verfügung? Wie sieht die Rolle von Blinden in der Informationsgesellschaft aus? Diese Fragen treten auf, wenn man selbst in die Rolle eines Blinden schlüpft. Verschiedene Umsetzungen sind möglich: Lernende können Websites mit verbundenen Augen mit einem gängigen Screenreader untersuchen. Oder zwei Lernende überprüfen die Barrierefreiheit, indem sie einen Screenreader simulieren. A bedient die Website, ohne sie zu sehen, und orientiert sich ausschließlich an den Anweisungen und Rückmeldungen des „Screenreaders“. B agiert als „Screenreader“, liest den Webseiteninhalt vor und gibt A akustisches Feedback zu Aktionen, wie das Vorlesen von Überschriften, Links und Schaltflächen.

Das Thema bietet die notwendige Offenheit, da eine breite Palette von Aspekten entdeckt werden kann. So kann beispielsweise Praktisches wie „Eine Homepage ohne Text lässt sich nicht vorlesen“ betrachtet werden. Vielleicht stellen die Schülerinnen die Designmaxime auf, dass eine Website nur dann eine gute Website ist, wenn auch Blinde und Sehbehinderte Zugang zu ihr finden. Oder sie schlagen die Brücke zu den Crawlern von Suchmaschinen und KI-Assistenten, die Websites

meist ebenfalls nur wie Blinde „sehen“. Daraus lassen sich wichtige Erkenntnisse für die Optimierung von Websites für die Verarbeitung durch Maschinen ableiten. Das Thema kann auch im größeren gesellschaftlichen Rahmen betrachtet werden: Websites von staatlichen Institutionen sind per Gesetz zur Barrierefreiheit verpflichtet. Werden diese Richtlinien eingehalten? Sind die Richtlinien überhaupt umsetzbar?

Beispiel 2: Erkunden von Farbmodellen

Die Merkmale, Unterschiede und Gemeinsamkeiten der RGB- und CMYK-Farbmodelle werden im Webdesign-Unterricht häufig durch Lehrmittel oder im klassischen Lehrervortrag vermittelt. Ein schülerzentrierter und handlungsorientierter Ansatz, der entdeckendes Lernen fördert, könnte effektiver sein. Schülerinnen und Schüler analysieren zunächst existierende Webseiten. Dabei untersuchen sie deren Quellcode, identifizieren Farbcodes und stellen Hypothesen auf, wie diese Farben technisch dargestellt werden. Daraufhin experimentieren sie selbstständig mit einem Onlinefarbcod-generator, um die Zusammenhänge zwischen den Farbwerten und ihrer Darstellung im RGB-Format zu erkunden.

Mithilfe des Tutorials eines Grafikbearbeitungsprogramms, welches beide Farbmodelle unterstützt, analysieren die Lernenden wichtige Merkmale und Unterschiede beider Farbmodelle und recherchieren typische Anwendungsgebiete. Zum Abschluss wenden die Schülerinnen und Schüler ihr neu erworbenes Wissen an: Sie entwerfen Farbkonzepte für eine fiktive Webseite, beispielsweise für Sonnencreme-Produkte, und begründen ihre Farbauswahl. Ihre Entscheidungen reflektieren sie anschließend kritisch, indem sie bestehende Webseiten mit ähnlichem Thema analysieren und mit ihrer Wahl vergleichen.

Beispiel 3: KI-Werkzeuge und Fake News

Entdeckendes Lernen im Informatikunterricht eignet sich auch gut für die Thematisierung gesellschaftlicher Wechselwirkungen der Informatik. So kann der Einfluss betrachtet werden, der durch Fake News auf Trainingsdaten von KI-Werkzeugen besteht. Da der direkte Zugriff auf diese Daten im Unterricht nicht möglich ist, bietet sich der Einsatz eines didaktischen Modells wie der Onlinelernumgebung SoekiaGPT an (Hielscher, 2023). Im statistischen Sprachmodell SoekiaGPT können Schülerinnen und Schüler selbst Dokumente in die Trainingsdaten hochladen und deren Einfluss auf den generierten Output beobachten.

Die Unterrichtseinheit beginnt damit, dass die Lernenden in Gruppen von etwa sechs Personen eine Dokumentensammlung zu einem aktuellen Thema erstellen, zu dem auch viele Fake News und Verschwörungstheorien kursieren. Geeignete Themen sind beispielsweise Klimawandel, Migration oder Biodiversität in der Landwirtschaft. Die Gruppen sammeln Medienberichte zum gewählten Thema in einer Ausgangssammlung. Im zweiten Schritt wird das Sprachmodell mit dieser Dokumentensammlung trainiert. Anschließend formulieren die Lernenden einfache Fragen als Prompts und testen, welche Antworten das trainierte Modell generiert. Im dritten Schritt fügen sie gezielt irreführende Fake News in die Trainingsammlung ein und analysieren, wie sich die Antworten des Modells verändern. Ziel

dieses Schrittes ist es, mit möglichst wenigen, aber gezielt formulierten Fake News die Inhalte der Antworten zu manipulieren. Dadurch werden der Einfluss der Trainingsdaten auf KI-Werkzeuge veranschaulicht und die Risiken von Fake News und Verschwörungstheorien für KI-basierte Anwendungen deutlich gemacht. Diese Unterrichtseinheit regt Diskussionen über die Notwendigkeit verlässlicher Quellen, die Auswirkungen digitaler Medien auf das Demokratieverständnis und andere gesellschaftliche Aspekte an.

Literatur

Hielscher, M. (2023). *SoekiaGPT – ein didaktisches Sprachmodell. Informatische Bildung in Schulen (IBiS):* Vol. 1, No. 1.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Am dritten Kurstag zum Thema Datenbanken klagt Kursleiter K. in der Pause: „In jedem Kurs das Gleiche: Die Teilnehmer wollen einfach nicht zuhören, wenn es darum geht, zuerst das Datenbankschema sorgfältig zu entwerfen. Normalisierung und Entwurfsmethoden sind nicht gefragt. Anstatt die Daten in Beziehungen zu strukturieren, wird munter alles in einer einzigen Tabelle zusammengeworfen.“

„Bei mir ist es nicht anders“, fügt Kollegin M. hinzu. „Da wir im Unterricht nur kleine Programme schreiben, werden diese nicht dokumentiert. Jeder programmiert einfach drauf los. Die von mir vermittelten Prinzipien des Softwareengineerings werden kaum beachtet.“

Informatiksysteme sind oft groß, komplex und das Werk von Entwicklungsteams. Im Unterricht fehlt in der Regel die Zeit, umfangreiche Informatiksysteme zu planen und zu entwickeln. Die Beschränkung auf kleinere Systeme blendet dabei wichtige Aspekte der Informatik aus.

Die Informatik stellt Strategien und Methoden bereit, um komplexe Probleme zu lösen und große Systeme zu bauen. Da im Unterricht aber meist die Zeit fehlt, werden Softwareengineering, Projektmanagement oder Modellierung nur in der Theorie behandelt. In den Unterrichtsbeispielen wendet man diese Strategien lediglich auf einfache Situationen an. Dadurch wird den Lernenden kein Bewusstsein für den Umgang mit komplexen Systemen vermittelt, was zu einem falschen Eindruck führt.

Wie lässt sich im Informatikunterricht trotz beschränkter Zeit ein Einblick in die bei realen Informatikprojekten auftauchenden Probleme – von der Definition von Schnittstellen bis hin zu zwischenmenschlichen Spannungen in einem Projektteam – mit vertretbarem Aufwand vermitteln? Die Projektmethode ist dafür prädestiniert: Wie in realen Informatikprojekten planen die Beteiligten, wie sie ein Problem angehen wollen. Sie diskutieren verschiedene Vorschläge, legen das Vorgehen

sowie den Zeitplan fest und setzen ihn anschließend im Team um. Dabei lernen die Beteiligten, erreichbare Ziele zu setzen, Meilensteine zu definieren, die anfallenden Arbeiten aufzuteilen, Teillösungen zu evaluieren und diese zu einem Gesamtergebnis zusammenzuführen. Die Projektmethode bildet somit viele Aspekte eines Informatikprojekts ab. Das Setzen von Meilensteinen und das laufende Evaluieren des Projektfortschrittes entspricht zum Beispiel dem agilen Vorgehen und der Scrum-Methode in professionellen Softwareprojekten.

Es gibt jedoch auch Unterschiede zwischen einem Informatikprojekt und der Projektmethode als Unterrichtsmethode: In Informatikprojekten sind die Projektleiter im Voraus festgelegt, die Projektziele werden in der Regel durch die Kunden definiert. Bei der Projektmethode im Unterricht übernehmen die Beteiligten selbst die Rolle des Projektleiters und legen Ziele des Projekts gemeinsam fest.

Zur Projektmethode im Unterricht gibt es umfangreiche Literatur, weshalb wir auf eine detaillierte Darstellung der Methode verzichten. Bewährt hat sich für den Informatikunterricht aufgrund seiner Praxisnähe und Strukturiertheit das siebenstufige Modell von Frey (2012), das wir hier kurz zusammenfassen:

Projektinitiative Hier werden Probleme, Aufgaben oder Ideen aufgeworfen. Die Fragestellungen können von der Lehrkraft, von den Lernenden oder aus anderen Quellen stammen. Die Projektinitiative muss offen sein und darf nicht darauf abzielen, ein klar definiertes Problem zu lösen.

Eingrenzung In dieser Phase werden die aufgeworfenen Fragen und Ideen gesichtet: Erste Projektideen sind oft zu vage und müssen präzisiert werden. Aufgrund beschränkter Ressourcen (Zeit, Material, Vorkenntnisse) kann die ursprüngliche Idee meist nicht in vollem Umfang verfolgt werden; daher müssen die Projektziele eingegrenzt und Schwerpunkte gesetzt werden.

Projektplan Nach Festlegung der Projektziele wird ein Arbeitsplan erstellt. Die Zielsetzungen und das geplante Vorgehen werden dokumentiert. Die auszuführenden Tätigkeiten werden einzelnen Gruppen zugeteilt und es wird festgelegt, wie die Informationen zwischen den Gruppen fließen. Es ist wichtig, den Lernenden mitzuteilen, dass nicht alle Arbeiten gemeinsam ausgeführt werden sollten; um effizient zu sein, sollte möglichst parallel gearbeitet werden.

Ausführung In dieser Phase werden die geplanten Aktivitäten umgesetzt, die Ergebnisse laufend evaluiert und mit dem Projektplan verglichen. Je nach Verlauf der Projektarbeiten muss in dieser Phase gegebenenfalls auch der Projektplan revidiert werden.

Abschluss Im Informatikunterricht endet ein Projekt oft mit der Fertigstellung eines kleinen Informatiksystems. Häufig handelt es sich beim Endprodukt nicht um ein produktives System, sondern um einen Prototyp. In der Regel schließt ein Projekt mit einer Präsentation der gemachten Erfahrungen und erzielten Ergebnisse ab.

Fixpunkte Gerade in länger dauernden Projekten oder solchen mit vielen Beteiligten ist es wichtig, von Beginn an Fixpunkte festzulegen. Diese Fixpunkte stellen sicher, dass der Austausch zum Projektfortschritt zwischen den Beteiligten regelmäßig stattfindet.

Metainteraktion Wichtig ist, dass sich die Beteiligten laufend über den Projektverlauf austauschen: Sind wir auf dem richtigen Weg? Gibt es Schwachstellen? Funktioniert die zwischenmenschliche Kommunikation in der Projektgruppe? Diese Metainteraktion – das Nachdenken über das eigene Denken und Handeln – ist zentral für die Projektmethode.

Möglichkeiten und Zeit, komplexe Informatiksysteme realitätsnah zu entwickeln, sind im Unterricht stark begrenzt. Mit der Projektmethode können die Lernenden dennoch zentrale Phasen und Aspekte typischer Prozesse der Systementwicklung nachvollziehen.

Direkt umsetzbare Beispiele für Unterrichtsprojekte lassen sich nicht in einem Buch zusammenstellen. Wirkliche Projekte müssen offen, aktuell und für die Beteiligten von unmittelbarem Interesse sein. Im Idealfall wecken Projekte im Unterricht die Neugierde der Beteiligten und entwickeln eine Eigendynamik. Die nachfolgenden Beispiele durchgeführter Unterrichtsprojekte vermitteln einen Eindruck von den vielfältigen Möglichkeiten.

Beispiel 1: Entwicklung eines virtuellen Klassenmaskottchens

In der Primarstufe verbindet die Entwicklung eines Klassenmaskottchens handlungsorientiertes Lernen mit kreativen Prozessen. Das Projekt spricht die Lebenswelt der Schülerinnen und Schüler direkt an und fördert gleichzeitig wichtige informatische Kompetenzen wie Grundlagen der Programmierung sowie den Umgang mit Grafik- und Animationssoftware.

In der Planungsphase sammeln die Schülerinnen und Schüler Ideen zur Persönlichkeit, Gestaltung und Funktion ihres Maskottchens. Sie erstellen erste Skizzen und diskutieren dabei auch ethische und kulturelle Aspekte: Wie kann sichergestellt werden, dass das Maskottchen alle Mitglieder der Klasse repräsentiert und niemanden ausschließt? Kann das Maskottchen mit Menschen interagieren, zum Beispiel sich mittels eines Bots in einem Chat unterhalten? Welche Aspekte des Datenschutzes müssen dabei berücksichtigt werden?

Nachdem die Arbeitsschritte festgelegt sind, bilden sich kleine Teams, die sich mit verschiedenen Aspekten des Maskottchens befassen. Eine Gruppe könnte sich um die Gestaltung und das visuelle Erscheinungsbild kümmern: Soll das Maskottchen direkt in einer Grafiksoftware entworfen oder zunächst als physisches Modell gebastelt und anschließend digitalisiert werden? Eine andere Gruppe könnte eine Hintergrundgeschichte entwickeln, Charaktereigenschaften des Maskottchens festlegen und einen Chatbot mit Informationen zum Maskottchen konfigurieren. Mit

einfachen Virtual-Reality-Anwendungen könnte das Maskottchen sogar animiert oder in virtuelle Welten integriert werden. Am Ende werden die Ergebnisse des Projektes den anderen Schulklassen und Lehrkräften oder auch den Eltern präsentiert. Dabei wird auch über den Projektverlauf, besondere Herausforderungen und die gemachten Lernerfahrungen berichtet.

Beispiel 2: Projekt tropfender Wasserhahn

Dieses Unterrichtsprojekt fand an einem Gymnasium statt und wurde von Kurt Doppler und Werner Hartmann durchgeführt. Den Anstoß hierzu gab der Krankenhausaufenthalt eines Schülers, der das Tropfverhalten einer Infusion beobachtete und interessante Muster feststellte. Zurück in der Schule stellte er das Thema im Unterricht vor. Der Schüler sowie die beiden Lehrer entschieden sich, der Sache auf den Grund zu gehen. Um weitere Schülerinnen und Schüler zur Mitarbeit am Projekt zu bewegen, wurde für das Projekt geworben:

„Das komplexe Verhalten eines tropfenden Wasserhahns ist angesichts der Einfachheit der beteiligten Kräfte recht überraschend: Normalerweise tropft ein Wasserhahn in einem regelmäßigen Rhythmus. Wird der Wasserhahn aber nur ein bisschen mehr geöffnet, fallen die einzelnen Tropfen in unregelmäßigen Zeitabständen. Die scheinbare Ordnung weicht einem turbulenten Chaos.

Die Fähigkeit, das Strömungsverhalten von Flüssigkeiten vorauszusagen, ist in vielen Anwendungsbereichen von großer Bedeutung. Der tropfende Wasserhahn zeigt die Schwierigkeiten auf beim Versuch, Ordnung in die Unordnung zu bringen.

Gelingt es, ein Experiment zum tropfenden Wasserhahn aufzubauen? Kann das Tropfverhalten genügend genau gemessen werden? Können die gemessenen Daten mit einem Computer erfasst werden? Welche Einsichten liefert die Auswertung der Daten? Kann das Phänomen des tropfenden Wasserhahns mit einem mathematischen Modell beschrieben werden?“

Ausgehend von der Projektinitiative des Schülers entstand ein Projektplan, und die Arbeiten wurden auf verschiedene Gruppen aufgeteilt: Eine Gruppe fertigte eine mechanische Tropfanlage, bei der durch Heben und Senken des Wasserstandes genaue Tropfenzählungen möglich waren. Eine zweite Gruppe widmete sich dem Bau eines elektronischen Tropfenzählers zur Messung der Tropfenfolge. Eine dritte Gruppe schrieb ein Programm zum Erfassen der gemessenen Daten. Die Aufgabe einer weiteren Gruppe war es, die gemessenen und erfassten Daten auszuwerten und zu visualisieren. Diese Gruppenbildung ist typisch für die Projektmethode; Teamwork ist in vielen Projekten zentral.

Im Laufe des Projekts traten nicht vorhergesehene Probleme auf: So führten kleinste Erschütterungen – zum Beispiel das Vorbeifahren eines Lastwagens auf der benachbarten Straße – zu empfindlichen Störungen der Messresultate. Die Experimentierumgebung musste in einen anderen Raum verlegt und die Experimente zu ruhigen Nachtzeiten durchgeführt werden. Anfänglich wurden nur Einer-, Zweier- und Viererzyklen von Tropfrhythmen festgestellt. Aufgrund der Chaostheorie (Feigenbaum) wurden auch Dreierzyklen erwartet, die sich aber nicht finden ließen. Eine erneute Revision des Projektplanes führte zu ununterbrochenen Messungen während einer ganzen Nacht, in der Hoffnung, so einen Dreierzyklus zu ent-

decken. Diese lange Messreihe bedingte eine Änderung der Tropfanlage, der Erfassung der gemessenen Daten und eine Mustererkennungssoftware, die gezielt nach Dreierzyklen suchte. So gelang es schließlich, reproduzierbar Dreierzyklen und daraus abgeleitete Sechserzyklen zu messen.

Den Abschluss des Projekts bildeten eine Pressekonferenz und ein Bericht in der lokalen Zeitung. Im Verlaufe des Projekts kam es zu viel Metainteraktion innerhalb der Projektgruppe. Bei zunächst nicht lösbar scheinenden Problemen wurden Brainstormings durchgeführt und „verrückte“ Ideen ausgeheckt: Als die Zeitmessung aufgrund der langsamen Software zu scheitern drohte, wurde beschlossen, das Problem im Kreis der Bekannten publik zu machen. Die resultierende Abkehr von einer softwaregestützten Zeitmessung hin zu einer Hardwarelösung war ein wichtiger Faktor für den erfolgreichen Abschluss des Projekts.

Beispiel 3: Projekt Leiterlenspiel

Wer kennt es nicht: das Leiterlenspiel, auch Schlangen und Leitern oder Snakes and Ladders genannt. Ein Leiterlenspielplan enthält eine Kette von Spielfeldern, die spiralförmig oder im Zickzack von einem Startfeld zu einem Zielfeld führt. An mehreren Stellen sind zwei nicht direkt aufeinander folgende Spielfelder durch Schlangen, Leitern oder andere Zeichnungen miteinander verbunden. Jeder Spieler beginnt mit einer Figur auf dem Startfeld. In der Regel wird abwechselnd mit einem einzigen Würfel gewürfelt. Das Ergebnis gibt an, wie viele Felder die Spielfigur vorrücken darf. Endet ihr Zug auf dem Startfeld einer Leiter, wird sie auf deren Endfeld vorgerückt. Endet sie auf einer Schlange, wird sie auf deren Endfeld, das wieder näher am Startfeld liegt, gezogen. Sieger ist, wer zuerst das Zielfeld erreicht.

Das Spiel ist in seiner Grundform ein reines Glücksspiel ohne strategische Elemente und ohne Interaktion zwischen den Spielerinnen. Es gibt unzählige Darstellungen des Spielplans mit unterschiedlicher Anzahl der Spielfelder und unterschiedlicher Positionierung und Länge der Leitern und Schlangen. Je nach Altersgruppe sollte das Spiel nicht zu lange, aber auch nicht zu kurz dauern. Es sollte ein ausgewogenes Verhältnis zwischen Leitern (die die Spieler schnell voranbringen) und Schlangen (die die Spieler zurückwerfen) bestehen. Zu viele Schlangen machen das Spiel frustrierend, während zu viele Leitern das Spiel schnell langweilig werden lassen. Die Spannung sollte gegen Ende des Spiels zunehmen. Das bedeutet, dass Leitern am Ende des Spielplans seltener oder kürzer sein sollten, während Schlangen die Gefahr erhöhen. Ein guter Spielplan zeichnet sich zudem dadurch aus, dass die meisten Spielerinnen möglichst gleichzeitig in die Nähe des Zielfeldes kommen und auch Spielerinnen, die zu Beginn mehrfach zurückgefallen sind, einen großen Rückstand aufholen können. Für einen vorgegebenen Spielplan kann die durchschnittliche Anzahl Würfe bis zum Erreichen des Zielfeldes entweder durch eine Monte-Carlo-Simulation oder rein mathematisch durch eine Modellierung mittels einer absorbierenden Markov-Kette bestimmt werden.

Im Rahmen eines Wahlpflichtkurses zu Spielstrategien bei traditionellen Brettspielen stellten sich Schülerinnen die Frage, wie man als Spielzeughersteller ein möglichst attraktives Spielfeld für ein Leiterlenspiel entwickeln und Spielfelder von im Handel angebotenen Leiterlspielen überprüfen könnte. In einem ersten Schritt

musste eine geeignete Datenstruktur zur Darstellung des Spielfeldes entwickelt werden. Anschließend musste die Grundsatzentscheidung getroffen werden, ob für die Untersuchung eines typischen Spielverlaufs eher auf eine Monte-Carlo-Simulation oder auf die Modellierung mit einer Markov-Kette gesetzt werden sollte.

Im Laufe der Softwareentwicklung entstand der Wunsch nach einer guten Visualisierung verschiedener Aspekte des Spiels, wie zum Beispiel die durchschnittliche Anzahl der Würfe bis zum Spielende oder bis zum Erreichen des Zielfeldes. Auch für die Optimierung des Spielplans wurde bald eine interaktive grafische Benutzeroberfläche benötigt, um Leitern und Schlangen neu zu positionieren und die daraus resultierenden Auswirkungen auf den Spielverlauf zu analysieren. Aus der einfachen Fragestellung entwickelte sich schnell ein interessantes Softwareprojekt. Eine wichtige Aufgabe des Informatiklehrers bei diesem Projekt war es, die Zielsetzungen des Projektes einzugrenzen und dafür zu sorgen, dass die Ziele der Softwareentwicklung in der verfügbaren Zeit auch erreichbar waren.

Beispiel 4: Offene Datenbestände und Schnittstellen nutzen

Die ersten drei Beispiele beschreiben Projekte, an denen mehrere Lernende beteiligt waren und Zusammenarbeit zentral für das Gelingen waren. Projektunterricht kann aber auch mit Kleingruppen, Zweierteams oder als Einzelarbeit stattfinden. Teamarbeit und soziale Kompetenzen rücken in den Hintergrund, aber die Lernenden setzen sich ebenfalls eigene Projektziele, bestimmen ihre Mittel und Methoden und machen sich laufend Gedanken über den Projektverlauf.

Neben Softwareprojekten eignen sich für den Projektunterricht mit Zweierteams oder als Einzelarbeit auch Themen aus dem Bereich Data Science. Heute ermöglichen Schnittstellen den Zugriff auf externe und aktuelle Daten und Funktionen von Diensten. Diese Möglichkeiten können genutzt werden, um die Analyse und Visualisierung von Daten in realen Projekten anzuwenden. Gängige Beispiele sind die Analyse von Wetterdaten oder Trends in Social-Media-Diensten, die Auswertung von Sportresultaten oder demografischen Entwicklungen.

Data Science und die Nutzung von Open Data sind zunächst ein Thema für höhere Schulstufen. Verschiedene Plattformen stellen Anwendungen im Bereich Big Data, maschinelles Lernen oder Data Mining samt Programmierumgebungen zur Entwicklung eigener Analysen zur Verfügung. Mithilfe von Tabellenkalkulationsprogrammen können aber auch in unteren Stufen größere Datenbestände analysiert werden. Allerdings müssen die Daten bereits in standardisierter Form vorliegen, damit sich die Lernenden nicht aufwendig mit Schnittstellen auseinandersetzen müssen.

Beispiel 5: Projektarbeit in angewandter Informationssicherheit

Angewandte Informationssicherheit ist eine Herausforderung: In der Praxis ist es schwierig, ein System funktional korrekt zu realisieren und keine Sicherheitsprobleme zu übersehen. Diese Schwierigkeiten können nur an Systemen mit einer gewissen Komplexität demonstriert werden, was einen entsprechenden Zeitaufwand voraussetzt. Dafür eignet sich die Projektarbeit ganz besonders.

An der ETH Zürich wurde im Rahmen des Applied Security Laboratory ein Projekt durchgeführt, bei dem Studierende angewandte Aspekte der Informationssicherheit in einem praxisnahen Kontext erlernten (Näf & Basin, 2005). Dieses Konzept lässt sich auf die Ausbildung an Berufsschulen übertragen. Zu Beginn der Lehrveranstaltung erhielten die Studierenden Selbststudienmaterial mit Experimenten, die sie „hands-on“ am Computer bearbeiteten, um das nötige Wissen für das anschließende Projekt zu erlangen. In Vierergruppen erhielten sie ein Pflichtenheft für eine Applikation, die sie unter Berücksichtigung von Sicherheitsaspekten realisieren sollten. Dabei konnten sie entweder bestehende Open-Source-Komponenten nutzen oder eigene Teile implementieren.

Ein zentraler Bestandteil war die Durchführung einer Risikoanalyse und die Umsetzung geeigneter Sicherheitsmaßnahmen. Nach der Realisierungsphase tauschten die Gruppen ihre Systeme aus, führten gegenseitige Reviews durch, bewerteten die Sicherheitsmaßnahmen und deckten Schwachstellen auf.

Literatur

Frey, K. (2012). *Die Projektmethode*. Beltz Verlag.

Näf, M., & Basin, D. (2005). Konflikt oder Review – zwei Ansätze für Labors in angewandter Informationssicherheit. *Informatik Spektrum*, 28(5), 407–412.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Teil V

Unterrichtstechniken

Einstieg: Im Modul „Pflegeinformatik“ an der Berufsfachschule für Pflegekräfte werden Aufbau und Funktionsweise des Internets behandelt. Heute steht der Aufbau von IP-Adressen auf dem Lehrplan. Lehrer S. erklärt, dass jedem Gerät im Internet eine IP-Adresse zugeordnet ist, und erläutert deren Struktur. Die Lernenden zeigen jedoch wenig Begeisterung für dieses Thema. Auch die Übung, die IP-Adresse der eigenen Geräte zu ermitteln, stößt auf wenig Interesse. Die meisten Lernenden sind gedanklich bei anderen Themen.

Aufgrund der Allgegenwärtigkeit von Informations- und Kommunikationstechnologien in nahezu allen Bereichen unseres Lebens bietet der Informatikunterricht ein besonderes Potenzial zur Motivation der Lernenden.

Der Bezug auf die uns umgebenden informatischen Anwendungen ermöglicht es, Unterrichtsinhalte direkt aus der realen Welt abzuleiten oder darauf zu übertragen. Dadurch wird nicht nur die eigene Lebenswelt besser verständlich, sondern das Gelernte auch nützlich und bedeutsam. Dieses besondere Potenzial der Informatik gilt es zu nutzen, um Lernende zu motivieren.

Motivierungen lassen sich jedoch nicht einfach improvisieren. Jäckel (2018) klassifiziert häufig genutzte Motivierungen samt Beispielen für den Unterrichtseinstieg im Informatikunterricht. In Anlehnung daran unterscheiden wir drei grundlegende Motivierungsmittel, die Lehrkräfte nutzen können:

Inhalt – Bedeutsamkeit Durch das Aufzeigen von Relevanz und Anwendungsmöglichkeiten des Lerngegenstands erkennen die Schülerinnen und Schüler, warum der Lerninhalt für sie wichtig ist.

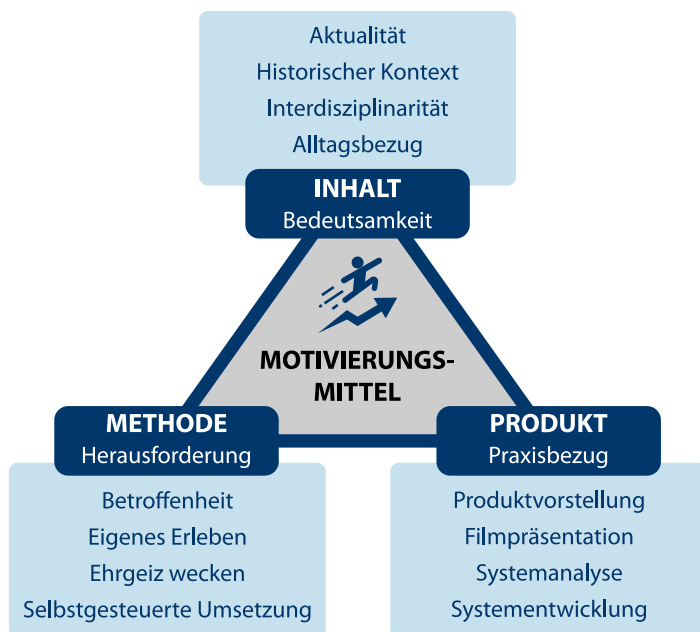


Abb. 18.1 Motivierungsmittel und zugeordnete Motivierungstypen

Methode – Herausforderung Der Einsatz spezieller Methoden, die den Unterricht problem- und handlungsorientiert gestalten, bindet die Lernenden direkt in den Lernprozess ein.

Produkt – Praxisbezug Ein direkter Praxisbezug anhand konkreter Produkte zeigt den Lernenden den unmittelbaren Nutzen ihres Wissens und Könnens.

Nachhaltige Motivation entsteht, wenn die Lehrkraft die persönlichen Motive und Interessen der Lernenden mit geeigneten Motivierungsmitteln anspricht. Ebenfalls basierend auf Jäckel (2018) beschreiben wir zu jedem Motivierungsmittel typische Herangehensweisen und illustrieren diese Motivierungstypen an Beispielen (siehe Abb. 18.1).

Inhalt – Bedeutsamkeit

Die Bedeutsamkeit ist nicht nur für die Motivation wichtig, sondern spielt auch eine Rolle bei der Auswahl der Unterrichtsinhalte. Drei der fünf Kriterien der fundamentalen Ideen – Zeitkriterium, Sinnkriterium und Horizontalkriterium – finden ihre Entsprechung in den Motivierungstypen historischer Kontext, Alltagsbezug und Interdisziplinarität.

Aktualität Aktuelle Ereignisse und Probleme aus der Lebenswelt oder den Medien fördern bei den Lernenden das Bewusstsein und Verständnis für die Relevanz des zu behandelnden Inhalts. Beispiele:

- Regelmäßig berichten die Medien über Hackerangriffe, bei denen Zugangsdaten zu Onlinediensten gestohlen werden. Dies bietet Anlass, die Sicherheit von Passwörtern zu thematisieren.
- Die politische Debatte über den Einsatz von Überwachungskameras im öffentlichen Raum verdeutlicht das Spannungsfeld zwischen Privatsphäre und staatlicher Kontrolle.
- Aktuelle Medienberichte zu Mobbingfällen bieten eine Gelegenheit, das Thema „Datenschutz-Einstellungen und Freigabebeschränkungen in sozialen Netzwerken“ zu behandeln.

Historischer Kontext Interessante Persönlichkeiten, Quellen und historische Problemstellungen schaffen Bezüge zur Geschichte der Informatik. Sie ermöglichen Vergleiche zwischen historischen und aktuellen Technologien und verdeutlichen die Einflüsse wissenschaftlicher und gesellschaftlicher Rahmenbedingungen auf die Entwicklung der Informatik. Beispiele:

- Mithilfe von Archiven alter Websites können die Entwicklung der Gestaltung von Webseiten und grundlegende Gestaltungsprinzipien untersucht werden.
- Die Geschichte des Zweiten Weltkriegs und die Verschlüsselung von geheimen Daten mittels der Chiffriermaschine Enigma können als Einstieg in das Thema Verschlüsselung dienen.
- Die Entwicklung der Schrift (Steinplatten, Handschrift, Buchdruck, Digitalisierung) zeigt verschiedene Methoden der Codierung von Informationen auf.

Interdisziplinarität Querbezüge zu anderen Wissenschaften und deren Arbeitsmethoden sprechen die Interessen der Schülerinnen und Schüler an, verdeutlichen die übergreifende Bedeutung von Informatikthemen und eröffnen neue Perspektiven. Beispiele:

- Ausgehend von der deutschen Sprache können Syntax, Semantik und Grammatik von HTML und anderen Markupsprachen eingeführt werden.
- Ameisenalgorithmen, inspiriert vom Verhalten realer Ameisen, sind besonders nützlich für das Lösen von kombinatorischen Optimierungsproblemen wie dem Travelling-Salesman-Problem oder dem Rucksackproblem.
- Die Funktionsweise von neuronalen Netzen kann anhand des menschlichen Gehirns erklärt werden. Neuronale Netze lernen durch das Erkennen von Mustern in Daten, ähnlich wie das Gehirn durch Erfahrungen lernt.

Alltagsbezug Eigene Tätigkeiten, vertraute Probleme und authentische Beispiele aus dem Alltag fördern das Verständnis der eigenen Lebenswelt. Beispiele:

- Die Darstellung eines Wetterberichts in Textform, illustriert mit Icons, auf einer Isobarenkarte oder anhand einer Sequenz von Satellitenbildern, verdeutlicht die Vor- und Nachteile verschiedener multimedialer Darstellungsformen.
- Die Analyse der Funktionalität geeigneter Alltagsgegenstände (z. B. Backofen, Stoppuhr) und deren Darstellung in Diagrammen und Tabellen führt zum Thema Sprachen und Automaten.
- Das Heraussuchen einer komplexen Bahnverbindung unter Berücksichtigung von Rahmenbedingungen (z. B. Fahrradmitnahme, Barrierefreiheit) in einem Onlinefahrplan dient als Einführung in relationale Datenbanken.

Methode – Herausforderung

Gut durchdachte methodische Herausforderungen sind ein Schlüssel zur Motivation im Informatikunterricht. Aufgaben, die Neugier wecken und aktives Mitdenken fördern, laden die Lernenden dazu ein, sich intensiv mit Inhalten auseinanderzusetzen. Besonders wirkungsvoll sind Aufgabenstellungen, die Elemente verschiedener in diesem Buch vorgestellter Methoden kombinieren – etwa aus dem Projektunterricht, dem Pair Learning, dem entdeckenden Lernen, der Arbeit mit Lernaufgaben oder der Puzzlemethode. Entscheidend ist dabei die richtige Dosierung der Schwierigkeit: Die Aufgaben sollen fordern, aber nicht überfordern.

Betroffenheit Ein Sachverhalt oder eine Problemstellung macht die Schülerinnen und Schüler zu direkt Betroffenen oder Akteuren. Ihre Vorerfahrungen und Erwartungen werden dabei bewusst erfüllt oder enttäuscht. Beispiele:

- Schülerinnen erstellen eigene Piktogramme als Rasterbilder mit sehr geringer Auflösung. Diese anspruchsvolle Aufgabe fördert nicht nur das Verständnis für Rasterbilder, sondern auch das Gespür für die Gestaltung von Bedienelementen auf Benutzeroberflächen.
- Angehende Webdesigner erkunden mit verbundenen Augen und einem Screenreader die Informationen auf verschiedenen Webseiten, um die Herausforderungen blinder oder sehbehinderter Menschen nachzuvollziehen. Diese Übung sensibilisiert für barrierefreie Webauftritte.
- Die Lehrperson liest den Cachespeicher und die History des Browsers am Lehrer-PC aus und zeigt die besuchten Webseiten am Beamer. So werden die Schülerinnen und Schüler für die digitalen Spuren sensibilisiert, die sie auf ihren Geräten und im Internet hinterlassen.

Eigenes Erleben Schülerinnen und Schüler schlüpfen in verschiedene Rollen oder agieren in Experimenten und szenischen Darstellungsformen, um ihre Vorstellungskraft, Kreativität und ihr Einfühlungsvermögen zu aktivieren. Beispiele:

- Die Schülerinnen und Schüler simulieren die Kommunikation mit einem Menschen, der aufgrund einer Beeinträchtigung nur noch mit den Augen blinzeln kann. Der „Blinzelcode“ dient als Einführung in die Codierung mittels Binärzahlen.
- Die Schülerinnen und Schüler stellen sich in einer Reihe auf und müssen sich möglichst schnell nach ihrer Größe, ihrem Geburtsdatum oder ihrem Namen im Alphabet sortieren. Je nach erlaubten Anweisungen können verschiedene Sortieralgorithmen erlebt werden.
- Die Lernenden der Berufsfachschule dürfen sich mit einer VR-Brille in einfachen VR-Anwendungen bewegen, um später selbst 3D-Anwendungen zu programmieren.

Ehrgeiz wecken Fragespiele, Rätsel und Wettbewerbe fördern Ehrgeiz und Leistungsbereitschaft, indem sie kognitive Herausforderungen mit Spiel und Unterhaltung kombinieren. Zeitvorgaben oder Aussicht auf Belohnung verstärken den Ehrgeiz zusätzlich. Beispiele:

- Logikrätsel oder Logicals sind Rätsel, die man mithilfe logischer Schlussfolgerungen lösen kann. Sie eignen sich sowohl als Einstieg als auch zur Repetition im Thema Logik.
- In Gruppen sollen die Schülerinnen und Schüler in einer vorgegebenen Zeit möglichst viele absichtlich eingebaute Syntax- und Logikfehler in einem Programm identifizieren.
- Eine Zuordnungsübung, ob Bilder von einem Menschen oder einem Bildgenerator erstellt wurden, führt ins Thema „Wahrheitsgehalt von Bildern“ ein.

Selbstgesteuerte Umsetzung Die Ankündigung, dass im Rahmen der Behandlung des Themas die Lernenden selbstgesteuert tätig werden, regt an, Inhalte und Zusammenhänge zu entdecken und eigene Strategien zu entwickeln. Beispiele:

- Arbeitsteilig analysieren die Schülerinnen die unterschiedlichen Funktionen (zivil, militärisch) von Drohnen, um gesellschaftliche Wechselwirkungen selbstständig zu recherchieren und zu bewerten.
- Die Lernenden simulieren digitale Angriffe und Verteidigungsstrategien in einer sicheren Umgebung. Sie lernen die Grundlagen der Cybersecurity und wie sie Systeme vor Bedrohungen schützen können.
- In Onlineshops und Vergleichsportalen informieren sich Schülerinnen und Schüler über aktuelle Kenngrößen, Herstellerfirmen und Preise von Hardware und deren Entwicklung. Sie lernen dabei gängige Hardwarekomponenten und -technologien kennen.

Produkt – Praxisbezug

Wenn Lernende konkrete Produkte entwickeln können, verknüpfen sie Konzept- und Produktwissen auf sinnvolle Weise. Auf diese Weise wird sowohl das Verständnis vertieft als auch die Fähigkeit gefördert, informatisches Denken auf reale Situationen zu übertragen und kritisch zu reflektieren. Besonders im Rahmen von Projektarbeiten bietet sich dieser Ansatz als wirkungsvolle didaktische Strategie an.

Produktvorstellung Die Präsentation von digitalen oder physischen Produkten macht Eigenschaften und Funktionen sichtbar. So können sie aus verschiedenen Blickwinkeln erforscht, ausprobiert und systematisch getestet werden. Beispiele:

- Von der Lehrerin wird ein interaktiver Modellrechner vorgestellt, der in das Thema Aufbau und Arbeitsweise der Von-Neumann-Architektur einführt.
- Suchen und Filtern in einem Onlineshop ermöglicht den ersten Zugang zu SQL-Abfrage auf großen Datenkollektionen.
- Experimentieren mit einfachen KI-Modellen bietet einen handlungsorientierten Einstieg in die Funktionsweise von Machine-Learning-Algorithmen.

Filmpräsentation Kurze Unterrichtsvideos und Filmsequenzen unterstützen die gedankliche und emotionale Verarbeitung von Inhalten. Videos sind den Lernenden aus ihrer Freizeit vertraut und ihre Ausdruckskraft weckt Interesse. Beispiele:

- Fernsehausschnitte über die Gewinnung und Entsorgung von Rohstoffen für digitale Geräte sensibilisieren für den Themenbereich Nachhaltigkeit und Digitalisierung.
- Der Dokumentarfilm „Plug & Pray“ mit Joseph Weizenbaum dient als Einstieg in Diskussionen über Informatik und Ethik.
- Onlineclips von Influencern über Kryptowährungen zeigen Möglichkeiten und Risiken digitaler Zahlungsmittel auf.

Systemanalyse Informatiksysteme, deren Modelle oder Bestandteile werden analysiert, um erste Erklärungen für ihre Funktionsweise zu erarbeiten. Diese analytische Herangehensweise fördert ein grundlegendes Verständnis für das jeweilige System. Beispiele:

- Experimente und Analysen zur Funktionsweise von Barcodes und QR-Codes zeigen deren Aufbau und Einsatzmöglichkeiten.
- Die Lernenden untersuchen den Speicherbedarf verschiedener Programme und Apps auf ihren eigenen Rechnern. So erhalten sie ein Gefühl für effiziente Speicherverwaltung.
- Die Schülerinnen und Schüler vergleichen verschiedene KI-Assistenten hinsichtlich ihrer Leistungsfähigkeit und Anwendungsbereiche und setzen sie sich mit deren Potenzialen und Grenzen auseinander.

Systementwicklung Die Ankündigung, dass im Rahmen eines Themas eigene Informatiksysteme und Anwendungen entwickelt werden, ermutigt die Lernenden, ihr Wissen praktisch anzuwenden und konkrete Ergebnisse zu erzielen. Beispiele:

- Den Lernenden wird in Aussicht gestellt, ein sensorgestütztes System zur automatischen Bewässerung von Pflanzen zu programmieren.
- Den Schülerinnen und Schülern wird das Ziel vorgegeben, ein System zur Überwachung der Torlinie im Fußball zu entwickeln, das mithilfe von Kameras und Bildverarbeitung arbeitet.
- Die Zielsetzung, ein einfaches Retrocomputerspiel wie Pong oder Snake zu entwickeln, bietet einen motivierenden Rahmen, um grundlegende Konzepte der Spieleentwicklung einzuführen.

Lehrer S. könnte sich in unserem einleitenden Beispiel an den Motivierungstypen „Betroffenheit“ und „eigenes Erleben“ orientieren: Im Gesundheitswesen spielt der Datenschutz eine zentrale Rolle, auch der Umgang mit Metadaten wie IP-Adressen, die bei jeder Nutzung von Internetdiensten anfallen. Pflegekräfte sollten deshalb ein Verständnis für IP-Adressen entwickeln. Zu Beginn der Unterrichtsstunde könnte Lehrer S. anhand der Logfiles der Zugriffe auf die Lernplattform aufzeigen, wie IP-Adressen protokolliert werden. Alternativ könnte er demonstrieren, wie bereits durch den Besuch vieler Websites die eigene IP-Adresse an Drittanbieter weitergeleitet wird und wie darauf spezialisierte Unternehmen nach und nach Personenprofile erstellen können.

Ein breites Repertoire an Motivierungstypen trägt dazu bei, den Unterricht lebendig und attraktiv zu gestalten. Da Lernende unterschiedliche Lernvoraussetzungen, Interessen und Zugänge zum Lernen mitbringen, ist es sinnvoll, verschiedene Ansätze zur Motivation gezielt zu kombinieren.

Literatur

Jäckel, S. (2018). *Zur Motivierung im Informatikunterricht: eine Charakterisierung unterrichtspraktischer Einstiege aus der Perspektive von Lehrenden und Lernenden*. <https://doi.org/10.22032/dbt.37548>. Zugriffen am 07.04.2025.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: „Guten Morgen. Nachdem wir in den ersten beiden Kurstagen die Grundlagen des Programmierens erarbeitet haben, setzen wir uns in den kommenden Stunden mit einigen wichtigen Datenstrukturen auseinander: mit normalen Listen, doppelt verketteten Listen, Hashes und Binärbäumen.“ Bei dieser Begrüßung beschleicht Kursleiter C. ein ungutes Gefühl. Die Kursteilnehmer haben ja noch keine Vorstellung von diesen Fachbegriffen.

Im Informatikunterricht werden oft Inhalte behandelt, von denen die Lernenden zunächst keine Vorstellung haben. Der direkte Einstieg mit neuen Fachbegriffen oder Definitionen ist nicht ideal. Die Lernenden können das Neue weder aufnehmen noch mit bestehendem Vorwissen verknüpfen.

In der Informatik gibt es viele Fachbegriffe und mehr oder weniger abstrakte Konzepte. Als Lehrkraft ist man manchmal verleitet, zunächst die Begriffe einzuführen, zu definieren und grundlegende Sachverhalte zu erläutern, bevor man die eigentliche Kernidee der Unterrichtseinheit vermittelt. Hier kann ein Advance Organizer – kurz AO – Abhilfe schaffen, wie bereits 1960 von David Ausubel (1960) vorgeschlagen. Der AO fasst die Hauptideen der zu vermittelnden Lerninhalte gleich zu Beginn des Unterrichts prägnant zusammen. Er baut ausschließlich auf bereits bekannten Begriffen und Konzepten auf. Auf diese Weise können die Lernenden den neuen Stoff mit bereits vorhandenem Wissen verknüpfen. Das Verstehen und Einordnen wird erleichtert und der neue Stoff gut verankert. Anders ausgedrückt: Ein Advance Organizer holt die Lernenden dort ab, wo sie gerade stehen. Er setzt möglichst wenig voraus und verknüpft die nachfolgenden Inhalte auf anschauliche Weise mit bereits Bekanntem.

Der AO ist als Unterrichtstechnik in der allgemeinen Didaktik gut dokumentiert; seine positiven Effekte gelten als gesichert. Je abstrakter das Thema, desto mehr drängt sich ein AO auf. Besonders wertvoll sind Advance Organizer, die eine Ana-

logie zu alltäglichen Themen herstellen; Berührungsängste vor komplexen Informatikthemen können so abgebaut werden: „Aha, das ist ja gar nicht wirklich anders als beim Kuchenbacken, beim Sortieren von Spielkarten oder beim Verschicken eines Briefes.“

Einen Advance Organizer kann man nicht noch schnell vor Beginn der Unterrichtsstunde entwickeln. Er setzt viel Sachkompetenz voraus und auch Kreativität. Die folgende Anleitung zeigt wichtige Punkte auf:

Kernkonzepte identifizieren Die zentralen Ideen des neuen Lerngegenstandes bestimmen und einfache Erklärungen und Analogien identifizieren.

Vorwissen ermitteln Das vorhandene Wissen der Lernenden zum Thema analysieren, ohne dabei die eigenen Vorkenntnisse mit denen der Lernenden zu wechseln.

Umgangssprachliche Formulierung Direkt ins Thema einsteigen ohne Verwendung von Fachbegriffen und Definitionen; wo möglich mit Geschichten oder Beispielen die Lernenden auf emotionaler Ebene ansprechen.

Relevanz des Themas aufzeigen Nutzen des neuen Wissens darlegen.

Neues mit Bekanntem verbinden Bezüge zwischen bereits bekanntem Wissen und neuen Informationen herstellen.

Visualisierungen nutzen Grafiken, kurze Audios oder Videos oder Gegenstände zur Veranschaulichung verwenden.

Zentrale Punkte hervorheben Zum Schluss die wichtigsten Punkte noch einmal deutlich hervorheben.

Advance Organizer erklären die wesentlichen Kernideen der neu zu vermittelnden Fachinhalte anhand bereits bekannter Begriffe und Konzepte und fördern so das Verknüpfen mit dem Vorwissen. Die Lernenden verstehen besser und vernetzen tiefgründiger.

Beispiel 1: Funktionsweise von KI-Assistenten

Im Rahmen eines Kurses an einer Volkshochschule zum Thema Künstliche Intelligenz wurden verschiedene Anwendungen von KI-Assistenten behandelt. Um das Potenzial und auch die Risiken besser zu verstehen, möchte die Kursleiterin auch einen Einblick in die Funktionsweise von KI-Assistenten vermitteln. Ein Advance Organizer dazu könnte wie folgt lauten:

„Die Arbeit von Komponisten und die Funktionsweise von KI-Assistenten weisen einige interessante Parallelen auf. In beiden Fällen geht es darum, Inhalte durch die Kombination

und Manipulation von Grundelementen zu erzeugen. Komponisten schaffen Musik, indem sie Melodie, Harmonie, Rhythmus und Textur kombinieren. Sie nutzen ihr Wissen, ihre Kreativität und Inspiration, um etwas Neues und Einzigartiges zu schaffen. Oft bauen sie auf bestehenden musikalischen Strukturen auf und variieren diese.

KI-Assistenten erzeugen zum Beispiel Text, indem sie Wörter und Sätze auf der Grundlage von Mustern und Strukturen kombinieren, die sie während ihres Trainings gelernt haben. Sie verwenden Algorithmen und statistische Modelle, um auf der Grundlage des gegebenen Kontexts vorherzusagen, welches Wort oder welcher Satz als Nächstes kommt.

Natürlich gibt es auch Unterschiede, insbesondere im Hinblick auf die menschliche Kreativität und das intuitive Verständnis von Emotionen und kulturellem Kontext, das ein Komponist einbringen kann. KI-Assistenten basieren auf Daten und Algorithmen und haben es schwer, eine persönliche Note oder absichtliche Unvollkommenheit zu berücksichtigen.“

Beispiel 2: Routing

Ein Advance Organizer für eine Doppelstunde zum Thema Adressierung und Routing im Internet an einer Technikerschule könnte so lauten:

„Sie surfen im World Wide Web. Bei jedem Klick auf einen Link schickt Ihr Webbrowser eine Anfrage an einen Webserver. Der Server antwortet mit der gewünschten Webseite. Aus Benutzersicht ist der Prozess einfach. Doch was geschieht hinter den Kulissen? Wie finden die Datenpakete den Weg zum Server und zurück? Mit dieser Frage beschäftigen wir uns heute.

Grundsätzlich funktioniert das ähnlich wie die Briefpost im Alltag. Stellen Sie sich vor: Sie werfen an der Bahnhofstraße im Kreis 1 in Zürich eine Postkarte ein, die an eine Bekannte in Berlin-Kreuzberg adressiert ist. Was geschieht? Zunächst wird die Postkarte zur lokalen Poststelle im Kreis 1 gebracht. Dort gibt es eine Grundregel: Alles, was nicht für den Kreis 1 adressiert ist, geht weiter zum städtischen Verteilzentrum. Also reist die Postkarte dahin. Auch im Verteilzentrum gibt es eine Grundregel: Was für Europa und nicht für Zürich ist, geht zum Bahnhof.

Die Postkarte landet also via Zug und Verteilzentrum in Berlin irgendwann in der Poststelle von Berlin-Kreuzberg. Dort wird sie aufgrund der Straßenangaben dem richtigen Briefträger zugewiesen. Dieser bringt die Karte dann an ihren Bestimmungsort.

Was ist wichtig an diesem Mechanismus? Die kleineren Poststellen am Anfang und am Ende müssen die genauen Straßen und Personennamen kennen. Alle Zwischenstellen dagegen müssen nur die ungefähre Richtung zum Ziel wissen. Zum Beispiel leitet das Verteilzentrum in Zürich alle Sendungen nach Berlin an den Bahnhof weiter. Die weiteren Stationen zwischen Bahnhof und Empfänger interessieren die Mitarbeitenden im Verteilzentrum nicht. Jede Sendung wird so in kleinen Sprüngen weitergereicht.“

Im anschließenden Unterricht beschäftigen sich die Studierenden mit der Adressierung von Computern im lokalen Netz und wie die Datenpakete den Weg von einem lokalen Netz in ein anderes finden. Dabei stellt die Lehrerin immer wieder den Bezug zur Alltagsanalogie Briefpost her. Zum Beispiel wenn die Studierenden die Netzinformationen konkret an einem Rechner im Netz anschauen. Der pro Netz eingetragene Default Gateway entspricht der lokalen Poststelle in der Analogie.

Beispiel 3: Funktionsweise von Suchsystemen

Ein Advance Organizer an einer Berufsschule für eine Doppelstunde zum Thema Webcrawler und Index einer Suchmaschine könnte wie folgt lauten:

„Sie benutzen fast täglich Suchmaschinen im Internet. Diese Werkzeuge arbeiten erstaunlich effizient und sind in der Lage, Resultate aus Milliarden von Webseiten innerhalb von Sekundenbruchteilen zu liefern. Nach dieser Doppelstunde werden Sie wissen, wieso die Suchmaschinen so schnell sind.

Den wichtigsten Grund für die Schnelligkeit kann ich Ihnen in einer Minute erklären: Nehmen Sie irgendein Sachbuch, zum Beispiel Computernetzwerke von Tanenbaum mit über 1000 Seiten. Wenn Sie wissen möchten, was MPLS ist, was tun Sie? Sie schauen im Stichwortverzeichnis nach. Dort sind alle wichtigen Begriffe aufgeführt, zusammen mit den Zahlen aller Seiten, auf denen der jeweilige Begriff diskutiert wird. Dank des Stichwortverzeichnisses brauchen Sie nicht das ganze Buch zu durchsuchen, sondern müssen nur die paar Seiten prüfen, die zum gesuchten Begriff angegeben sind. Der Zeitgewinn ist enorm.

Das Geheimnis liegt also in der guten Vorbereitung. Das Stichwortverzeichnis ist eine Datenstruktur, in welcher der Buchinhalt vorverarbeitet wurde, um das Suchen nach Begriffen zu beschleunigen. Eine Suchmaschine geht ebenso vor: Sie legt ein riesiges Stichwortverzeichnis mit den in Webseiten gefundenen Begriffen an und merkt sich zu jedem Begriff, auf welchen Webseiten er auftaucht.“

Im Anschluss könnte den Lernenden die Funktionsweise des Webcrawlers, der Indexierungskomponente und des Indexes vermittelt werden. Dabei ist immer wieder der Bezug zur Analogie „Stichwortverzeichnis“ möglich: Suchmaschinen können dank Computerhilfe die Listen verschiedener Begriffe miteinander kombinieren, weshalb eine Anfrage aus mehreren Begriffen bestehen kann. Im Buch ist das nicht direkt möglich, man müsste die Listen der Seitenzahlen zu mehreren Begriffen manuell kombinieren.

Beispiel 4 Einstieg in HTML

Ein Advance Organizer kann eine hilfreiche Methode sein, um den Einstieg in HTML zu erleichtern. Er bereitet die Lernenden auf das Thema vor und schafft so eine gedankliche Struktur. Beispiel:

„Stellt euch vor, ihr wollt ein Buch schreiben – aber nicht auf Papier, sondern im Internet, damit es jeder lesen kann. Mithilfe von HTML sagt ihr dem Computer, wie er euer Buch auf einer Webseite anzeigen soll. In einem Buch gibt es Kapitelüberschriften, Absätze, Bilder und Links zu anderen Kapiteln – all das braucht auch eine Webseite.

HTML ist wie die Struktur eures Buches. Ihr markiert mit den sogenannten Tags, welche Teile eurer Seite Überschriften sind, welche Teile Text und welche Bilder. Jeder dieser Tags funktioniert wie ein Befehl an den Computer: ‚Das hier ist eine Überschrift!‘ oder ‚Zeige hier ein Bild!‘.

Ohne HTML würde der Computer den gesamten Text als eine lange, unstrukturierte Zeile anzeigen – er wäre schwer zu lesen. Mit HTML zeigt ihr ihm, wie die Inhalte richtig strukturiert und dargestellt werden sollen. HTML ist der erste Schritt, um Webseiten zu bauen, die klar, verständlich und gut organisiert sind.“

Literatur

Ausubel, D. P. (1960). The use of advance organizers in the learning and retention of meaningful verbal material. *Journal of Educational Psychology*, 51, 267–272.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Nach Schulschluss sitzen O. und K. bei einem Kaffee. O.: „Pointers, ein so abstrakter Begriff und doch wichtig. Es gelingt mir einfach nicht, dieses Konzept der Programmiersprache C so zu vermitteln, dass es meine Lernenden verstehen.“ „Bei mir beginnen die Probleme schon viel früher“, meint K. „Ich habe heute versucht zu erklären, welche Wirkung das Kopieren einer Formel aus einer Zelle einer Tabellenkalkulation in eine andere Zelle hat.“

Abstraktion spielt in der Informatik eine große Rolle. Bits und Bytes kann man nicht anfassen, von Betriebssystemen und Programmen sieht man nur die Oberfläche. Viele Themen im Informatikunterricht sind deshalb für die Lernenden schwer zu verstehen.

Die im Beispiel beschriebenen Schwierigkeiten sind nicht ungewöhnlich: Viele Begriffe und Sachverhalte der Informatik sind abstrakt und per se wenig anschaulich. Der Vorgang der Abstraktion, bei dem ein Ausschnitt der realen Welt auf ein Modell reduziert wird, das nur die wesentlichen Elemente hervorhebt, ist für die Informatik von zentraler Bedeutung. Einmal verstanden, erweist sich ein abstraktes, von konkreten Gegebenheiten losgelöstes Modell als nützliches Werkzeug.

Auch eine zweite Art der Abstraktion findet in der Informatik häufig Anwendung: Gewisse Aspekte eines Sachverhaltes werden bewusst ausgeblendet oder – anders ausgedrückt – gekapselt. Der Programmierer muss und soll sich nicht um die Details einer Programmbibliothek kümmern. Auf einer höheren Abstraktionsebene muss man die sich darunter befindlichen Ebenen und deren Funktionsweise nicht kennen. Das Filesystem als Beispiel vereinfacht den Zugriff auf Dateien; nur wenige Leute müssen wissen, mit welcher Technik auf eine Festplatte zugegriffen wird.

Abstraktion ist eine nützliche Methode; sie ist aber anspruchsvoll und setzt viel Erfahrung voraus. Kinder lernen anhand konkreter Gegenstände und müssen sich

Ereignisse und Abläufe vorstellen können. Je älter wir werden, desto zugänglicher sind wir für Erklärungen, die bildlich oder in Textform vorliegen.

Von Jerome Bruner (1988) stammt eine für den Unterricht nutzbringende Klassifikation verschiedener Darstellungsformen. Aufbauend auf Piaget unterscheidet er drei Repräsentationsebenen:

Enaktive Repräsentation Erfassen von Sachverhalten durch eigenes Tun: Diese Repräsentationsebene ist bei Kindern besonders ausgeprägt. Kinder lernen durch eigenes Handeln, durch Abtasten von Gegenständen, durch Beobachten. Um Dreirad zu fahren, brauchen Kinder keine Gebrauchsanweisung.

Ikonische Repräsentation Auf dieser Ebene werden Sachverhalte durch Bilder dargestellt. Konkrete Gegenstände, Ereignisse und Abläufe kann sich ein Mensch auch anhand von Visualisierungen vorstellen. Ein Hotelprospekt oder ein Stadtplan reichen oft aus, um sich ein Bild zu machen.

Symbolische Repräsentation Erfassen von Sachverhalten durch Symbole (Text, Zeichen etc.). Unter dem frei gewählten Begriff „Baum“ können wir uns problemlos einen Baum vorstellen. Wir brauchen weder auf den Baum zu klettern noch ein Bild des Baumes zu sehen. Symbolische Darstellungen haben den großen Vorteil, präzise und kompakt zu sein, und eignen sich insbesondere, wenn man von einem Thema bereits eine zutreffende intuitive Vorstellung hat.

Alle Menschen haben die Fähigkeit, mehr oder weniger flexibel zwischen den verschiedenen Repräsentationsebenen zu wechseln. Gemäß Bruner ist dieser Wechsel im Unterricht wichtig: Denkopoperationen sollen, wenn immer möglich, auf mehreren Ebenen stattfinden.

Neben der echten enaktiven Repräsentation (jede Lernende wird selbst mit physischen Gegenständen aktiv) bietet sich oft auch an, dass nur einige Lernende oder die Lehrkraft aktiv tätig werden und die anderen zuschauen. Häufig werden enaktive Vorgänge auch nur durch Manipulationen von Objekten in einer computer-gestützten Umgebung simuliert. Bekannte Beispiele sind Lernumgebungen, in denen die Lernenden virtuelle Roboter auf dem Bildschirm steuern können.

Enaktive Repräsentationen oder Analogien erfordern kreative Ideen, die durch Alltagsbeobachtungen, Fachpublikationen oder das Internet inspiriert werden können. Am Ende eines Themas können solche Repräsentationen auch gemeinsam mit den Schülerinnen und Schülern als Metareflexion erarbeitet werden.

Der Vorbereitungsaufwand für enaktive Repräsentationen ist nicht zu unterschätzen und sollte optimiert werden. Enaktive Repräsentationen lassen sich bezüglich des Aufwands grob in drei Klassen einteilen:

Überall vorhanden Manche Dinge finden sich in fast jedem Unterrichtszimmer. Dazu gehören nicht zuletzt die Schülerinnen und Schüler selber: Sie lassen sich einfach platzieren, bewegen und mit ein wenig Fantasie sogar kopieren. Dazu gibt es Stühle, Tische, Jacken, Mützen, Bleistifte, Kugelschreiber und vieles mehr.

Einfach hergestellt Das heißt: (1) Die benötigten Materialien sind schnell und billig verfügbar. (2) Sie können in ausreichender Zahl beschafft werden, sodass möglichst alle Lernenden selbst enaktive Erfahrung sammeln können. (3) Die Materialien können problemlos ins Schulzimmer transportiert werden. Diese Anforderungen erfüllen Papier, Karton, Schnur, Klebeband, Büroklammern, Gummibänder, PET-Flaschen, volle oder leere WC-Papierrollen, Eierverpackungen, Alufolie und vieles mehr.

Aufwendig in der Herstellung Ein großer Aufwand rechtfertigt sich nur, wenn das Resultat wiederholt im Unterricht eingesetzt werden kann. Der Vorbereitungsaufwand und der Ertrag, also die Verbesserung des Lernerfolgs, müssen in einem vernünftigen Verhältnis stehen. Beispiele von aufwendigen Repräsentationen: Labyrinth aus Holz, um Backtrackingverfahren handfest zu zeigen; kleine Kartoncomputer, die das Funktionieren eines Prozessors aufzeigen; Kartonschachteln in unterschiedlichen Farben und Größen zur Illustration von Filesystemen.

Unterricht und Denken müssen nicht formal, abstrakt in Text, Symbolen und Formeln ablaufen. Viele Themen lassen sich gut in Bildern visualisieren oder in aktivem Handeln begreifen.

Beispiel 1: Teile-und-herrsche

Teile-und-herrsche ist eine grundlegende Problemlösungsmethode in der Informatik, die sich durch das Zerlegen eines Problems in kleinere, besser handhabbare Teilprobleme auszeichnet. Die Vorteile der Methode sind vor allem die Verbesserung der Effizienz und die Vereinfachung der Problemlösung. Durch das Aufteilen des Problems kann man zudem oft parallele Verarbeitung nutzen.

Zum Beispiel in der 5. Klasse können die Schülerinnen und Schüler anhand eines Haufens durcheinandergeratener Teile aus mehreren Puzzles Strategien zum Sortieren dieser Teile entwickeln. Eine Gruppe von etwa zehn Schülerinnen und Schülern erhält den Auftrag, aus einem Haufen ungeordneter Puzzleteile, die von vier verschiedenen Puzzles stammen, möglichst schnell vollständige Puzzles zusammenzusetzen. Die für den Auftrag benötigten Puzzles lassen sich in vielen Supermärkten günstig kaufen. Als geeignet haben sich Puzzles mit 50 bis 60 Teilen erwiesen. Die Schülerinnen und Schüler kennen vorab weder die Anzahl Puzzles noch die Bildmotive der Puzzles.

Bevor das eigentliche Zusammensetzen der Puzzles beginnt, müssen sich die Schülerinnen und Schüler überlegen, wie sie am effizientesten vorgehen. So können etwa die Teile auf verschiedene Haufen aufgeteilt und die Eckteile aussortiert werden. Die Anzahl der Ecken gibt dann Aufschluss über die Anzahl Puzzles. Anschließend können die Teile entsprechend ihren Eigenschaften (Farben, Rückseitenmuster, Beschaffenheit, Dicke usw.) weiter gruppiert werden. Innerhalb dieser Gruppen können dann Randteile aussortiert, alle blauen „Himmelteile“ zusammengeführt oder ähnliche Methoden zum Aufteilen der Arbeit genutzt werden.

Beispiel 2: Algorithmus für den kürzesten Weg

Das Finden des kürzesten Weges vom Start zum Ziel einer Reiseroute ist ein wichtiger Bestandteil von Routenplanern. Abstrahiert geht es darum, in einem gerichteten, gewichteten Graphen den kürzesten Weg von einem Startknoten S zu einem Zielknoten Z zu finden.

Das bewährte Verfahren von Dijkstra löst diese Aufgabe effizient. Die Grundidee hinter dem Algorithmus: Um den kürzesten Weg vom Startknoten S zum Zielknoten Z zu bestimmen, berechnet man alle Wege und wählt den kürzesten aus. Dabei wird folgende Eigenschaft genutzt: Führt der kürzeste Weg von S nach Z über den Knoten P, so sind die Teilwege von S nach P und von P nach Z ebenfalls kürzeste Wege. Das Verfahren von Dijkstra ist ein gieriger Algorithmus („greedy algorithm“), bei dem in jedem Schritt die Entscheidung getroffen wird, die den aktuell größten Vorteil bringt.

Diese Beschreibung ist für Schüler häufig ebenso wenig eingängig wie eine symbolische Beschreibung in Form von Programm- oder Pseudocode wie die folgende:

```
procedure matrixpfs;
  var k,min,t: integer;
begin
  for k:=1 to V do begin
    val[k]:=-unseen; dad[k]:=0
  end;
  val[0]:=-(unseen+1);
  min:=1;
  repeat
    k:=min; val[k]:=-val[k]; min:=0;
    if val[k]=unseen then val[k]:=0;
    for t:=1 to V do
      if val[t]<0 then begin
        if (a[k,t]>0) and (val[t]<-priority) then begin
          val[t]:=-(priority); dad[t]:=k
        end;
        if val[t]>val[min] then min:=t;
      end
    end
  until min=0;
end;
```

Die Beschreibung des Algorithmus in Pseudocode hat zudem den großen Nachteil, dass zuerst eine formale Notation eingeführt werden muss. Die Lernenden müssen erst die Notation verstehen, bevor sie sich auf die Kernidee hinter dem Algorithmus konzentrieren können.

Anschaulicher wird die Beschreibung des Ablaufs durch eine Bildfolge in Kombination mit erläuterndem Text. Die in Abb. 20.1 dargestellte Bildfolge zeigt eine statische, ikonische Repräsentation eines dynamischen Ablaufs. Man könnte sie auch animiert ablaufen lassen, eine auf den ersten Blick überzeugende Variante.

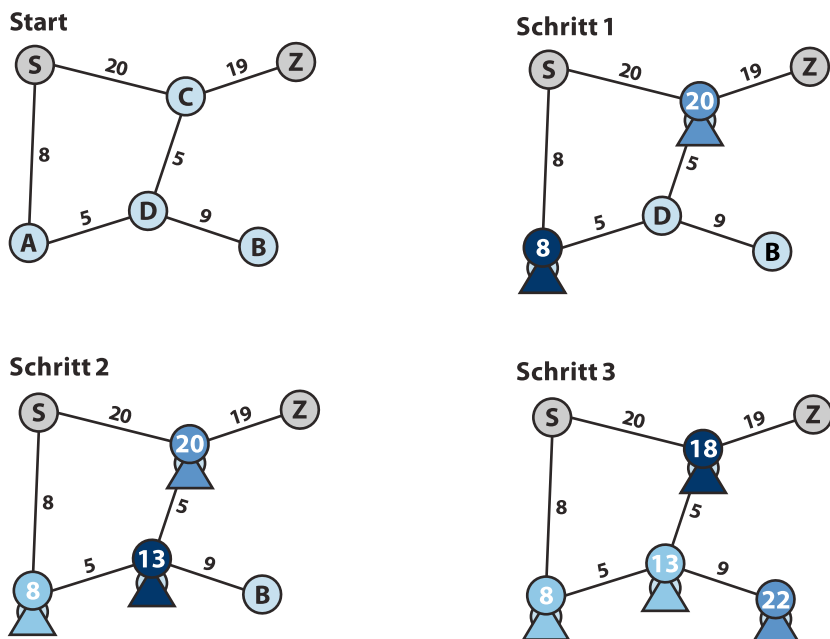


Abb. 20.1 Bildsequenz „Algorithmus kürzester Weg“

Dennoch kann eine statische Darstellung von Vorteil sein, um die Lernenden nicht kognitiv zu überfordern.

Eine enaktive Darstellung des Algorithmus kann auf dem Schulhof umgesetzt werden: Mit Kreide wird ein Straßennetz gezeichnet. Die Schüler starten im Ausgangspunkt, bewegen sich zu benachbarten Knoten und zählen dabei die zurückgelegten Schritte, die sie mit Kreide am jeweiligen Knoten notieren. Von den nun erreichten Knoten gehen die Schüler in alle möglichen Richtungen weiter und addieren laufend die zurückgelegten Schritte. Hierbei achten sie darauf, dass jeder Weg genau einmal abgeschritten wird. Trifft ein Schüler auf einen Knoten, der bereits eine kleinere Schrittzahl aufweist, übernimmt er diese kleinere Schrittzahl. Das Verfahren setzt genügend Schülerinnen und Schüler und einen nicht allzu großen Graphen voraus. Alternativ kann das Verfahren auch auf einem Tisch mit virtuellen Personen nachgestellt werden: Zur Distanzmessung dient Schnur, und Klebeband hilft beim Auslegen des Wegenetzes.

Das Beispiel der enaktiven Repräsentation eines Algorithmus zur Bestimmung des kürzesten Weges in einem Graphen zeigt einen Aspekt, der vielen enaktiven Repräsentationen gemeinsam ist: Die Lehrerin gibt die Lösung nicht vor, sondern ermöglicht den Lernenden, zu experimentieren und selbst Erkenntnisse zu gewinnen.

Beispiel 3: Farbräume und Farbtiefe in der Bildbearbeitung

In der Bildbearbeitung spielen Farbräume und Farbtiefe eine wichtige Rolle. Das Verwenden vieler Farben steigert die Qualität von Bildern, führt aber auch zu gro-

Abb. 20.2 Farbpaletten
enaktiv. (Idee: Beat Döbeli
Honegger)



ßen Bilddateien. Verwendet man nur wenige Farben, erhält man zwar kleinere Dateien, dafür leidet die Bildqualität.

Der Kompromiss zwischen guter Bildqualität und Dateigröße lässt sich wie folgt zeigen: Die Schüler erhalten alle dasselbe Bild sowie ein kariertes Blatt Papier mit verschiedenen großen Karomustern. Eine Gruppe erhält eine Schachtel mit einer großen Anzahl an Farbstiften, eine zweite Gruppe eine kleinere Schachtel mit rund einem Dutzend Farbstiften. Eine dritte Gruppe erhält genau drei Farbstifte (siehe Abb. 20.2). Nun soll das Bild als Rasterbild Pixel für Pixel möglichst exakt auf dem karierten Blatt Papier nachgezeichnet werden. Schnell zeigen sich so die Vor- und Nachteile der verschiedenen Farbtiefen. Darf eine vierte Gruppe aus der größten Farbstiftschachtel eine Auswahl von zwölf Stiften treffen, die sie für das Nachzeichnen des Bildes benötigt, wird zudem das Prinzip der Farbpalette enaktiv erfahrbar. Die Verwendung von kariertem Papier mit variierenden Karomustern zeigt zusätzlich den Einfluss der Auflösung auf die Bildqualität und die Bildgröße.

Beispiel 4: Polymorphismus in der objektorientierten Programmierung

Polymorphismus ist ein abstrakter Begriff in der objektorientierten Programmierung. In Lehrmitteln finden sich beispielsweise Erklärungen des Begriffs anhand von verschiedenen Fahrzeugtypen: Ausgehend von einer Basisklasse „Motorfahrzeuge“ wird eine Klasse „Sportwagen“ abgeleitet. Die Klasse „Sportwagen“ besitzt keine eigene Methode „beschleunigen“, sondern erbt die Methode der Basis-klasse. Ein Problem entsteht bei einem speziellen Typ von Sportwagen: „Raketwagen“ haben die gleichen Eigenschaften wie andere Sportwagen, aber die Methode „beschleunigen“ basiert auf einem anderen Funktionsprinzip. Benötigt werden zwei unterschiedliche Methoden „beschleunigen“. In einer objektorientierten Programmiersprache wird zur Laufzeit die Methode aufgerufen, die der Klasse eines Objekts zugeordnet ist. Dieses Verhalten bezeichnet man als Polymorphismus. Polymorphismus bezeichnet das Überlagern von Methodennamen: In einer Klassenhierarchie gibt es unterschiedliche Implementationen einer Methode, die

immer denselben Namen trägt. Je nachdem, welcher Klasse ein Objekt zur Laufzeit zugeordnet ist, verhält sich das Objekt anders – es ist polymorph.

Diese textuelle Beschreibung bewegt sich auf der symbolischen Repräsentationsebene, auch wenn darin versucht wird, einen Alltagsbezug zu konkreten Objekten in Form von Autos herzustellen. Um das Prinzip greifbarer zu machen, hilft eine körperlich nachvollziehbare Analogie: Alle stehen auf. Dann kommt die Anweisung: „Bitte machen Sie alle Ihren linken Schuh auf und wieder zu.“ Alle verstehen die Aufgabe, doch die Ausführung sieht unterschiedlich aus: Manche öffnen Klettverschlüsse, andere Reißverschlüsse, wieder andere binden Schnürsenkel auf unterschiedliche Weise. Das Verhalten ist verschieden, obwohl die Aufforderung identisch war. Genau das ist Polymorphismus: gleicher Befehl, unterschiedliche Ausführung – je nachdem, wie der Adressat intern funktioniert. Und oft ist dabei sogar ein Lächeln im Spiel.

Literatur

Bruner, J. S. (1988). *Studien zur kognitiven Entwicklung*. Klett-Cotta.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: In der Einführungswoche lässt K. ausgemusterte Rechner auseinander-schrauben. Die Schülerinnen schätzen diese Tätigkeit, denn sie erhalten einen konkreten Einblick in das Innenleben eines Computers: Die wichtigsten Komponenten wie Prozessor, Hauptspeicher, Festplatte oder Lüfter werden sichtbar und verständlicher als auf einer schematischen Abbildung. Leider lässt sich im Informatikunterricht nur selten etwas physisch aufschrauben und betrachten. Bei der Einführung von Internetprotokollen kann K. kein Netzkabel aufschneiden, und selbst visualisierte Signale bleiben lediglich Bitfolgen. Auch Algorithmen sind oft abstrakt und hinter Codezeilen verborgen. Die während der Ausführung ablaufenden Prozesse wie Zustandsänderungen, Schleifeniteration oder rekursive Aufrufe sind nicht unmittelbar sichtbar.

Informatik ist geprägt von Unsichtbarem. Nur wenige Inhalte lassen sich ohne Hilfsmittel direkt beobachten und untersuchen.

In der Informatik stehen insbesondere drei Hilfsmittel zur Verfügung: Unsichtbare Symbole und Zustände werden durch Visualisierungen sichtbar, dynamische Abläufe können durch Animationen veranschaulicht werden und Simulationen bilden Prozesse in veränderbaren Modellen ab.

Visualisierungen Computer arbeiten mit Symbolen. Auf der untersten Ebene der Digitaltechnik existieren genau zwei Symbole, die Zustände 0 und 1, repräsentiert durch Bits. Niemand kann die Bits, wenn sie zum Beispiel über die Internetanbindung eines Computers gesendet werden, direkt beobachten, ihre Bedeutung sofort erfassen oder sie sinnvoll manipulieren. Die in Bits gespeicherten Informationen werden für Menschen erst dann verständlich, wenn sie zu größeren Bedeutungseinheiten zusammengefasst werden. Dazu sind Visualisierungen, insbesondere in

interaktiver Form, geeignete Hilfsmittel. So können beispielsweise Netzwerkschichten oder Datenstrukturen anschaulich dargestellt werden.

Animationen Dynamische Abläufe können in Form einer Abfolge von Bildern veranschaulicht werden. Dabei sind der Inhalt und der zeitliche Ablauf festgelegt und das Verhalten kann nicht durch Eingaben der Lernenden beeinflusst werden. Animationen können Lernende unter Umständen kognitiv überfordern. In einer Animation müssen neben der bildlichen Darstellung des Sachverhaltes auch der zeitliche Ablauf verarbeitet und relevante, kohärente Information herauskristallisiert werden. Beispiele hierfür sind Animationen von Algorithmen oder Prozesse in Rechnerarchitekturen.

Simulationen Simulationen bilden dynamische Abläufe in einem Modell nach. Lernende können diese Modelle aktiv erkunden, indem sie Parametereinstellungen oder Komponenten des Modells variieren. Simulationen werden eingesetzt, um Systeme, Prozesse oder Experimente interaktiv zu erkunden und verschiedene Szenarien durchzuspielen. Auf diese Weise können etwa der Netzwerkverkehr unter verschiedenen Bedingungen oder das Verhalten endlicher Automaten anschaulich dargestellt und erforscht werden.

Die Gestaltung effektiver Visualisierungen, Animationen und Simulationen ist anspruchsvoll, da sie weder überladen noch zu vereinfacht sein dürfen. Sorgfältige didaktische Überlegungen sind erforderlich, um Konzepte wie die Schichten des TCP/IP-Protokollstacks, Entity-Relationship-Diagramme oder Graphenalgorithmen für Schülerinnen und Schüler verständlich darzustellen.

Visualisierung, Animation und Simulation gewinnen besonders an Wert, wenn sie eine aktive Interaktion zwischen Mensch und Maschine ermöglichen. Ihr volles Potenzial entfalten diese Hilfsmittel vor allem dann, wenn Lernende die Möglichkeit haben, selbst Einfluss zu nehmen. Diese Interaktionsmöglichkeiten lassen sich anhand der Stufen der Mensch-Computer-Interaktion nach Schulmeister (2003) klassifizieren (siehe Abb. 21.1).

Je höher die Interaktionsstufe, desto mehr Möglichkeiten haben Lernende, sich aktiv mit dem Lerngegenstand auseinanderzusetzen. Bei Visualisierungen können Lernende beispielsweise Ansichten wechseln, Ebenen ein- und ausblenden oder Detailgrade anpassen, um komplexe Strukturen aus verschiedenen Perspektiven zu betrachten. Interaktive Animationen erlauben es, den Ablauf zu steuern – zu pausieren, zu beschleunigen oder zu verlangsamen – und so das Tempo an individuelle Bedürfnisse anzupassen. Simulationen bieten den größten Spielraum für aktives Eingreifen: Hier können Lernende Parameter verändern, Szenarien durchspielen und unmittelbar die Auswirkungen ihrer Handlungen beobachten.

Für den Einsatz von Visualisierungen, Animationen und Simulationen stehen in der Regel drei Quellen zur Verfügung:

Standardwerkzeuge nutzen Standardwerkzeuge beinhalten oft nützliche Visualisierungsfunktionen, die im Unterricht eingesetzt werden können, zum Beispiel die Anzeige des Quelltextes einer Webseite im Browser oder die Visualisierung von Daten durch Diagramme.

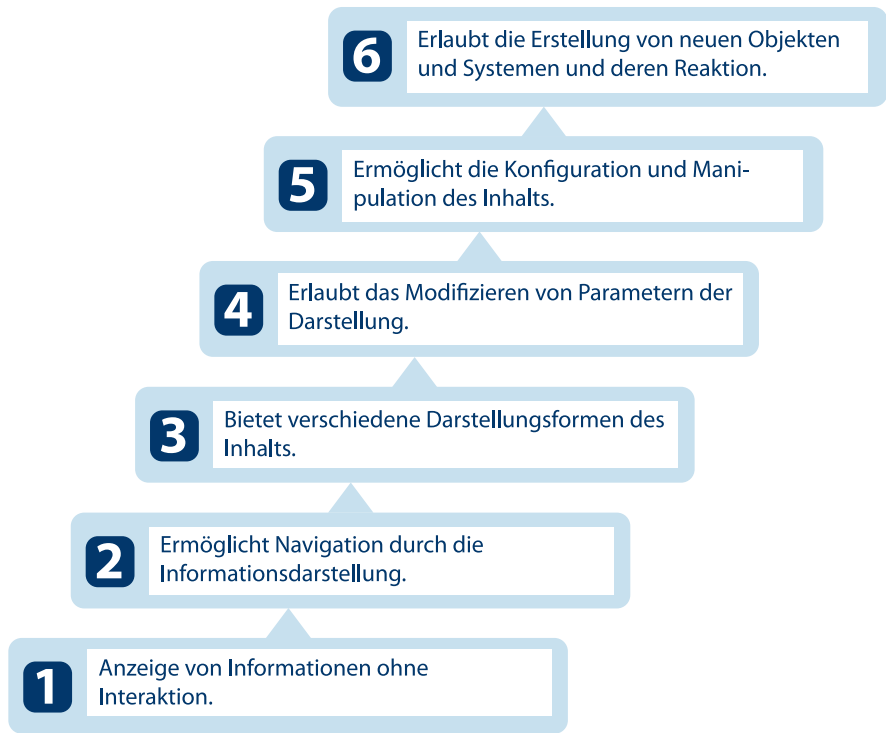


Abb. 21.1 Stufen der Mensch-Computer-Interaktion

Bestehendes suchen Zu vielen Themengebieten werden Visualisierungen, Animationen und Simulationen kostenfrei im Internet angeboten und auf speziell für Unterrichtszwecke ausgerichteten Portalen vorgestellt.

Selbst entwickeln Eigenentwicklungen sollten aufgrund des oft beträchtlichen Zeit- und Ressourcenaufwands sorgfältig abgewogen werden. Sie können besonders dann lohnenswert sein, wenn die Entwicklungen langfristig und von mehreren Lehrkräften genutzt werden oder wenn für spezifische Lernziele keine geeigneten bestehenden Ressourcen verfügbar sind.

Die unsichtbaren Informationen und Vorgänge im Computer können mit geeigneten Hilfsmitteln sichtbar gemacht werden. Visualisierungswerkzeuge ermöglichen es, statische Sachverhalte konkret zu beobachten. Animationen veranschaulichen dynamische Abläufe in einer festgelegten Sequenz. Simulationen helfen, Kenngrößen und Komponenten eines informatischen Systems oder Prozesses interaktiv zu untersuchen.

Beispiel 1: Netzverkehr – Unsichtbares sichtbar machen

K. setzt ein Werkzeug zur Visualisierung von Netzverkehr ein. Das Werkzeug kann die auf der Netzverbindung eines Rechners ausgehenden und ankommenden Daten „mithören“ und übersichtlich darstellen. Es verfügt über die Fähigkeit, viele Protokolle zu interpretieren und deren Daten anschaulich aufzubereiten. Damit steht den Lernenden ein Hilfsmittel zur Verfügung, mit dem sie das in der Theorie Gelernte im realen Umfeld beobachten und untersuchen können. Sie brauchen sich nicht mit der genauen Syntax der verschiedenen Datenpakete zu beschäftigen. Stattdessen können sie sich auf das Laufzeitverhalten konzentrieren, das heißt auf die zwischen Sender und Empfänger ausgetauschten Meldungen.

Ein weiterer Vorteil beim Einsatz von Netzwerkanalyse-Werkzeugen ist, dass die Lehrerin den zu untersuchenden Netzverkehr vor dem Unterricht in aller Ruhe produzieren und aufzeichnen kann. Später im Unterricht verteilt sie eine Datei mit den Aufzeichnungen: Die Lernenden erhalten eine wohldefinierte Ausgangslage für ihre Untersuchungen.

Beispiel 2: Betriebssysteme – Blick unter die „Motorhaube“ mit Standardwerkzeugen

Die meisten Betriebssysteme stellen integrierte Werkzeuge zur Verfügung, die einen Einblick in die sonst verborgenen Abläufe des Systems gewähren und Zustandsinformationen der wichtigsten Komponenten in Echtzeit anzeigen. Zum Beispiel lassen sich die CPU-Belastung und Festplattenzugriffe gut beobachten: Im Ruhezustand ist das System unbelastet; die Anzeige für die CPU-Auslastung zeigt (nahezu) 0 %. Wird dann eine größere Datei auf der Festplatte von einem Ort an einen anderen kopiert, erkennt man, dass die CPU-Belastung ansteigt und Datenblöcke von der Festplatte ein- und ausgelesen werden. Lehrkräfte können anhand dieser Visualisierung auf verschiedene andere Werte und damit auf das Systemverhalten eingehen.

Beispiel 3: K-Nächste-Nachbarn-Algorithmus

Der K-Nächste-Nachbarn-Algorithmus (KNN) zählt zu den populärsten und zugleich einfachsten Klassifikationsalgorithmen im heutigen maschinellen Lernen. Er nutzt die Nähe zwischen Datenpunkten, um Klassifizierungen oder Vorhersagen über deren Gruppenzugehörigkeit zu treffen.

Online verfügbare Simulationen ermöglichen es, mit unterschiedlichen K-Werten (Anzahl zu prüfender Nachbarn bei der Klassifizierung eines Datenpunktes) zu experimentieren und sofort die Auswirkung auf die Klassifikation zu beobachten. Solche Lernumgebungen sind besonders motivierend, wenn sie den Algorithmus anhand realer Daten veranschaulichen.

Neben dem KNN-Algorithmus lassen sich auch andere Clusteringalgorithmen, wie etwa der K-Means-Algorithmus für das unüberwachte Lernen, in ähnlicher Weise thematisieren. Darüber hinaus eignen sich viele Algorithmen aus der Graphentheorie oder der theoretischen Informatik, beispielsweise endliche Automaten oder reguläre Ausdrücke, hervorragend für eine Erkundung mittels Simula-

tionen. Abhängig von der Schulstufe können die Simulationen der Algorithmen auch von den Lernenden selbst programmiert werden.

Beispiel 4: Simulationen für Bevölkerungsentwicklungen und Klimadaten

Data Science hat im Informatikunterricht auf Sekundarstufe II eine wachsende Bedeutung, da sie Schülerinnen und Schüler befähigt, datenbasierte Zusammenhänge zu erkennen, kritisch zu hinterfragen und fundierte Entscheidungen zu treffen. Sie lernen, Daten zu erheben, aufzubereiten, zu analysieren und zu visualisieren. Darüber hinaus fördert Data Science das Verständnis zentraler Konzepte wie Algorithmen, Modellbildung und maschinelles Lernen und schafft so eine Brücke zwischen Informatik, Mathematik und realen Anwendungen.

In der Sekundarstufe II können zum Beispiel auf realen Datenbeständen basierende Simulationen eingesetzt werden, um ein Verständnis für die Dynamik von Bevölkerungsentwicklungen und Klimaveränderungen zu fördern. Die Schülerinnen und Schüler nutzen interaktive Modelle zur Bevölkerungsentwicklung, in denen sie Parameter wie Geburtenrate, Sterberate und Migration anpassen können, sowie eine Klimasimulation, die Faktoren wie CO₂-Emissionen und Temperaturveränderungen berücksichtigt. Die Aufgabe besteht darin, durch eigenständiges Experimentieren mögliche Zusammenhänge zwischen Bevölkerungsentwicklung und Klimaveränderungen herzustellen. In einer Abschlussreflexion setzen sich die Schülerinnen und Schüler kritisch mit den Möglichkeiten und Grenzen von Simulationen auseinander, insbesondere der Bedeutung des zugrunde liegenden Modells und der Qualität der genutzten Daten.

Beispiel 5: Simulation logischer Schaltungen und Modellrechner

Logische Schaltungen mithilfe von AND-, OR- und NOT-Gattern sind zentral in der Elektronik und für die Funktionsweise von Computern. Durch den Einsatz von Simulationswerkzeugen können Lernende die Grundlagen der digitalen Logik interaktiv erkunden und experimentell erfahren. Viele Tools ermöglichen es, Schaltungen zu entwerfen, zu testen und ihr Verhalten unter verschiedenen Bedingungen zu beobachten, ohne dass physische Hardware erforderlich ist.

Für den Informatikunterricht findet sich im Internet auch eine Reihe von Simulationen für einfache Von-Neumann-Computer. Die Simulationen visualisieren in der Regel die Funktion von Speicher, arithmetisch-logischer Einheit (ALU) und Steuereinheit. Maschinenprogramme können dabei mit einer benutzerfreundlichen grafischen Oberfläche erstellt und bearbeitet und Schritt für Schritt auf der virtuellen Maschine ausgeführt werden.

Literatur

Schulmeister, R. (2003). Taxonomy of multimedia component interactivity. A contribution to the current metadata debate. *Studies in Communication Sciences*, 3(1), 61–80.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Schüler T. schüttelt den Kopf: „Ich kann für meine Webseite kein dreispaltiges Layout erstellen, ohne dafür Tabellen zu verwenden! Laut W3C soll ich Tabellen aber nicht für Layoutzwecke missbrauchen. Egal wie ich meine Elemente verschachtle, es funktioniert einfach nicht!“ Der Lehrer erkennt das Problem sofort: Der Schüler hat versucht, einzelne Komponenten zu einer vollständigen Webseite zusammenzufügen, die nicht zueinander passen. Ihm fehlt das Verständnis für die Struktur einer anspruchsvolleren Webseite. Der Lehrer rät ihm, eine bestehende Webseite zu analysieren. „Aber das habe ich schon versucht“, moniert der Schüler, „ich habe die Webseite einfach nicht verstanden!“ Dem Lehrer wird klar: Auch das Lesen will gelernt sein.

Das Erstellen neuer Inhalte ist eine kognitiv anspruchsvolle Aufgabe. Beschäftigt man sich nicht zuerst mit dem Lesen und Verstehen bestehender Inhalte, neigen die Lernenden dazu, in ein Muster von Versuch und Irrtum zu verfallen: Sie fügen schrittweise neue Elemente hinzu, probieren aus, überarbeiten, testen erneut, verwerfen und beginnen von vorne. Die Trial-and-Error-Methode kann bei kleinen Problemen zum Erfolg führen, versagt aber bei anspruchsvollen Aufgaben.

In der Informatik erstellen die Lernenden oft selbst Inhalte, zum Beispiel eine Webseite, ein Programm, eine Datenbankabfrage, eine Suchanfrage oder einen Prompt für einen KI-Assistenten. Im Unterricht fangen sie meistens sofort damit an, ohne zuerst bestehende Beispiele anzuschauen. Lehrbücher erklären beispielsweise einzelne HTML-Tags wie `h1`, `p` oder `img`. Sie erwarten, dass die Lernenden, nachdem sie alle wichtigen Tags kennengelernt haben, umfangreiche Webseiten erstellen können, ohne vorher ein gutes Beispiel zu betrachten. Die Schülerinnen und Schüler lernen das „Schreiben“, bevor sie das „Lesen“ gelernt haben.

In etablierten Fächern ist es selbstverständlich, dass das Lesen vor dem Schreiben gelernt wird. Kinder lernen zwar früh das Schreiben einzelner Buchstaben, um die feinmotorischen Bewegungen zu üben. Das Schreiben ganzer Texte folgt aber erst später. Auch im Fremdsprachenunterricht kommt das Lesen vor dem Schreiben. Im Physikunterricht löst die Lehrerin viele Aufgaben vor, bevor sie von den Schülern das selbstständige Lösen von Aufgaben erwartet. Diese Reihenfolge berücksichtigt die Stufen kognitiver Lernziele, wie sie Blooms Taxonomie (1956) definiert: Kenntnisse und Verständnis gehen als Ebenen der Anwendung voraus; die Ebenen Analyse, Synthese und Beurteilung folgen auf den höheren Stufen.

Im Informatikunterricht wird dem Prinzip „Zuerst lesen, dann schreiben“ oft zu wenig Bedeutung geschenkt, insbesondere im Programmierunterricht. Die Lernenden erstellen von Beginn an kleine Programme; bestehende Programme werden nur selten analysiert. Studien belegen die These, dass das Lesen und Analysieren von Code vor dem Schreiben eigener Programme den Lernprozess erheblich verbessert. Das Lesen hilft, Konzepte zu verstehen, Fehler zu reduzieren und effektiver zu programmieren. Für eine vertiefte Auseinandersetzung empfehlen sich die Ausführungen in „On the Cognitive Development of the Novice Programmer“ von Lister (2020).

Das PRIMM-Modell (Predict, Run, Investigate, Modify, Make) unterstützt diesen Ansatz, indem es Lernende dazu anleitet, zunächst bestehenden Code vorherzusagen (Predict), auszuführen (Run), zu untersuchen (Investigate) und zu modifizieren (Modify), bevor sie eigenen Code schreiben (Make). Untersuchungen im Bereich der Kognitionspsychologie legen ebenfalls nahe, dass das passive Verarbeiten von Informationen, wie zum Beispiel das Lesen von Code, eine wichtige Vorstufe für das aktive Anwenden ist. Durch das Verständnis von Beispielen können Lernende mentale Modelle entwickeln, die für das Problemlösen notwendig sind. Zudem haben sie so auch eine Reihe von Beispielen zur Hand, auf die sie sich abstützen können.

Auch in der Praxis verbringen Entwickler viel Zeit damit, bestehenden Code zu lesen und zu verstehen, um Fehler zu beheben oder Funktionen zu erweitern. Außerdem werden Entwicklerinnen regelmäßig mit ihnen unbekannten Technologien konfrontiert und müssen sich dabei Quelltexte erschließen. Somit entfällt ein erheblicher Teil der Softwareentwicklungskosten auf das Studium von Quelltexten. Daher ist das frühe Training dieser Fähigkeiten im Bildungsprozess von großem Vorteil.

Trotz der großen Bedeutung ist „Program Comprehension“ selten ein Thema in Ausbildungen oder Literatur zur Softwareentwicklung. Entsprechend finden sich nur wenige Hinweise darauf, wie das Lesen von Programmen unterrichtet werden kann. Naheliegend sind drei Ansätze: Der erste Ansatz ist „bottom-up“, indem zunächst kleine Fragmente studiert und zu immer größeren semantischen Einheiten ausgebaut werden. Der zweite Ansatz ist „top-down“, indem man von der Grobstruktur des Programms ausgehend immer tiefer in die Details einsteigt. Der dritte Ansatz ist die opportunistische Mischung der beiden ersten Ansätze, die je nach Situation kombiniert werden.

Bevor Lernende im Informatikunterricht eigene Inhalte erstellen, kann es ihnen helfen, bestehende Inhalte zu lesen und zu analysieren.

Beispiel 1: Multimediale Gestaltungsprinzipien

Die Fähigkeit, Informationen multimedial aufzubereiten, ist in vielen Bereichen unverzichtbar: bei der Gestaltung von Textdokumenten oder Vortragsfolien, der Erstellung von Infografiken, dem Aufbau ansprechender Webseiten oder der Gestaltung von Benutzerschnittstellen. Eine kompetente Gestaltung von multimedialen Produkten erfordert nicht nur technisches Wissen, sondern auch ein grundlegendes Verständnis von gestalterischen Prinzipien. In seinem Standardwerk *Multimedia Learning* geht Mayer (2009) der Frage nach, ob Menschen nachhaltiger lernen, wenn Inhalte nicht nur verbal, sondern auch visuell vermittelt werden. Auf Basis zahlreicher experimenteller Studien formuliert er zwölf Prinzipien, die erklären, wie Menschen mithilfe von Texten und Bildern Informationen verarbeiten. Diese Leitlinien bieten eine Grundlage für die Gestaltung multimedialer Produkte. Im Unterricht lohnt es sich, vergleichende Beispiele kritisch zu untersuchen, also zu „lesen“: Indem man zum Beispiel gelungene und weniger gelungene Präsentationsfolien einander gegenüberstellt, können selbstständig konkrete Gestaltungskriterien herausgearbeitet werden.

Beispiel 2: HTML und CSS lesen

Technisch sauberes Webseitendesign bedingt ein Verständnis für die klare Trennung von Inhalt, Struktur und Layout. Die HTML-Tags definieren sowohl die Struktur als auch den Inhalt einer Seite. Die einzelnen Elemente dieser Struktur werden mit CSS-Anweisungen formatiert. Die Versuchung ist groß, Webseiten mit WYSIWYG-Editoren zu erstellen. Ein wirklich fundiertes Verständnis für den Aufbau einer Webseite erwirbt man aber nur durch eine vertiefte Auseinandersetzung mit HTML und CSS, zum Beispiel anhand des Quelltexts gelungener Websites.

Viele Webseiten sind unnötig kompliziert und eignen sich nicht als Musterbeispiele im Unterricht. CSS-Dateien beinhalten meist viele kleine, schwer verständliche und häufig nicht dokumentierte Tricks, um die Eigenheiten der diversen Browser zu berücksichtigen. Es gibt aber einzelne Websites, die sich explizit der technisch sauberen Umsetzung von grafisch ansprechenden Designs widmen: Ein Beispiel ist das Projekt CSS Zen Garden (csszengarden.com). Die Website basiert auf einer einzigen HTML-Seite, für deren Darstellung der Betrachter aus einer Vielzahl von CSS-Dateien auswählen kann. Ähnliche Beispiele finden sich im Projekt Style Stage (stylestage.dev).

Beispiel 3: Programmcode lesen

Open-Source-Projekte sind eine mögliche Quelle für das Studium von Programmcodes. Für den Unterricht geeignet sind bei Open-Source-Projekten vor allem Programmteile, die mehr oder weniger losgelöst verständlich sind, die also gelesen werden können, ohne sich mit viel Aufwand in die gesamte Architektur eines Sys-

tems einarbeiten zu müssen. Die Auswahl reicht von kleinen Tools bis zu umfangreichen Systemen. GitHub und Apache bieten eine besonders reichhaltige Auswahl von bekannten Open-Source-Projekten.

Wir betrachten als konkretes Beispiel die Implementation einer Methode für die Berechnung des Absolutwertes von komplexen Zahlen. Mathematisch ist der Absolutwert einer komplexen Zahl z mit Realteil x und Imaginärteil y definiert als:

$$|z| = \sqrt{x^2 + y^2}$$

In den meisten im Internet verfügbaren Java-Klassen wird die Methode direkt nach dieser Definition implementiert:

```
public double abs() {
    return Math.sqrt(x*x+y*y);
}
```

Anders sieht es bei der Implementierung in Apache Commons aus: Im Sourcecode findet sich eine wesentlich umständlichere Implementation für die Berechnung des Absolutwertes (hier gekürzt um die Überprüfung von Sonderfällen):

```
public double abs() {
    if (Math.abs(x) < Math.abs(y)) {
        if (y==0.0) {
            return 0.0;
        }
        double q=x/y;
        return (Math.abs(y) * Math.sqrt(1+q*q));
    }
    else {
        if (x==0.0) {
            return 0.0;
        }
        double q=y/x;
        return (Math.abs(x) * Math.sqrt(1+q*q));
    }
}
```

Wird diese Implementation übersetzt in einen mathematischen Ausdruck, liest sich die Berechnung wie folgt:

$$|x| \geq |y| : |z| = |x| \sqrt{1 + \left(\frac{y}{x}\right)^2}$$

$$|x| < |y| : |z| = |y| \sqrt{1 + \left(\frac{x}{y}\right)^2}$$

Das Beispiel zeigt, wie durch das Studium von fremdem Code Grundlegendes gelernt werden kann: Die nahe liegende Implementation mit `Math.sqrt(x*x + y*y)` versagt für einen großen Teil des positiven Wertebereichs von x und y . Die Quadrierung führt für x und y größer als $\sqrt{\text{MAX DOUBLE}}$ zu einem Überlauf. Die umständliche Implementierung umgeht dieses Problem mithilfe des Quotienten q , der immer kleiner oder gleich 1 ist. Doch auch die umständlichere Lösung ist nicht perfekt: Der Algorithmus verhindert zwar den Überlauf, ist aber nicht sehr effizient und nicht immer numerisch stabil.

Obiges Beispiel zeigt, dass die unscheinbar anmutende Aufgabe „Berechnung des Absolutwertes von komplexen Zahlen“ zu spannenden und grundlegenden Themen der Numerik wie Effizienz, numerische Stabilität und Überlauf führt.

Literatur

- Bloom, B. S. (1956). *Taxonomy of educational objectives*. Longmans.
- Lister, R. (2020). On the cognitive development of the novice programmer: And the development of a computing education researcher. In *ACM Proceedings of the 9th Computer Science Education Research Conference*.
- Mayer, R. E. (2009). *Multimedia learning*. Cambridge University Press.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Teil VI

Unterrichtsdurchführung

Einstieg: Lehrer M. plant eine Unterrichtseinheit zur Einführung in Künstliche Intelligenz und maschinelles Lernen. Neben grundlegenden Konzepten möchte er den Schülerinnen und Schülern auch praktische Erfahrungen mit einfachen KI-Modellen ermöglichen, insbesondere im Bereich der Bildklassifizierung. Professionelle Machine-Learning-Plattformen scheiden aus, da sie oft umfangreiche Programmierkenntnisse und leistungsfähige Hardware erfordern. Zudem ist ihr Funktionsumfang so groß, dass die Einarbeitung in der zur Verfügung stehenden Zeit für seine Klasse kaum möglich ist. Verschiedene Onlineplattformen bieten zwar vereinfachte KI-Experimente an, enthalten aber Werbung oder sind in ihrer kostenlosen Version eingeschränkt. Die meisten erfordern zudem eine Benutzerregistrierung, was datenschutzrechtliche Fragen aufwirft und zusätzlichen administrativen Aufwand bedeutet. Andererseits ermöglichen Benutzerkonten den Lernenden, ihre Modelle und Ergebnisse zu speichern und über mehrere Stunden hinweg daran zu arbeiten – für Lehrer M. eine echte Zwickmühle!

Die Auswahl geeigneter digitaler Werkzeuge erfordert ein Abwägen verschiedener Kriterien, wobei Lehrkräfte praktikable und pädagogisch wertvolle Lösungen finden müssen.

Für den Informatikunterricht stehen zahlreiche digitale Werkzeuge zur Verfügung: Simulations- und Programmierumgebungen, Visualisierungstools, virtuelle Labore und Onlinekurse. In Themenfeldern wie Programmieren, Netzwerke, Datenbanken, Bildbearbeitung oder Kryptografie gibt es oft komplette Lernumgebungen.

Professionelle Tools sind für Lernende attraktiv, aber komplexer als nötig. Für den schulischen Einsatz eignen sich speziell entwickelte Lernumgebungen meist besser, da sie didaktisch klarer strukturiert und auf das Lernniveau abgestimmt sind. Bei der Auswahl sollte deshalb der pädagogische Mehrwert im Vordergrund stehen – zugleich sind auch organisatorische und technische Fragen zu berücksichtigen.

Das perfekte digitale Werkzeug gibt es nicht. Kein Tool erfüllt alle Erwartungen und Anforderungen zugleich. Die Wahl hängt immer vom konkreten Unterrichtskontext ab – vom Thema, vom Vorwissen der Lernenden, von der Unterrichtsform und von den technischen Rahmenbedingungen. Die nachfolgende Kriterienliste versteht sich als Orientierungshilfe: Sie kann dabei unterstützen, ein geeignetes Werkzeug für ein bestimmtes Fachthema zu finden oder zwischen mehreren Optionen abzuwägen. Die Liste erhebt keinen Anspruch auf Vollständigkeit. Je nach Situation gewinnen einzelne Kriterien an Gewicht, andere treten in den Hintergrund. Eine pragmatische Möglichkeit zur Vorauswahl besteht darin, die Kriterien einem KI-Assistenten vorzulegen. So lässt sich rasch abschätzen, ob ein bestimmtes Werkzeug grundsätzlich für den eigenen Unterricht geeignet sein könnte.

Der Fokus der Kriterien liegt auf fachspezifischen Lernwerkzeugen. Nicht betrachtet werden digitale Lehrmittel oder allgemeine Lernplattformen, die Instrumente und Ressourcen zur Unterstützung von Lehr- und Lernprozessen sowie zur Verwaltung von Unterrichtsmaterialien bereitstellen.

Die IT-Infrastruktur der Schule spielt eine wichtige Rolle. Findet der Unterricht in einem Informatikraum oder einem sogenannten Computerkabinett statt, lässt sich Software oft lokal durch die Lehrperson installieren, was eine einheitliche Arbeitsumgebung ermöglicht. Werden hingegen mobile Endgeräte der Schülerinnen und Schüler genutzt – womöglich mit unterschiedlichen Betriebssystemen –, empfiehlt es sich, auf browserbasierte Werkzeuge zu setzen, die plattformunabhängig funktionieren.

Didaktische und pädagogische Aspekte

Kompetenzförderung: Wichtigstes Auswahlkriterium ist, dass das Werkzeug den Lernprozess unterstützt und das Erreichen der angestrebten Lernziele fördert.

Kurze Einarbeitungszeit: Der Funktionsumfang und die Benutzerschnittstelle des Werkzeugs sind überschaubar und ermöglichen schnelle Erfolgserlebnisse.

Differenzierung: Durch Anpassbarkeit an verschiedene Lernniveaus unterstützt das digitale Werkzeug Individualisierung und Binnendifferenzierung. Mit dem Werkzeug können sowohl einfache Aufgaben als auch anspruchsvolle Aufgaben mit offenem Ende bearbeitet werden.

Kooperative Bearbeitung: Das Werkzeug ermöglicht die kooperative Bearbeitung durch die Lernenden und fördert die Kommunikation unter den Nutzern.

Feedback: Das Werkzeug bietet automatische Rückmeldungen und personalisiertes Feedback. Lehrkräfte können den Lernstand verfolgen und die Lernenden gezielt unterstützen.

Generalisierung: Besonders geeignet sind Werkzeuge, die in unterschiedlichen Themenbereichen und auf verschiedenen Altersstufen genutzt werden können. Erfahrungen mit dem Werkzeug sollen den Lernenden zudem den Übergang zu weiteren, ähnlichen Werkzeugen erleichtern.

Modi der Darstellung: Das Werkzeug ermöglicht eine flexible Anpassung der Darstellungsarten des Themas, sodass Nutzer je nach Bedarf zwischen verschiedenen Ansichten wechseln können.

Technische Aspekte

Kompatibilität: Das Werkzeug ist webbasiert oder lässt sich problemlos in die bestehende IT-Infrastruktur der Schule integrieren. Es funktioniert auf unterschiedlichen Betriebssystemen und Endgeräten und unterstützt damit sowohl den Einsatz im Unterricht als auch das selbstständige Arbeiten der Lernenden zu Hause.

Nachhaltigkeit: Das Werkzeug ist langfristig verfügbar und wird kontinuierlich aktualisiert. Es ermöglicht zudem den Import und Export kompatibler Formate, sodass Arbeitsergebnisse problemlos übernommen oder mit anderen Werkzeugen weiterbearbeitet werden können.

Integration: Das Werkzeug lässt sich in digitale Lernplattformen und Lehrmittel einbetten.

Kosten: Das Werkzeug ist kostenlos oder mit geringen Kosten nutzbar. Ein all-fälliger Aufwand für die Installation, Lizenzverwaltung, Wartung und Nutzung von Erweiterungen ist gering.

Registrierung: Das Werkzeug ist ohne Registrierung nutzbar. Falls die Nutzung eine Registrierung voraussetzt, können sich die Lernenden selbst registrieren. Falls vorhanden, können Single-Sign-on-Lösungen oder andere Identitätsdienste genutzt werden.

Sicherung der Arbeitsergebnisse: Das Werkzeug speichert automatisch und regelmäßig die Arbeit der Lernenden. Die Arbeitsergebnisse können sowohl lokal als auch online gesichert werden. Das Werkzeug bietet außerdem die Möglichkeit, die Arbeitsergebnisse gedruckt zu sichern.

Barrierefreiheit: Das Werkzeug ist barrierefrei oder zumindest barrierearm gestaltet, um möglichst allen Lehrenden und Lernenden Zugang zu ermöglichen. Durch das Angebot verschiedener Sprachversionen können zusätzlich auch Sprachbarrieren abgebaut werden.

Datenschutz: Das Werkzeug erlaubt die Nutzung ohne die Erfassung personenbezogener Daten, ist konform mit Datenschutzgesetzen und man behält die Rechte an den Arbeitsergebnissen.

Werbefreiheit: Das Werkzeug ist werbefrei und lenkt damit nicht vom Lernen ab.

Aspekte des Supports

Verbreitung: Das Werkzeug wird auch von anderen Schulen und Lehrkräften genutzt und erleichtert so den Austausch von Erfahrungen und Best Practices.

Begleitmaterial: Das Werkzeug bietet unterstützende Aufgabenstellungen, Übungen und Materialien zur eigenständigen Erarbeitung. Diese können entweder über Lehrmittel, online oder direkt im Werkzeug integriert bereitgestellt sein.

Unterstützung: Die Nutzung wird durch integrierte Hilfefunktionen, Tutorials und von der Community bereitgestellte Erklärvideos unterstützt.

Im Informatikunterricht bietet sich naturgemäß die Nutzung digitaler Werkzeuge an. Bei der Auswahl sollte dabei die Unterstützung des Lernprozesses im Vordergrund stehen. Daneben gibt es viele weitere Kriterien zu berücksichtigen. Perfekte Lösungen, die alle Anforderungen erfüllen, wird man in den meisten Fällen nicht finden.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Alles ist bestens geplant: In den vergangenen vier Lektionen wurden verschiedene Sortieralgorithmen besprochen, Insertion Sort und Quicksort wurden vertieft behandelt, erste Überlegungen zur Implementation angestellt. Nun sollen die Schülerinnen und Schüler ein Verfahren selbst implementieren und testen. Ein Programmgerüst steht auf der Lernplattform zum Download bereit, das die Erzeugung einer Zufallszahlenfolge zur Verfügung stellt. Auch die Ausgabemethode für die sortierte Zahlenfolge auf dem Bildschirm ist vorgegeben. Die Schülerinnen und Schüler sollen sich auf den Algorithmus konzentrieren können.

Doch bereits nach fünf Minuten meldet I., dass sie ihr Programm nicht editieren kann. V. kann nicht auf das Programmgerüst zugreifen. D. fragt, ob sie beim Quicksortalgorithmus das Pivotelement zufällig auswählen könne. Das Programm von B. läuft zwar zu Beginn korrekt, bricht dann aber mit einer Null-Pointer-Exception ab. M. wiederum hat Probleme mit der Verwendung der Suchen-und-Ersetzen-Funktion in der Eclipse-Entwicklungsumgebung. Alle rufen nach dem Lehrer, der von einem zur nächsten eilt, Stress pur!

Informatikerkräfte bemühen sich, den Schülerinnen und Schülern bei auftretenden Problemen Strategien zur eigenständigen Lösungsfindung zu vermitteln, was zeitaufwendig ist. Besonders bei Übungen am Computer oder in Makerspace-Umgebungen führt das zu Hektik für die Lehrkraft, da eine Vielzahl unterschiedlicher Probleme gleichzeitig auftreten kann. Diese Probleme werden durch die Heterogenität des Vorwissens der Lernenden sowie durch eine hohe Schülerzahl verstärkt und erfordern eine große Flexibilität und Belastbarkeit der Lehrkraft.

Im Informatikunterricht betreffen die Fragen der Lernenden häufig nicht nur reine Verständnisschwierigkeiten. Oft geht es um Detailinformationen, technische Besonderheiten der eingesetzten Software oder spezielles Produktwissen. Beson-

ders bei praktischen Übungen ist die Lehrkraft mit vielfältigen Fragestellungen und Problemen auf unterschiedlichen Ebenen konfrontiert und muss sich stets flexibel in neue Situationen einarbeiten, die sich nicht immer unmittelbar lösen lassen.

Wenn viele Lernende auf Unterstützung warten, ist es für die Lehrkraft verlockend, im Sinne einer Hotline Probleme schnell selbst zu lösen und so Zeit zu sparen. Dadurch erhalten Lernende, die aktiv um Hilfe bitten, mehr Unterstützung, während zurückhaltende Lernende übersehen werden. Hinzu kommt, dass viele der aufgeworfenen Fragen nicht ohne Weiteres zu beantworten sind: Es ist zum Beispiel sehr herausfordernd, einen Logikfehler in einem umfangreichen Programm während des laufenden Unterrichts zu finden. Daher muss sich die Lehrkraft Strategien überlegen, um sinnvoll mit diesen Problemen und Fragen umzugehen.

Im Folgenden stellen wir fünf Empfehlungen vor, die bei der Gestaltung praktischer Übungen am Computer hilfreich sind.

Empfehlung 1: Trennung von Theorie und Praxis

In Fächern wie Chemie wird neben der Theorievermittlung auch praktisch experimentiert und mit Werkzeugen gearbeitet – ähnlich wie im Informatikunterricht. Dabei sind Theorie und Experimente im Labor jedoch klar voneinander abgegrenzt. Im Informatikunterricht drängt sich – abgesehen von Spezialfällen wie Makerspace-Umgebungen oder Robotik-Labs – keine Trennung auf zwei Räume auf. Theorie und Praxis lassen sich dennoch in zweifacher Hinsicht trennen: räumlich durch eine geeignete Gestaltung des Unterrichtsraums, zeitlich durch eine klare Unterteilung in Theorie- und Praxiseinheiten.

Räumliche Trennung Der Unterrichtsraum kann so gestaltet werden, dass in einem Bereich die Theorie vermittelt wird, während ein anderer Bereich für die praktische Arbeit vorgesehen ist. Eine solche räumliche Trennung verhindert, dass Schülerinnen und Schüler während der Theorie durch den Computer abgelenkt werden und umgekehrt. Nutzen alle Lernenden ein mobiles Endgerät (z. B. Notebook), hat dieses Modell jedoch Grenzen. Umso wichtiger ist es dann, Theorie- und Praxisphasen organisatorisch deutlich voneinander zu trennen.

Zeitliche Trennung Die Trennung von Theorie und Praxis kann durch eine klare Strukturierung des Unterrichts erfolgen und auch in den Arbeitsmaterialien sichtbar gemacht werden. Während der Theorieeinheiten spricht die Lehrkraft die Klasse als Ganzes an, während der Praxisphasen hingegen nur noch einzelne Schülerinnen und Schüler. Diese Struktur ermöglicht es allen, in ihrem eigenen Tempo zu arbeiten, ohne durch Zwischenrufe oder Erklärungen unterbrochen zu werden.

Empfehlung 2: Werkzeuge und Objekte auseinanderhalten

In der Informatik wird oft mit Objekten irgendwelcher Art gearbeitet, zum Beispiel mit Bildern. Die Arbeit am Objekt geschieht mit Werkzeugen, zum Beispiel mit Bildbearbeitungssoftware. Auf beiden Ebenen – Objektebene und Werkzeugebene –

Tab. 24.1 Werkzeuge und Objekte, Konzepte und Umsetzung

	Konzepte	Umsetzung
Werkzeuge	Bilder bearbeiten mit Ebenen, Masken ...	Produkte wie Photoshop, Gimp, Canva ...
Objekte	Raster- und Vektorgrafiken, Komprimierung mit und ohne Informationsverlust ...	JPG, PNG, TIFF, SVG, EPS ...

Tab. 24.2 Werkzeuge und Objekte, Konzepte und Umsetzung in Malerberufen

	Konzepte	Umsetzung
Werkzeuge	Verschiedene Arbeitsvorgänge und die dabei verwendeten Materialien und Werkzeuge bei Vorarbeiten auf Holz und Holzwerkstoffen, mineralischen Untergründen, Metallen, Kunststoffen, alten Anstrichen und Beschichtungen, Geweben, Vliesen usw. kennen	Untergrundvorbereitung wie Schleifen, Entrosten, Isolieren, Neutralisieren, Aufhellen usw. Lasurarbeiten und Imprägnierungen auf mineralischen Untergründen und Holz sowie Beiz- und Lackierarbeiten
Objekte	Grundlagen der Farbenlehre, Mischen von Farben, Einfluss des Glanzgrades und der Untergrundstruktur auf den Farbton	Gängige Abbeiz-, Ablauge- und Neutralisationsmittel, Aufheller und Reinigungsmittel, Pigmente, Bindemittel, Lösungs- und Verdünnungsmittel, Additiv, Farben und Lacke kennen

existieren grundlegende Konzepte. Ein Bild kann beispielsweise als Bitmap oder mithilfe von Vektoren codiert sein, die Komprimierung verlustbehaftet oder verlustfrei erfolgen. Eine Bildbearbeitungssoftware verwendet Konzepte wie Ebenen, Masken oder Filter. Zudem gibt es auf beiden Ebenen konkrete Umsetzungen: GIF und JPEG sind Umsetzungen von Bitmapbildern. Bei den Werkzeugen variieren die Produkte in der Art und Weise, wie sie diese grundlegenden Konzepte umsetzen. Tab. 24.1 veranschaulicht Konzepte und ihre Umsetzung auf Objekt- und Werkzeugebene am Beispiel Bildverarbeitung.

Die Unterscheidung zwischen Objekten und Werkzeugen ist für die Lernenden eine anspruchsvolle Aufgabe, aber sie ist entscheidend, um auftretende Probleme richtig einzuordnen. Liegt die Problemursache in zu hoher Komprimierung des Bildes, im falschen Bildformat oder im unzureichenden Funktionsumfang des Werkzeuges?

Ein Blick über den Tellerrand der Informatik zeigt, dass die Unterscheidung zwischen Konzepten und deren Umsetzung in Produkten sowie die Trennung von Objekten und Werkzeugen nicht nur spezifisch für die Informatik ist. Ein anschauliches Beispiel, das thematisch in der Nähe der Bildbearbeitung liegt, findet sich in der Malerausbildung. Tab. 24.2 zeigt eine Auswahl der in Malerberufen wichtigen Konzepte auf Werkzeug- und Objektebene sowie das für die praktische Arbeit benötigte Produktwissen und die erforderlichen Fertigkeiten.

Empfehlung 3: Informatiklehrkräfte müssen nicht alles wissen

Lehrkräfte sollten sich nicht unter Druck setzen (lassen), auf jede Frage unmittelbar eine Antwort haben zu müssen. Im Gegenteil: Auftretende Fragen und Probleme können genutzt werden, um gemeinsam mit der Klasse über Lösungsstrategien und den Umgang mit Unsicherheiten zu reflektieren. Viele Fragen – insbesondere zur Bedienung der im Unterricht eingesetzten Werkzeuge – können die Lernenden selbstständig klären.

KI-Assistenten bieten oft wertvolle Unterstützung, etwa beim Debugging, beim Testen und Optimieren von Programmen oder beim Aufzeigen alternativer Lösungswege. Wichtig ist jedoch, dass Lernende diese Werkzeuge nicht lediglich zur automatischen Lösung von Aufgaben nutzen, sondern als Impulsgeber, um eigene Denkprozesse anzuregen. Dabei sollten sie lernen, KI-generierte Antworten kritisch zu hinterfragen – denn diese können fehlerhaft, unvollständig oder veraltet sein.

Auch klassische Quellen wie Hilfeseiten, Dokumentationen oder Diskussionsforen liefern oft hilfreiche Informationen. Die Aufgabe der Lehrperson besteht darin, die Lernenden dazu anzuleiten, Probleme einzugrenzen, geeignete Informationsquellen zu identifizieren und präzise Fragen zu formulieren. So wird der selbstständige und verantwortungsbewusste Umgang mit digitalen Hilfsmitteln gefördert.

Empfehlung 4: Supportwarteschlange

Auch die Lernenden können sich gegenseitig unterstützen. Dafür eignet sich beispielsweise eine sogenannte Supportwarteschlange: Die Lehrkraft beantwortet Fragen nicht mehr unmittelbar, sondern fordert die Lernenden auf, ihre Fragen zunächst auf einem Flipchart, an der Wandtafel oder in einem gemeinsamen Online-dokument einzutragen. Hier werden der Name der fragenden Person sowie eine möglichst präzise Problembeschreibung notiert. Das Vorgehen ähnelt der Internetplattform Stack Overflow für Softwareentwickler, die es Nutzern ermöglicht, Fragen zu stellen und zu beantworten. Dazu finden sich Empfehlungen zum Formulieren von Fragen, zum Beispiel:

- Fasse das Problem zusammen.
- Nenne das Ziel, das du zu erreichen versuchst.
- Beschreibe die erwarteten und tatsächlichen Ergebnisse.
- Füge alle Fehlermeldungen bei.
- Beschreibe, was du zur Lösung versucht hast, was dabei herauskam und warum es nicht weiterhilft.
- Zeige den kleinstmöglichen Code oder Handlungsablauf, mit dem andere das Problem reproduzieren können.

Diese Vorgehensweise bietet mehrere Vorteile: Zum einen nehmen sich die Lernenden mehr Zeit, um das Problem eigenständig oder gemeinsam mit anderen zu lösen, bevor sie es offiziell eintragen. Zum anderen erzwingt die schriftliche Beschreibung ein erneutes Nachdenken über die Fragestellung. Vage Anfragen wie „Bei mir geht es nicht!“ kommen somit kaum vor. Die Lehrkraft kann nun nach und nach die eingetragenen Fragen bearbeiten, ohne jemanden zu bevorzugen. Wieder-

holt sich ein Problem, können sich die Betroffenen direkt an jene Lernenden wenden, die dieses Problem bereits erfolgreich gelöst haben. Schließlich kann die Lehrkraft am Ende der Übungseinheit anhand der Liste die häufigsten Schwierigkeiten zusammenfassen und gemeinsam mit der Klasse diskutieren.

Empfehlung 5: Hände auf den Rücken

Bei der praktischen Arbeit am Computer ist es für Lehrkräfte verlockend, bei auftretenden Problemen – insbesondere, wenn ihnen die Lösung sofort klar ist – einfach selbst in die Tastatur der Lernenden zu greifen oder ein paar Mausklicks zu machen. So lässt sich etwa eine fehlende Programmzeile rasch hinzufügen, ohne dass die Lernenden aktiv in den Lösungsprozess eingebunden sind.

Hier kommt die Regel „Hände auf den Rücken!“ zum Tragen. Diese Vorgabe einzuhalten, fällt den meisten Lehrkräften anfangs schwer und bedarf etwas Übung. Ebenso ist es wichtig, die Schülerinnen und Schüler über den Zweck dieser Vorgehensweise zu informieren: Indem die Lehrkraft ihre Hände buchstäblich auf den Rücken nimmt, wird sie zur Mentorin, die mit gezielten Fragen auf Probleme hinweist, ohne selbst direkt einzugreifen.

Beispiele dafür sind Fragen wie: „Stimmt die Abbruchbedingung in Ihrer Schleife?“, „Können Sie die Fehlerquelle eingrenzen, indem Sie alle unwichtigen Teile des Programms entfernen?“ oder „Hatten Sie schon früher Probleme beim Drucken und erinnern Sie sich noch an die Ursachen?“. Auf diese Weise werden die Lernenden zum selbstständigen Nachdenken angeregt. Sie sollen über die Ursachen der Probleme nachdenken und eigene Lösungswege finden. Ihr Verständnis wird vertieft und ihre Fähigkeit zur selbstständigen Problemlösung nachhaltig gestärkt.

Aufgrund negativer Erfahrungen im praktischen Informatikunterricht neigen viele Lehrkräfte dazu, ihre Übungen bis ins kleinste Detail durchzuplanen. Dadurch werden die einzelnen Lernschritte immer kleiner, und der Unterricht verkommt zu einer reinen Instruktion. Nachhaltig ist ein solches Vorgehen nicht. Wenn in einer Datenbankschulung die Lehrkraft im Vorfeld alle wesentlichen Einstellungen vornimmt oder in einem Softwareentwicklungsprojekt die kritische Einbindung einer Schnittstelle eigenständig umsetzt und den Lernenden anschließend nur noch die fertige Lösung präsentiert, dann mag am Ende zwar Zufriedenheit herrschen, doch ein ähnliches Problem können die Lernenden später meist nicht eigenständig lösen.

Gerade für den Berufsalltag ist es entscheidend, dass die Lernenden Kompetenzen erwerben, um auftretende Probleme selbstständig zu erkennen, einzuordnen und zu bewältigen. Aus diesem Grund ist die Hilfe zur Selbsthilfe im Informatikunterricht von zentraler Bedeutung.

Insbesondere bei der praktischen Arbeit unterstützen die obigen fünf Empfehlungen das selbstständige Lernen der Schülerinnen und Schüler und entlasten gleichzeitig die Lehrkraft.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: Die Schülerinnen und Schüler nutzen ein Versionskontrollsystem für die gemeinsame Arbeit an einem Programmierprojekt. Das ist nicht ohne Tücken: Ein Schüler beklagt, dass er eine neue Datei hinzugefügt habe, die von den anderen nicht gesehen werden könne. Die Lehrerin überprüft die Ordnerstruktur auf dem Computer des Schülers und fragt nach einer Weile, ob er Ordner oder Dateien umbenannt habe. Der Schüler bejaht dies. Möglicherweise führte dies zu Synchronisationsproblemen. Nach etlichen Klicks funktioniert alles wieder, und alle können ihre Arbeit fortsetzen. Dennoch bleibt ein ungutes Gefühl zurück: Was war genau das Problem? Muss man damit rechnen, dass es erneut auftritt?

Im Informatikunterricht treten insbesondere bei praktischen Arbeiten immer wieder Probleme auf, deren Behebung Zeit in Anspruch nimmt und den Unterrichtsablauf stört. Als Lehrkraft ist man versucht, die Probleme so schnell wie möglich selbst zu lösen. So wird die Gelegenheit verpasst, den Lernenden Strategien zum Beheben von Fehlern zu vermitteln.

Testen, Fehlersuche und das anschließende Debugging gehören zu jedem Entwicklungsprozess. Gängige Schätzungen gehen bei Standardsoftware vor der Testphase von mehreren Dutzend Fehlern pro 1000 Zeilen Code aus. Die Konsequenzen dieser Zahlen sind erheblich: Ein modernes Betriebssystem umfasst etwa 100 Mio. Zeilen Code. Selbst bei einer niedrigen Fehlerdichte von nur 1 Fehler pro 1000 Zeilen ergeben sich insgesamt rund 100.000 Fehler. Diese Größenordnung verdeutlicht, wie wichtig die Qualitätskontrolle und Fehlerbehebung in der Softwareentwicklung sind.

Anstatt vereinfachter Schulversionen arbeiten Schülerinnen und Schüler häufig mit denselben Programmen, die auch im professionellen Umfeld verwendet werden. Das Spektrum potenzieller Fehlerquellen wird dadurch erheblich erweitert,

was sowohl von den Lernenden als auch von der Lehrkraft ein hohes Maß an Flexibilität und Kompetenz bei der Fehlersuche erfordert.

Die Ursachen für Fehler bei der Arbeit am Computer sind vielfältig: Anwender machen Flüchtigkeitsfehler. Mit jeder neuen Version eines Programms gibt es Änderungen. Die gleiche Software auf verschiedenen Betriebssystemen verhält sich unterschiedlich. Software kann zudem wenig benutzerfreundlich oder intuitiv gestaltet sein. Auch ein Übermaß an Features kann zu Überforderung führen. Darüber hinaus enthält nahezu jede Software Programmierfehler. In seltenen Fällen können sogar Hardwarefehler auftreten, die oft indirekte Auswirkungen haben und die Ursachensuche erschweren. Unverständliche Fehlermeldungen lassen Anwender ratlos zurück. Bei Schülerinnen und Schülern kommen mangelndes Vorwissen und fehlende Erfahrung hinzu. Malware und andere Übeltäter tun ein Weiteres, Anwender zu verunsichern.

Für den Informatikunterricht stellt sich die Frage, wie Fehler thematisiert werden können. Ziel ist, den Schülerinnen und Schülern zu mehr Selbstständigkeit beim Lösen der Probleme zu verhelfen. Dazu muss zunächst ein Bewusstsein geschaffen werden, dass das Auftreten von Problemen und Fehlern zum Lernprozess gehört und es sich sogar lohnt, ihnen nachzugehen. Grob kann man, angelehnt an Grams (1990), drei Arten von Fehlern unterscheiden:

- *Flüchtigkeitsfehler*, die beispielsweise durch Unachtsamkeit oder fehlende Motivation entstehen: Ein Semikolon in einer Programmiersprache vergessen, eine Variable nicht initialisieren, den falschen Knopf in einer Entwicklungsumgebung drücken und den Code löschen statt speichern. Solche Fehler sind oft leicht zu erkennen, stören aber den Arbeitsablauf.
- *Irrtümer* aufgrund unzureichender Ausbildung oder Begabung: beim Programmieren beispielsweise die Verwendung eines falschen Datentyps, der Einsatz eines ineffizienten Algorithmus oder die Nichtberücksichtigung von Grenzfällen. Bei diesen Fehlern gilt es, das Verständnis und die Kenntnisse der Lernenden zu verbessern.
- *Denkfallen*, die selbst erfahrenen Informatikerinnen passieren. Typische Beispiele sind „Um-Eins-daneben-Fehler“ in Schleifen oder bei Verwendung von Arrays oder Stack-Overflow-Fehler. Diese Fehler sind oft schwer zu erkennen, da sie sich aus allgemein verbreiteten Denkweisen ergeben.

Im Informatikunterricht geht es nicht nur um das Erkennen und Korrigieren von Fehlern, sondern auch darum, Lernende auf typische Fallstricke vorzubereiten und Strategien zur Fehlervermeidung zu vermitteln. Dazu schlagen wir folgende Vorgehensweise vor:

Fehlerakzeptanz schaffen Fehler gehören zum Alltag, ob man sie selbst macht oder über die Folgen eines Fehlers von anderen stolpert. Und Fehler gehören zur Informatik. Man lernt viel, wenn man Fehlern nachgeht und sie nicht einfach beiseite wischt. Fehler können ein Indikator sein, dass man etwas noch nicht richtig verstanden hat. Die Klassifikation von Fehlern kann den Lernenden helfen, ihre eigenen

Fehler in einem größeren Kontext zu sehen. Ausgewählte Beispiele von Fehlern, die durch die Medien gegangen sind, können die Bedeutung des Themas zusätzlich veranschaulichen.

Fehlerursache finden lassen Es braucht Zeit und kostet Nerven, aber es ist wichtig, dass die Lernenden Gelegenheit haben, selbst die Fehlerursache zu finden. Die Lehrkraft sollte dabei helfen, indem sie Fragen formuliert oder Vorschläge für das Vorgehen macht. Sie sollte aber möglichst nicht selbst eingreifen, sondern nur den Prozess der Fehlersuche explizit machen.

Problemlösungsstrategien behandeln Es gibt verschiedene Strategien für die rationale Problemlösung. In der Informatik kommt Teile-und-herrsche oft zum Einsatz: Mit geschickten Fragen werden möglichst viele Ursachen ausgeschlossen und der Fehler Schritt für Schritt eingegrenzt. Das Vorgehen hilft außerdem dabei, Fehler den verschiedenen Schichten eines Computersystems zuzuordnen: Hardware, Peripherie, Betriebssystem, Treiber oder Anwendung, lokaler Computer, Server oder Netzwerk.

Hilfe zur Selbsthilfe Für viele Probleme lassen sich mithilfe lokal oder im Internet zugänglicher Informationsquellen schnell nützliche Lösungen finden. Bei gängigen Problemen reicht es, die Fehlermeldung eins zu eins im Internet zu suchen, da andere Personen dasselbe Problem schon gelöst haben. Bei komplexeren Fragestellungen kann eine präzise Problembeschreibung an einen KI-Assistenten ebenfalls wertvolle Unterstützung bieten.

Im Unterricht kann das Thema auch an einer Auswahl von klassischen Fehlern behandelt werden. Eine gute Quelle dafür ist das Buch *Software-Fehler erkennen und vermeiden* von Duschl (2024). Es beschreibt ein breites Spektrum von Problemen, die durch Fehlmanipulation, Sicherheitslücken, bösartige Angriffe oder fehlerhafte Hard- und Software hervorgerufen wurden. Zahlreiche Beispiele für Computerpannen sind durch die Medien gegangen und gut dokumentiert: Java-Log4j Sicherheitslücke (2021), Airbus-A400M Absturz (2015), Knight-Capital-Börsen Softwarefehler (2012), Explosion der Ariane 5 (1996), Denver-Kofferdebakel (1995), Pentium-Prozessor Divisionsfehler (1994).

Guter Informatikunterricht fördert das Verständnis dafür, dass Fehler alltäglich sind. Lehrerinnen und Lehrer sollten die Fehler nicht selbst beheben, sondern systematisch den Umgang mit Fehlern thematisieren und Strategien und Methoden zur Fehlersuche und -behebung vermitteln.

Beispiel 1: Probleme beim Einloggen auf einer Website

Einloggen auf einer Website scheint eine einfache Aufgabe zu sein – und doch treten immer wieder Probleme auf. Zu den Flüchtigkeitsfehlern zählen Tippfehler beim Eingeben des Passworts, Vertippen beim Eingeben des Benutzernamens oder

Falscheingabe des Passworts aufgrund aktivierter Feststelltaste. Diese Fehler entstehen meist durch Unachtsamkeit. Sie sind schnell erkannt und können durch gezielte Aufmerksamkeit oder technische Hilfsmittel wie Passwortmanager vermieden werden.

Zu den Irrtümern zählt der Versuch, sich auf dem falschen Server oder in der falschen Region eines Dienstes anzumelden (z. B. us.website.com statt eu.website.com), Vergessen, dass das Passwort kürzlich geändert wurde, dass die E-Mail zum Setzen eines neuen Passwortes im Spamordner gelandet ist oder dass eine Zwei-Faktor-Authentifizierung nötig ist. Diese Fehler entstehen durch unzureichendes Wissen.

Zu den Denkfallen zählt die Eingabe der Logindaten auf einer Phishingwebsite, die das echte Design nachahmt, aber nur Passwörter abgreift. Unwahrscheinlich, aber nicht ausgeschlossen: Das betreffende Konto wurde kompromittiert und man kann sich deshalb nicht anmelden (Denkfalle: Das passiert mir doch nicht!). Diese Art von Fehlern ist besonders gefährlich, weil sie tief verwurzelte Annahmen über Sicherheit und Gewohnheiten betreffen. Sie lassen sich nur durch kritisches Hinterfragen der eigenen Vorgehensweise und durch sicherheitsbewusstes Verhalten minimieren.

Beispiel 2: Implizite Annahmen bei Programmieraufgaben

Eine einfache Programmieraufgabe aus Grams (1990) illustriert, wie sehr implizite Annahmen und der eigene „Scheinwerfer der Aufmerksamkeit“ (Metapher für selektive Fokussierung) in die Irre führen. Für die Aufgabe nehmen wir an, dass der <-Operator in unserer Programmiersprache nicht existiert. Daher soll eine Funktion LESS(a, b: REAL): BOOLEAN implementiert werden, die genau dann den Wert TRUE liefert, wenn $a < b$ gilt. Die Lösungen von 19 befragten Entwicklern sahen wie folgt aus:

5 Nennungen:

```
IF (b-a>0) THEN less:=TRUE ELSE less:=FALSE
```

4 Nennungen:

```
IF (b>a) THEN less:=TRUE ELSE less:=FALSE
```

3 Nennungen:

```
IF (NOT(a>=b)) THEN less:=TRUE ELSE less:=FALSE
```

2 Nennungen:

```
IF (a>b) OR (a=b) THEN less:=FALSE ELSE less:=TRUE
```

2 Nennungen:

```
IF (a>=b) THEN less:=FALSE ELSE less:=TRUE
```

2 Nennungen:

```
less:=NOT (a>=b)
```

1 Nennung:

```
less:=b>a
```

Alle genannten Lösungen sind korrekt. Allerdings ist die einfachste Lösung am schwersten zu finden: `less:=b>a`. Dafür scheint es im Wesentlichen zwei Denkfallen zu geben: Für die Entwickler scheinen Boole'sche Ausdrücke an IF-THEN-ELSE-Entscheidungen gebunden zu sein. Die Hälfte der genannten Lösungen lässt zudem die syntaktische Reihenfolge von a und b unverändert. Dabei entsteht gerade durch die Veränderung der Reihenfolge die einfachste Lösung. Hier scheint sich eine implizite Annahme zu verbergen: Die Reihenfolge wird übernommen und nicht hinterfragt.

Das Beispiel zeigt, wie an kleinen, gut ausgewählten Problemstellungen die verschiedenen Arten von Fehlern im Unterricht thematisiert werden können. Oft sind die vermeintlich trivialen Beispiele die besten „Augenöffner“.

Beispiel 3: Denkfalle „binäre Suche“

Die binäre Suche ist ein effizienter Algorithmus, mit dem zum Beispiel eine bestimmte Zahl in einer sortierten Liste von Zahlen gefunden werden kann. Die Umsetzung in ein kurzes Programm scheint auf der Hand zu liegen und könnte als Pseudocode wie folgt aussehen:

```
def binaere_suche(array,zahl):
    links=0, rechts=len(array)-1
    while links<rechts:
        mitte=(links+rechts)//2
        if array[mitte]==zahl:
            return mitte #Index der gesuchten Zahl
        elif array[mitte]<zahl:
            links=mitte+1
        else:
            rechts=mitte-1
    return -1 #Zahl nicht gefunden
```

Die Aufgabe hat aber ihre Tücken. Bentley (1986) beschreibt in seinem Buch *Programming Pearls*, wie auch viele erfahrene Programmierer an dieser Aufgabe scheitern. Der erste Fehler betrifft Grenzfälle, zum Beispiel wenn die gesuchte Zahl

an letzter Stelle im Array steht. Dieser Fehler kann als Irrtum eingestuft werden. Die eigentliche Denkfalle liegt in der Berechnung

```
mitte=(links+rechts)//2.
```

Die Berechnung kann zu einem Overflow-Fehler führen. In den meisten Fällen wird dieser Fehler in der Praxis keine Auswirkungen haben. Aber er ließe sich sehr einfach vermeiden mit der Berechnung

```
mitte=links+(rechts-links)//2.
```

Literatur

- Bentley, J. L. (1986). *Programming pearls*. Addison-Wesley.
Duschl, D. (2024). *Software-Fehler erkennen und vermeiden*. Springer Vieweg.
Grams, T. (1990). *Denkfallen und Programmierfehler*. Springer.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Einstieg: K. vermittelt im Unterricht umfassend die Funktionsweise von Netzwerken und des Internets. Dabei erklärt er unter anderem den Aufbau von IP-Adressen, den Einsatz von Routern sowie die Funktionen von DNS- und Webservern. Theorie allein genügt ihm nicht – seine Lernenden sollen auch praktisch arbeiten. Im Computerlabor experimentieren sie mit Hardware und für die Prüfung nutzen sie eine Simulationsumgebung. Die Aufgabe: Ein kleines Netzwerk aus Computer, Drucker, Internetanbindung, DNS-Server und Webserver einrichten. Eine Woche vor der Prüfung testen die Lernenden die lokal auf ihren Rechnern installierte Simulationsumgebung. Dennoch scheitert die Prüfung: Bei einigen läuft die Simulation nicht, zwei Lernende löschen versehentlich mitten in der Prüfung ihren Arbeitsstand, und selbst die beste Schülerin hadert mit der Benutzeroberfläche der Simulationsumgebung. Von einer fairen Prüfung keine Rede. Auf Empfehlung einer Kollegin setzt K. beim nächsten Mal eine Onlineumgebung ein: keine lokale Installation, intuitive Bedienung, automatische Speicherung. Perfekt! Doch schon zu Beginn der Prüfung gibt es Probleme, einige Lernende können nicht auf die Onlineumgebung zugreifen, da der Anbieter den gleichzeitigen Zugriff aus einem Netzwerk beschränkt.

Der Informatikunterricht vermittelt ein breites Spektrum an Kompetenzen – von theoretischen Grundlagen über den Umgang mit digitalen Werkzeugen bis hin zur Entwicklung eigener Produkte. Ein Prüfungsformat allein kann diese Leistungen nicht vollständig messen. Außerdem können bei Prüfungsformaten, die auf digitale Anwendungen setzen, technische Störungen auftreten.

Viele Themen des Informatikunterrichts lassen sich mit den gleichen Methoden prüfen wie in anderen Fächern. Bewusst gehen wir deshalb hier nicht auf die Kriterien für gute Prüfungen ein, wie zum Beispiel klar formulierte Lernziele und trans-

parente Bewertungskriterien. Auch der Einsatz digitaler Prüfungsumgebungen oder die Frage, inwieweit eigene Unterlagen, Informationen aus dem Internet oder Anwendungen der Künstlichen Intelligenz in einer Prüfung genutzt werden dürfen oder sollen, ist nicht nur im Informatikunterricht ein Thema.

Eine Besonderheit des Informatikunterrichts ist hingegen, dass die Lernenden konkrete Produkte erstellen. Dabei kommen oft professionelle und zum Teil komplexe Werkzeuge zum Einsatz. Wenn technische Probleme wie Systemabstürze, ein überlastetes Schulnetz oder eine unterbrochene Internetverbindung auftreten, können die Lernenden ihr Können nicht richtig zeigen. Auch Unsicherheiten bei der Bedienung von Werkzeugen, Syntaxfehler beim Schreiben von Programmen oder versehentliches Löschen oder Überschreiben des Arbeitsstandes können dazu beitragen, dass Lernende unter Zeitdruck geraten und damit die eigentliche Leistungsbewertung verfälscht wird.

Die beschriebenen Probleme lassen sich weitgehend vermeiden, wenn man reine Unpluggedprüfungen durchführt. Damit würden aber Kompetenzen der Leistungsmessung entzogen, die den Informatikunterricht besonders auszeichnen, zum Beispiel der Umgang mit Werkzeugen und Fehlern. Im Folgenden formulieren wir Empfehlungen, wie sich Schwierigkeiten bei Prüfungen vermeiden oder zumindest reduzieren lassen.

Empfehlung 1: Technikcheck vor Prüfungen am Computer

Der Großteil der Prüfungen – ob Klausuren, Klassenarbeiten und Lernkontrollen – findet in der Regel innerhalb von einer bis zwei Unterrichtsstunden statt. Sobald solche Leistungserhebungen jedoch in praktischer Form stattfinden, können allerlei Probleme auftreten. Während man im regulären Unterricht bei technischen Problemen meist auf eine der folgenden Stunden ausweichen kann, ist dies bei Prüfungen nur eingeschränkt möglich: Die Lernenden haben sich – hoffentlich – intensiv auf die Leistungskontrolle vorbereitet. Wenn eine Prüfung aufgrund unerwarteter technischer Probleme abgebrochen und neu angesetzt werden muss, kann dies erheblichen Frust bei den Schülerinnen und Schülern auslösen. Zudem muss die Lehrkraft eine neue Prüfung zusammenstellen. Um derartigen Schwierigkeiten vorzubeugen, ist eine sorgfältige Vorbereitung unerlässlich.

Findet der Unterricht in einem Informatikraum statt, kann die Lehrkraft meist von einer einheitlichen Infrastruktur ausgehen. Oft genügt es, alle Computer über ein zentrales Image auf denselben Stand zu bringen und anschließend die Prüfungsumgebung auf einem Gerät gründlich zu testen. Dennoch besteht die Gefahr, dass Lernende anderer Klassen zuvor Änderungen an den Einstellungen vorgenommen haben. Nutzen die Lernenden hingegen persönliche mobile Endgeräte (z. B. Notebooks), kann die Lehrkraft im Vorfeld nicht sicherstellen, dass alle Geräte einwandfrei funktionieren. Dies gilt besonders dann, wenn Schülerinnen und Schüler ihre privaten Rechner mitbringen, da sich deren Ausstattung stark unterscheidet. Andererseits kann die Arbeit an persönlichen Geräten die Leistungsbeurteilung fairer gestalten, weil die Lernenden in ihrer vertrauten Arbeitsumgebung bleiben. Unabhängig von der Infrastruktur sollten immer ein oder zwei Reserverechner zur Verfügung stehen.

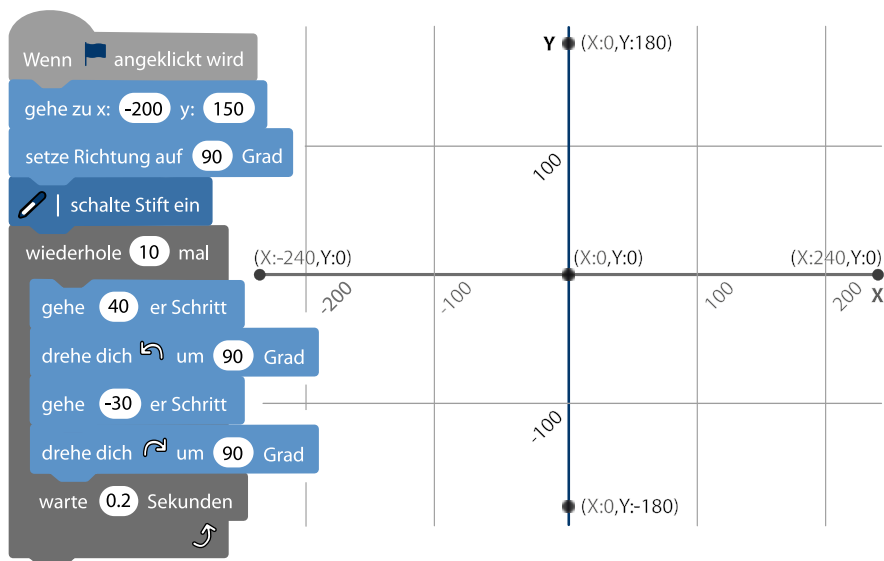


Abb. 26.1 Scratch-Programm „Treppe“

Empfehlung 2: Kompetenzen „unplugged“ prüfen

Der Verzicht auf Prüfungen am Rechner schließt technische Probleme sowie falsche Handhabung weitgehend aus. Oft trägt diese Herangehensweise sogar dazu bei, dass die tatsächlich angestrebten Kompetenzen geprüft werden. Zudem stellt sich das Problem nicht, die Nutzung von Internetquellen, Kommunikationskanälen oder KI-Assistenten zu unterbinden, wenn sie in dieser Prüfungssituation nicht erlaubt sind.

Beispiel 1: Lesen statt Schreiben

Anstatt von den Lernenden zu verlangen, sofort eigene Programme zu entwickeln, ist es oft angebracht – etwa in der Primarstufe –, den Schwerpunkt auf das Verstehen und Nachvollziehen bestehender Programme zu legen. Das Scratch-Programm in Abb. 26.1, das eine Treppe im Koordinatensystem zeichnet, illustriert diesen Ansatz. Für die Prüfung werden der Code und das zugehörige Ausgabefeld ausgedruckt. Die Schülerinnen und Schüler übertragen händisch die Programmausgabe in das Koordinatensystem. So wird sichergestellt, dass die Leistungsmessung das Verständnis des Programms und der zugrunde liegenden Konzepte widerspiegelt, nicht in erster Linie den Umgang mit technischen Details.

Das Konzept „Lesen statt Schreiben“ eignet sich nicht nur fürs Programmieren. So kann etwa beim Lesen eines einfachen HTML- und CSS-Codes die zu erwartende Darstellung einer Webseite auf Papier skizziert werden. Eine schematische Darstellung eines Netzwerks hilft, mögliche Verbindungsfehler zu erkennen. Ebenso ermöglicht die Untersuchung vorgegebener regulärer Ausdrücke, deren Funktion zu analysieren, ohne dass typische Eingabefehler oder Syntaxprobleme ablenken.

Beispiel 2: Zusammensetzen statt Schreiben

Ein vergleichbarer Ansatz zu „Lesen statt Schreiben“ zeigt sich in den sogenannten Parsons-Problemen: Hier bauen die Lernenden Programme aus vorgegebenen Codefragmenten zusammen, anstatt sie von Grund auf selbst zu erstellen. So fließen in die Bewertung nicht primär Schreibfehler ein, sondern die Fähigkeit, die Struktur des Programms zu erkennen und deren Logik zu verstehen. Der Schwierigkeitsgrad kann flexibel angepasst werden, etwa indem zusätzliche, nicht benötigte Codefragmente angeboten werden. So wird die Leistungsbewertung gezielt auf die Fähigkeit ausgerichtet, relevante von irrelevanten oder unpassenden Bausteinen zu unterscheiden.

Beispiel 3: Anpassen statt Schreiben

Im Bereich des Datenbankentwurfs bietet es sich für die Leistungsmessung an, die Lernenden zunächst mit einem Ausschnitt aus einer bestehenden Datenbank zu konfrontieren. Sie lesen sich in die vorhandene Struktur ein, um zu verstehen, warum bestimmte Felder existieren und wie diese miteinander in Beziehung stehen. Anschließend sollen sie die Datenbank schrittweise anpassen und erweitern. Diese Vorgehensweise – „zuerst lesen, dann anpassen, dann schreiben“ – richtet den Fokus der Beurteilung auf das analytische Verständnis sowie die zielgerichtete Weiterentwicklung einer funktionierenden Struktur, bevor eigene Datenbankmodelle erstellt werden müssen.

Empfehlung 3: Praktische Fertigkeiten gehören dazu

Es mag für Lehrkräfte verlockend erscheinen, Prüfungen mit Praxisanteil am Computer ganz zu vermeiden. Insbesondere in höheren Schulstufen sollten Prüfungsformate jedoch nicht nur theoretisches Wissen messen, sondern auch praktische Fähigkeiten wie Systementwurf, den effektiven Umgang mit Entwicklungsumgebungen, Debugging oder Versionskontrolle einbeziehen. Dabei sollte auch in die Bewertung einfließen, auf welche Weise die Lernenden mit Fehlern umgehen und eigenständig Lösungen entwickeln. Klassische 45-Minuten-Prüfungen sind dafür meist ungeeignet. Stattdessen ist es ratsam, großzügig Zeit für Prüfungen mit einem hohen Praxisanteil einzuplanen, damit die Lernenden ihre praktischen Fertigkeiten wirklich unter Beweis stellen können.

Empfehlung 4: Bewertung von Informatikprojekten

Die Entwicklung eigener kleiner Produkte oder Systeme lässt sich kaum in einer herkömmlichen Prüfungsform über eine oder zwei Stunden realisieren. Daher empfiehlt es sich, solche Vorhaben in Form von Informatikprojekten umzusetzen, die ohne strikten Zeitrahmen und in Teams durchgeführt werden. So wird der Zeitdruck gemindert und durch die Zusammenarbeit im Team lassen sich Fehler in der Regel schneller beheben.

Bei Projekten stellt sich allerdings die Frage, wie – neben der Leistung des gesamten Projektteams – auch die individuellen Beiträge einzelner Lernender möglichst fair bewertet werden können. Einzelbewertungen widersprechen dem kooperativen Charakter von Gruppenprojekten. Zwar ist die Bewertung von Gruppen-

arbeiten kein informatikspezifisches Thema, doch kommt diese Unterrichtsform weitaus häufiger zum Einsatz als in vielen anderen Fächern. Deshalb empfehlen wir, dass sich Informatiklehrkräfte besonders intensiv mit der Beurteilung von Projekt- und Gruppenarbeiten auseinandersetzen. Zur fairen Bewertung individueller Leistungen in Gruppenprojekten empfehlen sich verschiedene Methoden: Versionskontrollsysteme zur Nachverfolgung von Beiträgen, Peerbewertungen, regelmäßige kurze Standortgespräche sowie individuelle Portfolios oder Reflexionen. Auch Codereviews, Projektmanagement-Tools und Einzelpäsentationen oder Interviews können eingesetzt werden. Bewertet werden sollten sowohl das entwickelte Produkt als auch der Entwicklungsprozess, ebenso Soft Skills wie Teamarbeit, Kommunikation und Problemlösung.

Neben der Herausforderung, individuelle Leistungen in Gruppenprojekten angemessen zu bewerten, stellt sich die Frage, inwieweit KI-Assistenten bei der Erstellung der Arbeiten zum Einsatz kamen. Gemeint ist dabei nicht die Nutzung von KI als Ideengeber oder Unterstützung bei einzelnen Arbeitsschritten, sondern dass wesentliche Teile eines Projekts nahezu vollständig durch KI erstellt wurden. Die wirksamste Prävention gegen KI-generierte Projektarbeiten sind Projekte mit einer offenen Ausgangssituation. Solche Projekte geben das Endprodukt oder den Lösungsweg nicht von vornherein vor. Ein gutes Projekt besteht nicht darin, vorhandenes Wissen lediglich zu sammeln und zu ordnen. Vielmehr erlaubt eine durchdachte Projektidee den Lernenden, sich mit einem realen, bedeutsamen oder persönlich interessanten Problem auseinanderzusetzen und das Projekt zu ihrer eigenen Sache zu machen. Daneben sollte zu Beginn eines Projekts transparent gemacht werden, in welchem Rahmen der Einsatz von KI erlaubt oder erwünscht ist und wo die Grenze zur unerlaubten Fremderledigung verläuft. So wird die KI-Nutzung nicht tabuisiert, sondern als Teil des Lernprozesses thematisiert und reflektiert.

Eine Kombination aus klassischen Prüfungen ohne Rechner, Prüfungen am Rechner und projektbasierten Ansätzen ermöglicht eine faire und realitätsnahe Bewertung der Kompetenzen im Informatikunterricht. Lehrkräfte sollten Prüfungsformate bewusst wählen und dabei sowohl technische Rahmenbedingungen als auch didaktische Zielsetzungen berücksichtigen. Die Bewertung sollte nicht nur fertige Produkte, sondern auch den Lösungsprozess, Fehlermanagement und Teamarbeit miteinbeziehen.

Open Access Dieses Kapitel wird unter der Creative Commons Namensnennung - Nicht kommerziell - Keine Bearbeitung 4.0 International Lizenz (<http://creativecommons.org/licenses/by-nc-nd/4.0/deed.de>) veröffentlicht, welche die nicht-kommerzielle Nutzung, Vervielfältigung, Verbreitung und Wiedergabe in jeglichem Medium und Format erlaubt, sofern Sie den/die ursprünglichen Autor(en) und die Quelle ordnungsgemäß nennen, einen Link zur Creative Commons Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Die Lizenz gibt Ihnen nicht das Recht, bearbeitete oder sonst wie umgestaltete Fassungen dieses Werkes zu verbreiten oder öffentlich wiederzugeben.

Die in diesem Kapitel enthaltenen Bilder und sonstiges Drittmaterial unterliegen ebenfalls der genannten Creative Commons Lizenz, sofern sich aus der Abbildungslegende nichts anderes ergibt. Sofern das betreffende Material nicht unter der genannten Creative Commons Lizenz steht und die betreffende Handlung nicht nach gesetzlichen Vorschriften erlaubt ist, ist auch für die oben aufgeführten nicht-kommerziellen Weiterverwendungen des Materials die Einwilligung des jeweiligen Rechteinhabers einzuholen.



Stichwortverzeichnis

A

Abstraktion, 14, 15, 32, 105, 135
Advance Organizer, 129
Aktualität, 17, 123
Algorithmus, 16, 44, 76, 83, 99, 138, 146, 171
Animation, 144
Anwenderschulung, 4, 6
Ausubel, D., 129

B

Barrierefreiheit, 107, 159
Bell, T., 28
Bentley, J. L., 171
Benutzerfreundlichkeit, 38
Berechnung, 26
Betriebssystem, 46, 81, 146, 159
Big Ideas in Computer Science, 28
Bildungsstandards, 27
Black Box, 14
Bloom, B., 57, 150
Bottom-up, 60, 150
Bruner, J., 22, 32, 136

C

Charakteristik, 13
Chemieunterricht, 4
Cloud-Computing, 26
Copy and Paste, 45

D

Dagstuhl-Dreieck, 6, 7
Data Science, 116, 147
Datenschutz, 5
Datensicherheit, 5
Datenstruktur, 16
Denkstil, 69

Denning, P., 26

Design, 26, 41
Digitales Werkzeug, 3, 157, 162, 173
Digitalität, 5, 14–15
Duschl, D., 169
Dynamik, 17, 44

E

Eigenverantwortung, 83
E-Learning, 5
Enaktive Darstellung, 32
Entdeckendes Lernen, 70, 105
Evaluation, 26, 41

F

Fachsystematik, 22, 26
Fehler, 16, 17, 87, 167
Formale Antwortstruktur, 74
Fortbildung, 17, 63
Frey, K., 74, 112
Fundamentale Ideen, 22, 32, 56

G

Genderaspekt, 53
Git, 47
Grams, T., 168
Great Principles of Computing,
26, 39
Grell, J., 69
Gruppenarbeit, 64, 80, 176–177

H

Handlungsorientierung, 43
Heterogenität, 51, 53, 70, 161
Horizontalkriterium, 32, 122

I

Individualisierung, 70, 71, 73, 106, 158
Informatikschulung, 63
Informatiksystem, 5, 13, 51, 79
Infrastruktur, 52, 64, 70, 158
Ingenieurwissenschaft, 16
Inhaltsbereich, 27
Interdisziplinarität, 123

J

Jäckel, S., 122

K

Keller, F. S., 102
KI-Assistent, 57, 89, 108, 130, 158, 164, 169, 177
Kommunikation, 5, 26, 39
Kompetenz, 55, 57
Kompetenzmodell, 25, 27
Komplexität, 16, 87, 111
Kontext
 historischer, 123
Konzeptwissen, 22, 43, 57
Kooperation, 5
Koordination, 26, 40

L

Lebenswelt, 22, 31, 36, 121, 123
Lehrplan, 17, 21, 25, 28, 63
Leitidee, 56
Leitprogramm, 70
Lernaufgabe, 70
Lernsoftware, 3, 4
Lernziel, 22, 55, 57, 64, 173
Lister, R., 150
Logik, 31
LUKAS-Modell, 103

M

Malerberuf, 163
Mastery Learning, 101
Mathematikunterricht, 4, 52
Mayer, R. E., 151
Medienbildung, 5
Methodenkompetenz, 70, 83, 105
Meyer, H., 69
Modell, 15, 16, 111, 135
Motivation, 16, 121

N

Nachhaltigkeit, 84, 159
Näf, M., 117
Netzwerk, 59, 97, 131, 146

P

Pair Learning, 70
Pair Programming, 87
Parsons-Problem, 176
Partnerarbeit, 88
Piaget, J., 52
Polymorphismus, 140
PRIMM-Modell, 52, 150
Problemlösung, 15, 107, 150, 161, 165, 169
Produktwissen, 22, 43, 57, 79
Programmieren, 60, 76, 90, 107, 150
Projektunterricht, 71, 111, 126, 176
Prozessbereich, 27
Prüfung, 173
Puzzlemethode, 70

Q

QR-Code, 74

R

Rastergrafik, 8, 34
Realisierbarkeit, 16–17
Relevanz, 31, 56, 63, 121, 130
Repräsentation
 enaktive, 136
 ikonische, 136
 symbolische, 136
Repräsentationskriterium, 32
Repräsentationstrias, 135
Rollenspiel, 70

S

Schulmeister, R., 144
Schwill, A., 22, 32
Selbsthilfe, 73, 165, 169
Sicherheit, 38
Simulation, 31, 98, 144, 147
Sinnkriterium, 32, 122
SoekiaGPT, 108
Speicherung, 26
Sprachmodell, 9, 89
Standardwerkzeug, 144
Supportwarteschlange, 164
Systemdenken, 15–16

TTeile-und-herrsche, [16](#), [37](#), [137](#), [169](#)Top-down, [60](#), [150](#)**U**Unterrichtsmethode, [69](#), [70](#)**V**Vektorgrafik, [8](#), [34](#)Vertikalkriterium, [32](#)Visualisierung, [16](#), [116](#),
[130](#), [143](#)Vorkenntnisse, [51](#), [64](#), [79](#),
[101](#), [130](#)**W**Wechselwirkung, [15](#)Weinert, F .E., [57](#)Wiechmann, J., [69](#)**Z**Zeitkriterium, [32](#), [122](#)Zendler, A., [69](#)Zielgruppe, [21](#), [22](#), [39](#), [63](#)